

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Ranjan, Chitta. *Understanding Deep Learning: Application in Rare Event Prediction*.

<https://www.understandingdeeplearning.com/>.

- **Required pages:** Chapter 7
- **Estimated reading time:** 2 hours

2. Plaut, Elad. "From Principal Subspaces to Principal Components with Linear Autoencoders." *arXiv preprint*, 2018,

<https://arxiv.org/pdf/1804.10253.pdf>.

- **Required pages:** Entire article
- **Estimated reading time:** 45 minutes

Module 6: Lesson 1

Introduction to autoencoders



Outline

- ▶ Introduction to autoencoders
- ▶ Autoencoders as neural networks
- ▶ Linear autoencoders as principal component analysis (PCA)

Autoencoders

Autoencoders are a type of unsupervised ML method that is designed as a reconstruction problem.

Training autoencoders involves the minimization error from reconstructing its inputs $\mathbf{x}$ in two steps

$$\underbrace{\mathbf{x} \longrightarrow f(\mathbf{x}) \longrightarrow \mathbf{z}}_{\text{Encoding}} \underbrace{\longrightarrow g(\mathbf{z}) \longrightarrow \hat{\mathbf{x}}}_{\text{Decoding}}$$

The encoding function f transforms the input $\mathbf{x}$ into encodings $\mathbf{z}$ (latent representations of the data).

The decoding function g transforms the encodings $\mathbf{z}$ back to a reconstruction of the input $\hat{\mathbf{x}}$.

At first, the problem of reconstructing an input may be trivial. But building an autoencoder model may be useful for several tasks, such as:

- ▶ Dimensionality reduction.
- ▶ Denoising.
- ▶ Pre-processing inputs before classification (“whitening”).
- ▶ Outlier detection.

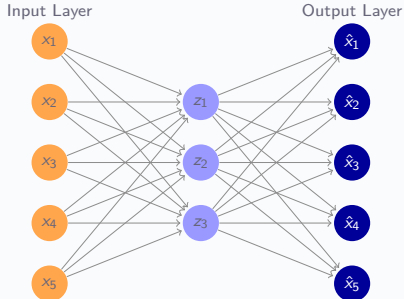
Autoencoders as neural networks

Autoencoders can be represented as a neural network with one or many fully connected hidden layers.

- ▶ The output layer is identical to the input layer, $\mathbf{x}$.
- ▶ Training the neural network implies finding an identity mapping between inputs and outputs.

The hidden activations are called encodings, latent features, or “factors,” $\mathbf{z}$.

- ▶ The encodings capture the dependencies between the different observed variables included in $\mathbf{x}$.



Autoencoders as neural networks

Suppose that we want to encode our feature vector $\mathbf{x}$, with dimension $n \times 1$, into a $k \times 1$ vector of encodings $\mathbf{z}$.

Suppose also that we have p observations of $\mathbf{x}$, say returns of n different stocks over p periods, or information of n features from p individuals.

We stack the information in an $n \times p$ matrix X .

We can represent a single-layer autoencoder as follows, where f and g are activation functions:

$$X \longrightarrow f \left(W_1 X + \mathbf{b}_1 \iota_p^T \right) \longrightarrow Z \longrightarrow g \left(W_2 Z + \mathbf{b}_2 \iota_p^T \right) \longrightarrow \hat{X}$$

where:

- ▶ Encoding weights: W_1 , $k \times n$ matrix. Decoding weights: W_2 , $n \times k$ matrix.
- ▶ Bias terms: $\mathbf{b}_1$, $k \times 1$ vector, and $\mathbf{b}_2$, $n \times 1$ vector.
- ▶ Encodings: Z , $k \times p$ matrix.
- ▶ ι_p , $p \times 1$ vector of ones.
- ▶ $\hat{X}$, $n \times p$ reconstructed input.

Autoencoders as neural networks

Given a choice of activation functions, f and g , and encoding dimension, k , we can train the autoencoder to minimize the reconstruction error computed, say, with the MSE:

$$\frac{1}{np} \left\| X - \hat{X} \right\|_F^2$$

where F denotes the Frobenius norm.

As in any other neural network, we can add regularization terms that penalize the weights and the encodings.

Depending on the number of encodings k , we can differentiate among:

- ▶ Undercomplete autoencoders ($n > k$). They gather the most salient features of the data by learning small representations (a “bottleneck”).
- ▶ Overcomplete autoencoders ($n \leq k$). They necessarily need regularization to avoid learning an identity transformation.

We can also increase the depth of the network with additional hidden layers on both sides of the encoding layer.

Linear autoencoders

Suppose you have an autoencoder model with one hidden layer, whose activation functions f and g are linear. The autoencoder can be represented as:

$$X \longrightarrow W_1 X + \mathbf{b}_1 \iota_p^T \longrightarrow Z \longrightarrow W_2 Z + \mathbf{b}_2 \iota_p^T \longrightarrow \hat{X}$$

Then, assuming an MSE loss, training this autoencoder involves the choice of weights and biases that solve:

$$\min_{W_1, W_2, b_1, b_2} \left\| X - W_2 \left(W_1 X + \mathbf{b}_1 \iota_p^T \right) - \mathbf{b}_2 \iota_p^T \right\|_F^2$$

which can be reduced to choosing W_2 to minimize $\left\| X_0 - W_2 (W_2^T W_2)^{-1} W_2^T X_0 \right\|_F^2$.

- ▶ $X_0 = X - X \iota_p / p$ is the matrix of centered observations, i.e., X minus the mean of each feature.
- ▶ W_2 is a solution to the previous problem if its columns form a basis that spans the same subspace as the principal eigenvectors of the variance-covariance matrix of X .
- ▶ Training a single-layer linear autoencoder is roughly equivalent to performing principal component analysis on X .

Principal Components Analysis (PCA)

Principal component analysis consists of a linear transformation of the data to a new coordinate system that explains its variation.

PCA finds a low-dimensional (linear) representation of a data set that contains the maximum level of variation.

For instance, the first principal component of our data X is the linear combination of the uncentered features

$$Z_1 = w_1^T X_0$$

which is the encoding with the largest variance, i.e., $w_1^T X_0 X_0^T w_1$ is maximized.

$w_1 = (w_{11}, w_{21}, \dots, w_{n1})^T$ is a normalized $n \times 1$ vector, such that $\sum_i^n w_{i1}^2 = 1$.

- ▶ The coefficients in w_1 are the *loadings* of the first principal component.
- ▶ We normalize the loadings, otherwise setting these elements to be arbitrarily large could result in an arbitrarily large variance.

PCA: Second and further principal components

The second principal component is the linear combination of X_0 with maximal variance out of all linear combinations that are uncorrelated with Z_1 .

Constraining Z_2 to be uncorrelated with Z_1 is equivalent to constraining the direction w_2 to be orthogonal (perpendicular) to the direction w_1 , $w_1^T w_2 = 0$.

- We also normalize the loadings to have unit norm $\sum_i^n w_{i2}^2 = 1$.

We can proceed in the same manner until reaching the n th principal component, such that the elements $(Z_1, \dots, Z_k)$ explain all the variance of X .

The scores and loadings from PCA are unique up to a sign flip: the variance of Z is the same as that of $-Z$.

PCA and linear autoencoders

Let Σ denote the variance-covariance matrix of X , $\Sigma = \frac{X_0^T X_0}{p-1}$. We can perform the following decomposition:

$$\Sigma = P \Lambda P^T$$

Λ is the diagonal matrix of eigenvalues, sorted from highest to lowest, and P is a matrix whose columns are orthonormal eigenvectors.

- ▶ The eigenvectors are called the *principal directions* of the data.
- ▶ PCA generates uncorrelated encodings equal to $Z_k = P_k^T X_0$, for some choice of k eigenvectors P_k .

P_k is also a solution to the following problem with orthonormality constraints, which is similar to the linear autoencoder ($I_{k \times k}$ is the $k \times k$ identity matrix):

$$\min_{W \in \mathbb{R}^{n \times k}} \|X_0 - WW^T X_0\|_F^2 \text{ s.t. } W^T W = I_{k \times k}$$

The solution P_k is not unique, $W = QP_k$ is also a solution for any orthogonal Q .

PCA and the minimum reconstruction error

Let λ_i denote the i th eigenvalue of Σ , and let w_i denote its associated eigenvector.

- ▶ The value of each λ_i corresponds to the variance of X explained in the direction of eigenvector w_i .
- ▶ As we include more principal components, λ_i 's, we explain more of the variation in X .

If $k \leq n$, the optimal reconstruction error of the single-layer linear autoencoder is bounded below by the sum of the eigenvalues beyond the k th:

$$\min_{W_1, W_2, b_1, b_2} \|X - \hat{X}\|_F^2 = \min_{W_1, W_2, b_1, b_2} \|X - W_2 (W_1 X + b_1 \ell_p^T) - b_2 \ell_p^T\|_F^2 = \sum_{i=k+1}^n \lambda_i$$

If Σ has rank k , we would have that $\lambda_{k+1} = \lambda_n = 0$ and the minimum reconstruction error would be zero.

If Σ has full rank, i.e., there are no perfect linear dependencies between any combination of features, the minimum reconstruction error would be zero only in the case where we chose $k = n$.

PCA and autoencoders

A linear autoencoder applies PCA to the input data by generating a projection of the data into a low-dimensional space. However:

- ▶ The encodings of the bottleneck are correlated and not sorted.
- ▶ The solutions to each choice of k are not nested, we need to train for each k .

When building and training an autoencoder, PCA serves as a useful benchmark.

- ▶ Autoencoders are more flexible, they may capture richer features through deep models and non-linearities.
- ▶ We can use loss functions that are not restricted to MSE.

Besides, through autoencoders, we can easily employ regularization techniques to avoid overfitting.

- ▶ PCA may detect noise as a latent encoding that is irrelevant for generalization out of sample.

Summary of Lesson 1

In Lesson 1, we have:

- ▶ Described the role and design of autoencoders.
- ▶ Represented autoencoders as neural networks.
- ▶ Described PCA and its relationship with autoencoders.

⇒ **References for this lesson:**

Chapter 7 in [Ranjan, C. *Understanding Deep Learning*](#).

[Plaut, Elad. "From Principal Subspaces to Principal Components with Linear Autoencoders." *arXiv*, 2018](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement an autoencoder to compress the information contained in stock prices.

See you there!

DEEP LEARNING FOR FINANCE

MODULE 6 | LESSON 2

APPLICATION OF AUTOENCODERS: EXTRACTING FEATURES FROM STOCK RETURNS

Reading Time	50 minutes
Prior Knowledge	Neural Networks, machine learning
Keywords	Autoencoder, Principal Components, Factors

1. INTRODUCTION

In this notebook, we will describe how to build a basic autoencoder model with TensorFlow using stock returns data. The exercise here will illustrate the tools that we described in Lesson 1 and the typical outputs generated by autoencoders to understand the variation that drives security prices. In Lesson 4, we will extend the analysis allowing an autoencoder model to become a tool for portfolio choice.

Let's first load some libraries needed in this notebook altogether with the data. We are going to analyze 20 tech stocks and their evolution over the 2017-2022 period. Using autoencoders, we will extract the common encodings, or latent "factors", that determine the variation of stock returns and how each stock is exposed to those factors.

```
In [1]: import numpy as np
import pandas as pd
```

```

In [2]: # List of tickers (same order)
tickers = [
    "AAPL", "MSFT", "GOOG", "AMZN", "TSLA",
    "TCEHY", "NVDA", "TSM", "META", "BABA",
    "ASML", "AVGO", "ORCL", "CSCO", "ADBE",
    "CRM", "NFLX", "TXN", "QCOM", "SAP"
]

# Read CSV directly with Date as index
print("Loading data from CSV...")
data = pd.read_csv('adjusted_prices.csv', index_col='Date',
parse_dates=True)

print(f"Data loaded: {data.shape}")
print(f"Complete range: {data.index[0].date()} to
{data.index[-1].date()}")

# Calculate daily returns (pct_change) for all tickers
print("Calculating daily returns...")
returns_data = data[tickers].pct_change()

# Remove first row (NaN from pct_change)
df = returns_data.dropna()

df.head()

```

```

Loading data from CSV...
Data loaded: (1509, 20)
Complete range: 2019-01-02 to 2024-12-30
Calculating daily returns...

```

Out[2]:

	A A P L	M S F T	G O O G	A M Z N	T S L A	T C E H Y	N V D A	T S M	M E T A	B A B A	A S M L	A V G O	O R C L	C S C O	A D B E	C R M	N F L X	T X N	Q C O M	S A P
D a t e																				
2019-01-03 00:00:00	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	0.0	-0.0	-0.0	-0.0
2019-01-03 00:00:00	9.9	3.6	2.8	2.5	3.1	2.8	6.0	5.9	2.9	4.4	5.5	8.8	0.9	3.6	3.9	3.7	1.3	5.8	2.9	3.7
2019-01-03 00:00:00	6.0	7.8	4.8	2.4	4.7	3.4	4.1	1.4	0.3	6.2	1.5	9.5	7.3	6.6	4.9	9.9	2.2	9.7	6.1	0.8
2019-01-03 00:00:00	7.7	8.8	4.4	1.2	2.7	7.7	7.7	5.5	9.9	3.3	0.0	1.1	0.0	8.8	8.8	3.3	6.6	3.3	7.7	8.8
2019-01-03 00:00:00	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2019-01-03 00:00:00	4.4	4.4	5.5	5.5	5.5	5.5	6.6	1.7	4.7	7.0	4.3	0.9	4.3	4.5	4.8	5.7	7.7	4.5	1.6	3.4
2019-01-03 00:00:00	2.6	5.5	7.7	0.0	6.6	0.0	0.0	7.7	1.0	0.2	8.8	1.1	0.0	6.6	9.9	2.2	2.2	1.1	3.5	6.6
2019-01-03 00:00:00	8.9	0.9	8.6	6.4	9.7	8.8	6.8	5.3	6.8	1.1	9.9	9.9	0.0	4.5	3.2	7.6	3.4	3.0	5.8	0.0
2019-01-03 00:00:00	-0.0	0.0	-0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	-0.0	0.0
2019-01-03 00:00:00	0.0	0.0	0.0	3.3	5.5	0.0	5.5	0.0	0.0	2.2	2.2	0.2	1.0	0.6	3.3	0.3	5.9	1.7	0.2	0.0
2019-01-03 00:00:00	2.2	2.2	1.3	4.3	3.3	8.9	2.9	4.3	7.7	9.9	8.8	3.3	8.8	7.7	5.5	8.8	7.7	2.2	8.8	8.8
2019-01-03 00:00:00	2.2	7.5	6.7	5.3	6.1	9.3	4.0	3.5	2.5	7.1	4.7	6.6	4.2	5.7	7.3	7.8	1.7	2.3	2.7	2.4
2019-01-03 00:00:00	6.6	5.5	7.7	3.3	1.1	3.3	0.0	5.5	5.5	1.1	7.7	6.6	2.2	7.7	3.3	8.8	7.7	3.3	7.7	4.4

	A A P L	M S F T	G O O G	A M Z N	T S L A	T C E H Y	N V D A	T S M	M E T A	B A B A	A S M L	A V G O	O R C L	C S C O	A D B E	C R M	N F L X	T X N	Q C O M	S A P
D a t e																				
2019-08-08 00:00:00	0.	0.	0.	0.	0.	-0	-0	-0	0.	0.	-0	-0	0.	0.	0.	0.	0.	-0	-0	0.
	0	0	0	0	0	.0	.0	.0	0	0	.0	.0	0	0	0	0	0	.0	.0	0
	1	1	0	0	1	0	0	2	0	3	2	0	0	0	1	2	1	0	0	1
	9-	9	7	7	6	1	3	4	8	2	5	7	8	9	8	4	4	5	1	8
	0	0	2	3	6	1	4	8	2	4	7	3	0	0	1	9	6	6	2	6
	1-	6	5	8	1	6	2	9	3	5	8	6	2	6	0	1	1	3	7	8
	0	3	1	5	2	4	3	5	2	2	6	1	6	2	0	8	0	4	0	5
	8																			
	0																			
	0:																			
	0:																			
	0:																			
	0:																			
	0-																			
	0																			
	5:																			
	0																			
	0																			
2019-08-08 00:00:00	0.	0.	-0	0.	0.	0.	0.	0.	0.	0.	0.	0.	-0	-0	0.	0.	-0	0.	0.	0.
	0	0	.0	0	0	0	0	0	0	0	0	0	.0	.0	0	0	.0	0	0	0
	1	1	1	0	0	0	3	1	1	1	3	4	4	0	0	1	0	0	1	1
	9-	6	4	1	1	9	8	9	9	1	4	1	3	2	5	1	1	0	9	1
	0	9	3	5	7	4	5	6	1	9	9	4	2	0	5	8	8	9	7	7
	1-	8	0	0	1	8	1	6	7	2	4	8	5	8	0	1	5	6	0	9
	0	1	0	5	4	3	8	7	6	7	8	8	0	9	9	9	3	8	7	6
	9																			
	0																			
	0:																			
	0:																			
	0:																			
	0:																			
	0-																			
	0																			
	5:																			
	0																			
	0																			

Use 80% of the data as a training sample (below we will use an additional 20% of the training data for validation purposes) and the remaining 20% belongs to the test part.

```
In [3]: print(f"Final DataFrame: {df.shape}")
print("Tickers included:", df.columns.tolist())

# Train/test split (80/20)
train_size = int(0.8 * df.shape[0])
Y_train = df.iloc[:train_size + 1].to_numpy()
Y_test = df.iloc[train_size + 1:].to_numpy()
```

Final DataFrame: (1508, 20)

Tickers included: ['AAPL', 'MSFT', 'GOOG', 'AMZN', 'TSLA', 'TCEHY', 'NVDA', 'TSM', 'META', 'BABA', 'ASML', 'AVGO', 'ORCL', 'CSCO', 'ADBE', 'CRM', 'NFLX', 'TXN', 'QCOM', 'SAP']

```
In [4]: print(f"\nBeginning of training sample: {df.index[0].date()}")
print(f"End of training sample: {df.index[train_size].date()}")
print(f"End of test sample: {df.index[-1].date()}")
print(f"Training samples: {Y_train.shape}")
print(f"Test samples: {Y_test.shape}")
```

Beginning of training sample: 2019-01-03

End of training sample: 2023-10-18

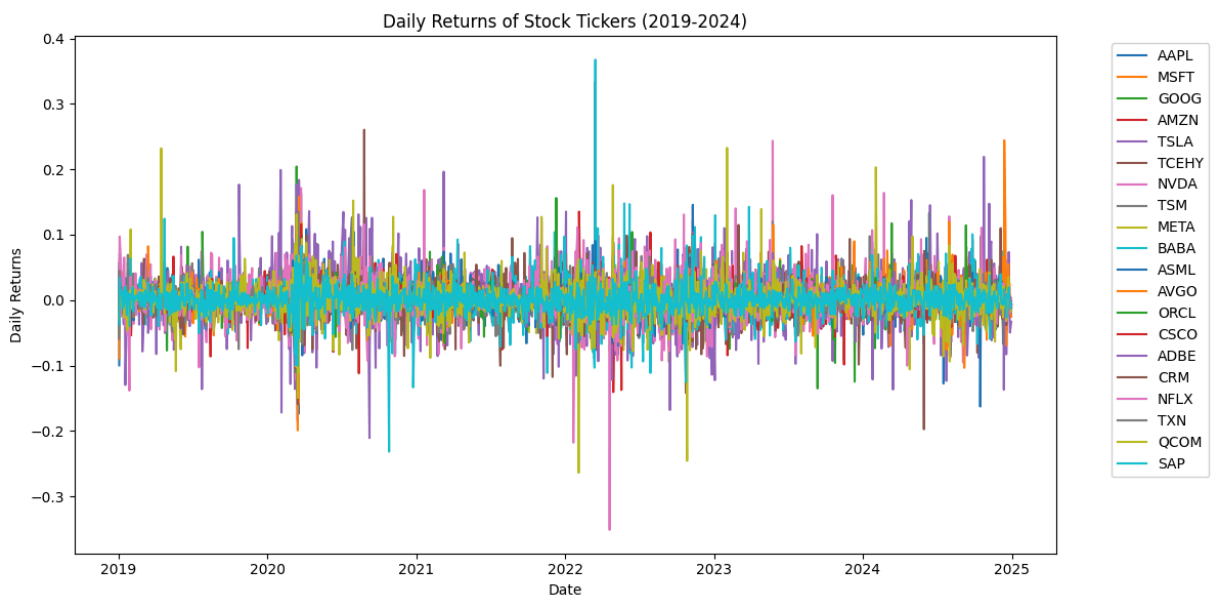
End of test sample: 2024-12-30

Training samples: (1207, 20)

Test samples: (301, 20)

```
In [5]: import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
plt.plot(df.index, df)
plt.title(f'Daily Returns of Stock Tickers ({df.index[0].year}-
{df.index[-1].year})')
plt.xlabel('Date')
plt.ylabel('Daily Returns')
plt.legend(df.columns, bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



2. APPLYING PRINCIPAL COMPONENT ANALYSIS (PCA)

Before building the autoencoder, we are going to perform principal component analysis (PCA) on the data. This analysis serves as a good benchmark when we later train autoencoders to learn the latent factors.

Remember that PCA consists on a linear transformation of the data to a low-dimensional space of factors that are orthogonal between each other. Below, we perform PCA on the data, extracting the first 3 principal components. The

principal components are the columns of the matrix of eigenvectors from diagonalizing the variance-covariance matrix of the data. We have chosen the first 3 principal components to explain the stock return variation of 20 stocks, meaning that we obtain three 20x1 vectors of loadings. The loadings capture how much each single stock is "exposed" to the principal component.

We can also extract the proportion of the variation of the data explained by the first 3 principal components. See how the first principal component explains more than half of the variation in the data. The second and third components explain, respectively, roughly 10 percent and 5 percent of the variation.

The factors that drive stock returns and that are extracted from PCA represent latent variables that are sometimes hard to interpret or to give them a "name," particularly as we consider the second, third, and further components. However, when we analyze stock returns, the first principal component can be most of the time attributed to a "market factor" that generates positive comovement in returns across stocks. As we show, the loadings on this first factor are all negative across stocks (remember that the variance of Z is the same as the variance of $-Z$). This confirms that the stock price of a single stock tends to move in the same direction as all the remaining stocks, and this explains more than 50% of the variation of returns.

```
In [6]: from sklearn.decomposition import PCA

pca = PCA(n_components=3)
pca.fit(Y_train)
print(pca.components_.T)
# Cumulative share of the variation of the data predicted the factors
print(np.cumsum(pca.explained_variance_ratio_))

[[-0.20332611 -0.08862559 -0.00286622]
 [-0.1996453  -0.13252331  0.01765695]
 [-0.19036866 -0.11370978  0.03904352]
 [-0.20425213 -0.06864972  0.00102689]
 [-0.3344536   0.47144555 -0.79461501]
 [-0.1901963   0.487918    0.37368002]
 [-0.3408636  -0.15417502 -0.02877177]
 [-0.20345294 -0.04641713  0.04437633]
 [-0.24227164 -0.11326835  0.10667999]
 [-0.21053143  0.57238145  0.44041048]
 [-0.26026589 -0.09684741  0.04523965]
 [-0.22736039 -0.12364523  0.00800747]
 [-0.13365969 -0.1512764   0.07292133]
 [-0.13219045 -0.14066919  0.05780975]
 [-0.23419577 -0.12478373  0.02581529]
 [-0.21978161 -0.08489683 -0.00770036]
 [-0.22835938  0.03082332  0.00750172]
 [-0.19340585 -0.13229862  0.05043233]
 [-0.24460667 -0.12143996  0.04582046]
 [-0.15779398 -0.0831969   0.05263593]]
[0.52145945 0.60819791 0.69199829]
```

Notice that the factors generated by PCA are indeed uncorrelated and that the loadings are orthonormal. All rights reserved WQU WorldQuant University QQQQ

```
In [7]: # Factors (encodings) generated by PCA
pca_factors = np.dot(pca.components_, Y_train.T)
# Show that factors are uncorrelated
print(np.round(np.cov(pca_factors, rowvar=True), 5))
# Show that loadings are orthonormal
print(np.round(np.dot(pca.components_, pca.components_.T), 5))
```

```
[[ 0.00672 -0.      -0.      ]
 [-0.      0.00112  0.      ]
 [-0.      0.      0.00108]]
[[ 1.  0. -0.]
 [ 0.  1. -0.]
 [-0. -0.  1.]]
```

An important remark is that the loadings and factors from PCA are not unique. Any linear, orthogonal transformation of the eigenvectors also maximizes the variance generated by the projections. Different software packages may generate different solutions. However, all the solutions should explain the same amount of variation in a given dataset.

Below we perform PCA extracting the first 6 principal components.

```
In [8]: pca = PCA(n_components=6)
pca.fit(Y_train)
print(pca.components_.T)
# Cumulative share of the variation of the data predicted the factors
print(np.cumsum(pca.explained_variance_ratio_))
```

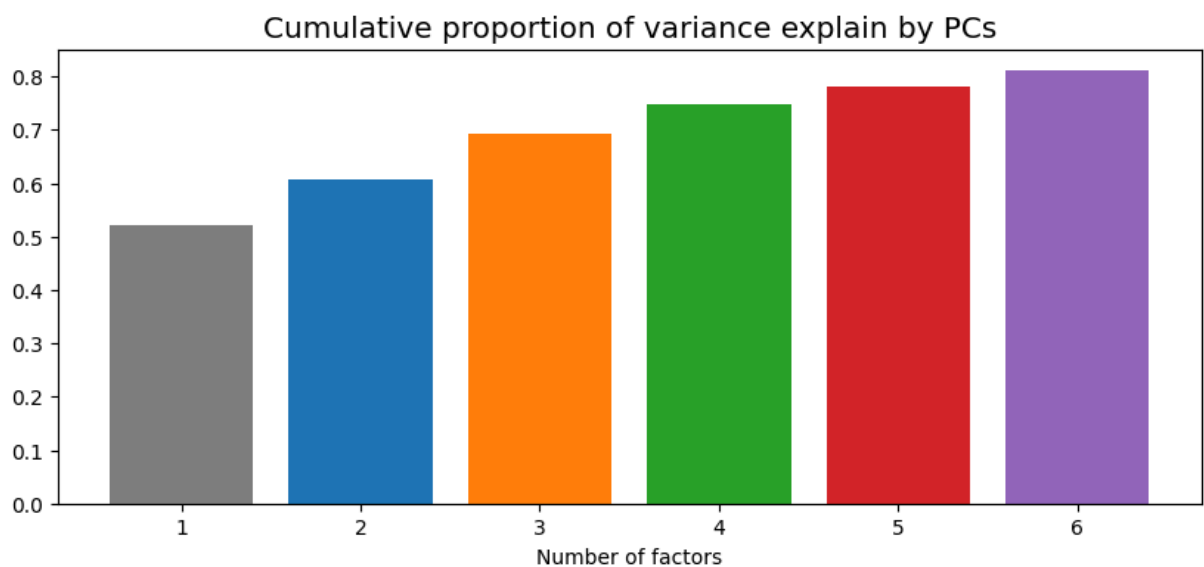
```

[[-2.03326113e-01 -8.86305513e-02 -2.86892556e-03 -2.18050311e-03
  1.39588981e-01 1.28823724e-02]
[-1.99645303e-01 -1.32516970e-01 1.76537624e-02 -4.81170301e-02
  1.71106411e-01 5.93128190e-02]
[-1.90368657e-01 -1.13710192e-01 3.90304093e-02 -1.16007207e-01
  1.85887877e-01 -1.35038202e-01]
[-2.04252134e-01 -6.86576446e-02 1.03641457e-03 -2.39601579e-01
  4.00280187e-02 -1.01450360e-01]
[-3.34453600e-01 4.71445859e-01 -7.94615233e-01 4.42999102e-02
  1.25302888e-01 4.66791209e-04]
[-1.90196303e-01 4.87912683e-01 3.73670019e-01 1.13861766e-01
  1.42086030e-02 5.28128873e-02]
[-3.40863598e-01 -1.54182844e-01 -2.87800844e-02 1.77631264e-01
  -3.02636192e-01 -2.48178199e-01]
[-2.03452942e-01 -4.64344566e-02 4.43868884e-02 2.80096037e-01
  -2.01393272e-01 -7.76834049e-02]
[-2.42271644e-01 -1.13266798e-01 1.06681564e-01 -3.87717440e-01
  2.42011811e-01 -6.54256789e-01]
[-2.10531430e-01 5.72387453e-01 4.40417989e-01 -6.95735330e-03
  9.04838690e-02 -1.43220865e-02]
[-2.60265891e-01 -9.68224394e-02 4.52474096e-02 2.35928622e-01
  -1.61806236e-01 -2.68969329e-02]
[-2.27360388e-01 -1.23630789e-01 8.01110306e-03 2.37829369e-01
  -5.97119217e-02 8.00429060e-02]
[-1.33659691e-01 -1.51271071e-01 7.29213556e-02 1.08568379e-02
  2.54283356e-01 3.95803673e-01]
[-1.32190446e-01 -1.40679185e-01 5.78071144e-02 7.91530861e-02
  2.17840845e-01 2.46379970e-01]
[-2.34195770e-01 -1.24760700e-01 2.58375904e-02 -1.10667433e-01
  1.49822631e-01 1.56191581e-01]
[-2.19781612e-01 -8.49051109e-02 -7.70505031e-03 -2.04613672e-01
  2.21465526e-01 2.42335076e-01]
[-2.28359384e-01 3.08221802e-02 7.49818855e-03 -6.11911575e-01
  -6.34343821e-01 3.21201283e-01]
[-1.93405853e-01 -1.32295528e-01 5.04350341e-02 1.88358479e-01
  -4.28439101e-02 6.10902382e-02]
[-2.44606667e-01 -1.21448248e-01 4.58119833e-02 2.59948448e-01
  -2.13862509e-01 -8.99871562e-02]
[-1.57793978e-01 -8.32232783e-02 5.26238628e-02 3.60058548e-02
  1.92232203e-01 2.15636056e-01]]
[0.52145945 0.60819791 0.69199829 0.7478485 0.78188589 0.81024463]

```

You can verify that the solution with 3 principal components is nested by the solution when we extract additional principal components. That is, the loadings on the first three components and the variation explained by them is the same as we obtained above.

```
In [9]: # plotting the coefficient score
fig, ax = plt.subplots(figsize=(10, 4))
color = [
    "tab:gray",
    "tab:blue",
    "tab:orange",
    "tab:green",
    "tab:red",
    "tab:purple",
    "tab:brown",
    "tab:pink",
    "tab:gray",
    "tab:olive",
    "tab:cyan",
    "tab:orange",
    "tab:green",
    "tab:blue",
    "tab:olive",
]
ax.bar(list(range(1, 7)), np.cumsum(pca.explained_variance_ratio_),
color=color)
plt.style.use("ggplot")
plt.title("Cumulative proportion of variance explain by PCs")
plt.xlabel("Number of factors")
plt.show()
```



3. BUILDING A LINEAR AUTOENCODER WITH TENSORFLOW

Below we build a single-layer linear autoencoder with TensorFlow via [subclassing](#). The autoencoder has two Dense (fully connected) layers: an encoder, which compresses the data into a latent vector with a dimension set by "bottleneck_dim," and a decoder, which reconstructs the original input from the latent space.

```
In [10]: import tensorflow as tf
```

```
2025-05-26 10:46:01.846294: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-26 10:46:01.890793: I
tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16
AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
2025-05-26 10:46:03.404539: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

```
In [11]: class autoencoder(tf.keras.Model):
    def __init__(self, output_dim, bottleneck_dim=1):
        super(autoencoder, self).__init__()
        self.output_dim = output_dim
        self.bottleneck_dim = bottleneck_dim
        initializer = tf.keras.initializers.GlorotUniform()
        # Build the encoder layer
        self.encoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(
                    bottleneck_dim,
                    kernel_initializer=initializer,
                    bias_initializer=tf.keras.initializers.Zeros(),
                )
            ]
        )
        # Build the decoder layer
        self.decoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(
                    output_dim,
                    kernel_initializer=initializer,
                    bias_initializer=tf.keras.initializers.Zeros(),
                )
            ]
        )

    def call(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded
```

Below we compile and train the model using our training data. We choose a "bottleneck" with 3 latent factors. We train the model for at most 1,000 epochs, setting an early stopping criterion using a validation sample of 20% of all the training observations. The early stopping criterion may not bind, depending on the particular seed used in the training algorithm.

```
In [12]: # Choose the number of latent factors
nlatent = 3
# Choose the proportion of training data used for validation
val_split = 0.2
# Determine early stopping criterion
early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss",
patience=50)
# Choose optimizer
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
# Build and compile the model
model = autoencoder(Y_train.shape[1], nlatent)
model.compile(optimizer=adam, loss="mse")
# Train the model
model.fit(
    x=Y_train,
    y=Y_train,
    batch_size=32,
    epochs=350,
    validation_split=val_split,
    callbacks=[early_stop],
    verbose=1,
)
```

Epoch 1/350
31/31  **1s** 7ms/step - loss: 9.2919e-04 - val_loss: 7.5377e-04
Epoch 2/350
31/31  **0s** 4ms/step - loss: 9.0401e-04 - val_loss: 7.3931e-04
Epoch 3/350
31/31  **0s** 4ms/step - loss: 8.4248e-04 - val_loss: 7.2670e-04
Epoch 4/350
31/31  **0s** 4ms/step - loss: 8.6337e-04 - val_loss: 7.1399e-04
Epoch 5/350
31/31  **0s** 4ms/step - loss: 9.2274e-04 - val_loss: 7.0303e-04
Epoch 6/350
31/31  **0s** 4ms/step - loss: 8.8731e-04 - val_loss: 6.9276e-04
Epoch 7/350
31/31  **0s** 4ms/step - loss: 7.6780e-04 - val_loss: 6.8293e-04
Epoch 8/350
31/31  **0s** 3ms/step - loss: 8.1681e-04 - val_loss: 6.7424e-04
Epoch 9/350
31/31  **0s** 3ms/step - loss: 8.1545e-04 - val_loss: 6.6614e-04
Epoch 10/350
31/31  **0s** 3ms/step - loss: 8.1747e-04 - val_loss: 6.5837e-04
Epoch 11/350
31/31  **0s** 4ms/step - loss: 8.1108e-04 - val_loss: 6.5109e-04
Epoch 12/350
31/31  **0s** 4ms/step - loss: 8.1877e-04 - val_loss: 6.4440e-04
Epoch 13/350
31/31  **0s** 4ms/step - loss: 7.9767e-04 - val_loss: 6.3781e-04
Epoch 14/350
31/31  **0s** 4ms/step - loss: 7.8676e-04 - val_loss: 6.3212e-04
Epoch 15/350
31/31  **0s** 4ms/step - loss: 7.2650e-04 - val_loss: 6.2640e-04
Epoch 16/350
31/31  **0s** 4ms/step - loss: 8.4396e-04 - val_loss: 6.2102e-04
Epoch 17/350
31/31  **0s** 4ms/step - loss: 7.5019e-04 - val_loss: 6.1572e-04
Epoch 18/350
31/31  **0s** 3ms/step - loss: 7.1734e-04 - val_loss: 6.1096e-04
Epoch 19/350
31/31  **0s** 3ms/step - loss: 6.8568e-04 - val_loss: 6.0638e-04
Epoch 20/350
31/31  **0s** 4ms/step - loss: 7.2140e-04 - val_loss: 6.0150e-04

Epoch 21/350
31/31  0s 4ms/step - loss: 7.4381e-04 - val_loss: 5.9690e-04
Epoch 22/350
31/31  0s 4ms/step - loss: 6.6287e-04 - val_loss: 5.9267e-04
Epoch 23/350
31/31  0s 4ms/step - loss: 6.9261e-04 - val_loss: 5.8892e-04
Epoch 24/350
31/31  0s 4ms/step - loss: 6.9214e-04 - val_loss: 5.8469e-04
Epoch 25/350
31/31  0s 4ms/step - loss: 6.7579e-04 - val_loss: 5.8076e-04
Epoch 26/350
31/31  0s 4ms/step - loss: 6.8817e-04 - val_loss: 5.7696e-04
Epoch 27/350
31/31  0s 3ms/step - loss: 6.8490e-04 - val_loss: 5.7286e-04
Epoch 28/350
31/31  0s 3ms/step - loss: 6.1713e-04 - val_loss: 5.6921e-04
Epoch 29/350
31/31  0s 4ms/step - loss: 6.7855e-04 - val_loss: 5.6550e-04
Epoch 30/350
31/31  0s 4ms/step - loss: 6.6751e-04 - val_loss: 5.6142e-04
Epoch 31/350
31/31  0s 4ms/step - loss: 6.4237e-04 - val_loss: 5.5770e-04
Epoch 32/350
31/31  0s 4ms/step - loss: 6.4744e-04 - val_loss: 5.5383e-04
Epoch 33/350
31/31  0s 4ms/step - loss: 6.6856e-04 - val_loss: 5.5003e-04
Epoch 34/350
31/31  0s 4ms/step - loss: 6.3021e-04 - val_loss: 5.4650e-04
Epoch 35/350
31/31  0s 4ms/step - loss: 6.3122e-04 - val_loss: 5.4242e-04
Epoch 36/350
31/31  0s 4ms/step - loss: 6.4134e-04 - val_loss: 5.3859e-04
Epoch 37/350
31/31  0s 4ms/step - loss: 6.0461e-04 - val_loss: 5.3535e-04
Epoch 38/350
31/31  0s 4ms/step - loss: 6.1082e-04 - val_loss: 5.3173e-04
Epoch 39/350
31/31  0s 4ms/step - loss: 6.8435e-04 - val_loss: 5.2781e-04
Epoch 40/350
31/31  0s 4ms/step - loss: 5.9851e-04 - val_loss: 5.2407e-04

Epoch 41/350
31/31  0s 4ms/step - loss: 5.9480e-04 - val_loss: 5.2038e-04
Epoch 42/350
31/31  0s 4ms/step - loss: 6.1261e-04 - val_loss: 5.1671e-04
Epoch 43/350
31/31  0s 4ms/step - loss: 6.3842e-04 - val_loss: 5.1291e-04
Epoch 44/350
31/31  0s 4ms/step - loss: 5.7130e-04 - val_loss: 5.0945e-04
Epoch 45/350
31/31  0s 4ms/step - loss: 5.7881e-04 - val_loss: 5.0573e-04
Epoch 46/350
31/31  0s 3ms/step - loss: 6.1606e-04 - val_loss: 5.0204e-04
Epoch 47/350
31/31  0s 4ms/step - loss: 5.4292e-04 - val_loss: 4.9840e-04
Epoch 48/350
31/31  0s 4ms/step - loss: 5.8006e-04 - val_loss: 4.9462e-04
Epoch 49/350
31/31  0s 3ms/step - loss: 5.7763e-04 - val_loss: 4.9101e-04
Epoch 50/350
31/31  0s 3ms/step - loss: 5.4772e-04 - val_loss: 4.8748e-04
Epoch 51/350
31/31  0s 3ms/step - loss: 5.7466e-04 - val_loss: 4.8358e-04
Epoch 52/350
31/31  0s 4ms/step - loss: 5.4748e-04 - val_loss: 4.8021e-04
Epoch 53/350
31/31  0s 4ms/step - loss: 5.4614e-04 - val_loss: 4.7637e-04
Epoch 54/350
31/31  0s 4ms/step - loss: 5.3049e-04 - val_loss: 4.7280e-04
Epoch 55/350
31/31  0s 4ms/step - loss: 5.3070e-04 - val_loss: 4.6932e-04
Epoch 56/350
31/31  0s 4ms/step - loss: 5.0853e-04 - val_loss: 4.6543e-04
Epoch 57/350
31/31  0s 4ms/step - loss: 5.1897e-04 - val_loss: 4.6169e-04
Epoch 58/350
31/31  0s 4ms/step - loss: 5.4831e-04 - val_loss: 4.5808e-04
Epoch 59/350
31/31  0s 4ms/step - loss: 4.6257e-04 - val_loss: 4.5483e-04
Epoch 60/350
31/31  0s 4ms/step - loss: 5.2082e-04 - val_loss: 4.5117e-04

Epoch 61/350
31/31  **0s** 4ms/step - loss: 4.8605e-04 - val_loss: 4.4763e-04
Epoch 62/350
31/31  **0s** 4ms/step - loss: 5.0857e-04 - val_loss: 4.4419e-04
Epoch 63/350
31/31  **0s** 4ms/step - loss: 4.9395e-04 - val_loss: 4.4064e-04
Epoch 64/350
31/31  **0s** 5ms/step - loss: 5.0147e-04 - val_loss: 4.3713e-04
Epoch 65/350
31/31  **0s** 4ms/step - loss: 4.7816e-04 - val_loss: 4.3351e-04
Epoch 66/350
31/31  **0s** 4ms/step - loss: 5.0089e-04 - val_loss: 4.3027e-04
Epoch 67/350
31/31  **0s** 4ms/step - loss: 4.9325e-04 - val_loss: 4.2655e-04
Epoch 68/350
31/31  **0s** 4ms/step - loss: 4.9347e-04 - val_loss: 4.2344e-04
Epoch 69/350
31/31  **0s** 4ms/step - loss: 4.6400e-04 - val_loss: 4.1986e-04
Epoch 70/350
31/31  **0s** 4ms/step - loss: 4.6083e-04 - val_loss: 4.1635e-04
Epoch 71/350
31/31  **0s** 4ms/step - loss: 4.6817e-04 - val_loss: 4.1292e-04
Epoch 72/350
31/31  **0s** 4ms/step - loss: 4.4257e-04 - val_loss: 4.0943e-04
Epoch 73/350
31/31  **0s** 4ms/step - loss: 4.6016e-04 - val_loss: 4.0576e-04
Epoch 74/350
31/31  **0s** 4ms/step - loss: 4.4633e-04 - val_loss: 4.0281e-04
Epoch 75/350
31/31  **0s** 4ms/step - loss: 4.8629e-04 - val_loss: 3.9924e-04
Epoch 76/350
31/31  **0s** 4ms/step - loss: 4.8398e-04 - val_loss: 3.9635e-04
Epoch 77/350
31/31  **0s** 4ms/step - loss: 3.9727e-04 - val_loss: 3.9289e-04
Epoch 78/350
31/31  **0s** 3ms/step - loss: 4.1785e-04 - val_loss: 3.8952e-04
Epoch 79/350
31/31  **0s** 4ms/step - loss: 4.4408e-04 - val_loss: 3.8617e-04
Epoch 80/350
31/31  **0s** 4ms/step - loss: 4.3119e-04 - val_loss: 3.8283e-04

Epoch 81/350
31/31  **0s** 4ms/step - loss: 4.2323e-04 - val_loss: 3.7961e-04
Epoch 82/350
31/31  **0s** 4ms/step - loss: 4.1451e-04 - val_loss: 3.7675e-04
Epoch 83/350
31/31  **0s** 4ms/step - loss: 4.1517e-04 - val_loss: 3.7340e-04
Epoch 84/350
31/31  **0s** 4ms/step - loss: 4.0943e-04 - val_loss: 3.7014e-04
Epoch 85/350
31/31  **0s** 4ms/step - loss: 4.1974e-04 - val_loss: 3.6712e-04
Epoch 86/350
31/31  **0s** 4ms/step - loss: 4.2334e-04 - val_loss: 3.6435e-04
Epoch 87/350
31/31  **0s** 4ms/step - loss: 4.1801e-04 - val_loss: 3.6121e-04
Epoch 88/350
31/31  **0s** 4ms/step - loss: 3.9369e-04 - val_loss: 3.5826e-04
Epoch 89/350
31/31  **0s** 4ms/step - loss: 4.1828e-04 - val_loss: 3.5541e-04
Epoch 90/350
31/31  **0s** 4ms/step - loss: 4.2806e-04 - val_loss: 3.5233e-04
Epoch 91/350
31/31  **0s** 4ms/step - loss: 4.1618e-04 - val_loss: 3.4964e-04
Epoch 92/350
31/31  **0s** 4ms/step - loss: 3.9535e-04 - val_loss: 3.4657e-04
Epoch 93/350
31/31  **0s** 4ms/step - loss: 3.7432e-04 - val_loss: 3.4396e-04
Epoch 94/350
31/31  **0s** 4ms/step - loss: 3.7788e-04 - val_loss: 3.4136e-04
Epoch 95/350
31/31  **0s** 4ms/step - loss: 3.4525e-04 - val_loss: 3.3849e-04
Epoch 96/350
31/31  **0s** 4ms/step - loss: 3.8217e-04 - val_loss: 3.3589e-04
Epoch 97/350
31/31  **0s** 4ms/step - loss: 3.8639e-04 - val_loss: 3.3339e-04
Epoch 98/350
31/31  **0s** 4ms/step - loss: 3.7903e-04 - val_loss: 3.3091e-04
Epoch 99/350
31/31  **0s** 4ms/step - loss: 3.6542e-04 - val_loss: 3.2864e-04
Epoch 100/350
31/31  **0s** 4ms/step - loss: 3.3699e-04 - val_loss: 3.2649e-04

Epoch 101/350
31/31  **0s** 4ms/step - loss: 3.5413e-04 - val_loss: 3.2395e-04
Epoch 102/350
31/31  **0s** 4ms/step - loss: 3.5483e-04 - val_loss: 3.2177e-04
Epoch 103/350
31/31  **0s** 5ms/step - loss: 3.4037e-04 - val_loss: 3.1961e-04
Epoch 104/350
31/31  **0s** 4ms/step - loss: 3.2986e-04 - val_loss: 3.1754e-04
Epoch 105/350
31/31  **0s** 4ms/step - loss: 3.3308e-04 - val_loss: 3.1543e-04
Epoch 106/350
31/31  **0s** 4ms/step - loss: 3.4204e-04 - val_loss: 3.1346e-04
Epoch 107/350
31/31  **0s** 4ms/step - loss: 3.4981e-04 - val_loss: 3.1108e-04
Epoch 108/350
31/31  **0s** 4ms/step - loss: 3.3718e-04 - val_loss: 3.0934e-04
Epoch 109/350
31/31  **0s** 4ms/step - loss: 3.4511e-04 - val_loss: 3.0761e-04
Epoch 110/350
31/31  **0s** 4ms/step - loss: 3.1178e-04 - val_loss: 3.0590e-04
Epoch 111/350
31/31  **0s** 4ms/step - loss: 3.1879e-04 - val_loss: 3.0442e-04
Epoch 112/350
31/31  **0s** 3ms/step - loss: 3.2512e-04 - val_loss: 3.0249e-04
Epoch 113/350
31/31  **0s** 3ms/step - loss: 3.2146e-04 - val_loss: 3.0099e-04
Epoch 114/350
31/31  **0s** 4ms/step - loss: 3.2598e-04 - val_loss: 2.9937e-04
Epoch 115/350
31/31  **0s** 4ms/step - loss: 3.2396e-04 - val_loss: 2.9784e-04
Epoch 116/350
31/31  **0s** 4ms/step - loss: 3.1402e-04 - val_loss: 2.9641e-04
Epoch 117/350
31/31  **0s** 3ms/step - loss: 3.0830e-04 - val_loss: 2.9478e-04
Epoch 118/350
31/31  **0s** 4ms/step - loss: 2.9192e-04 - val_loss: 2.9323e-04
Epoch 119/350
31/31  **0s** 4ms/step - loss: 3.0044e-04 - val_loss: 2.9223e-04
Epoch 120/350
31/31  **0s** 4ms/step - loss: 3.2041e-04 - val_loss: 2.9080e-04

Epoch 121/350
31/31  0s 4ms/step - loss: 2.9978e-04 - val_loss: 2.8943e-04
Epoch 122/350
31/31  0s 4ms/step - loss: 3.0148e-04 - val_loss: 2.8807e-04
Epoch 123/350
31/31  0s 4ms/step - loss: 3.0802e-04 - val_loss: 2.8688e-04
Epoch 124/350
31/31  0s 4ms/step - loss: 3.3759e-04 - val_loss: 2.8565e-04
Epoch 125/350
31/31  0s 4ms/step - loss: 3.0873e-04 - val_loss: 2.8480e-04
Epoch 126/350
31/31  0s 4ms/step - loss: 3.0583e-04 - val_loss: 2.8332e-04
Epoch 127/350
31/31  0s 4ms/step - loss: 2.9842e-04 - val_loss: 2.8229e-04
Epoch 128/350
31/31  0s 4ms/step - loss: 3.1516e-04 - val_loss: 2.8133e-04
Epoch 129/350
31/31  0s 4ms/step - loss: 2.9283e-04 - val_loss: 2.8027e-04
Epoch 130/350
31/31  0s 3ms/step - loss: 2.9061e-04 - val_loss: 2.7938e-04
Epoch 131/350
31/31  0s 4ms/step - loss: 3.0848e-04 - val_loss: 2.7841e-04
Epoch 132/350
31/31  0s 4ms/step - loss: 2.9066e-04 - val_loss: 2.7748e-04
Epoch 133/350
31/31  0s 3ms/step - loss: 2.8932e-04 - val_loss: 2.7647e-04
Epoch 134/350
31/31  0s 4ms/step - loss: 2.9497e-04 - val_loss: 2.7555e-04
Epoch 135/350
31/31  0s 4ms/step - loss: 2.9395e-04 - val_loss: 2.7474e-04
Epoch 136/350
31/31  0s 4ms/step - loss: 3.1819e-04 - val_loss: 2.7409e-04
Epoch 137/350
31/31  0s 4ms/step - loss: 2.8042e-04 - val_loss: 2.7303e-04
Epoch 138/350
31/31  0s 5ms/step - loss: 2.9634e-04 - val_loss: 2.7220e-04
Epoch 139/350
31/31  0s 3ms/step - loss: 2.7763e-04 - val_loss: 2.7167e-04
Epoch 140/350
31/31  0s 4ms/step - loss: 2.9456e-04 - val_loss: 2.7072e-04

Epoch 141/350
31/31  **0s** 4ms/step - loss: 3.1669e-04 - val_loss: 2.6984e-04

Epoch 142/350
31/31  **0s** 4ms/step - loss: 3.0520e-04 - val_loss: 2.6929e-04

Epoch 143/350
31/31  **0s** 4ms/step - loss: 2.7530e-04 - val_loss: 2.6838e-04

Epoch 144/350
31/31  **0s** 4ms/step - loss: 2.6872e-04 - val_loss: 2.6786e-04

Epoch 145/350
31/31  **0s** 3ms/step - loss: 2.6615e-04 - val_loss: 2.6707e-04

Epoch 146/350
31/31  **0s** 4ms/step - loss: 2.7653e-04 - val_loss: 2.6634e-04

Epoch 147/350
31/31  **0s** 3ms/step - loss: 2.7918e-04 - val_loss: 2.6569e-04

Epoch 148/350
31/31  **0s** 4ms/step - loss: 2.8134e-04 - val_loss: 2.6496e-04

Epoch 149/350
31/31  **0s** 4ms/step - loss: 2.7893e-04 - val_loss: 2.6437e-04

Epoch 150/350
31/31  **0s** 3ms/step - loss: 2.9071e-04 - val_loss: 2.6380e-04

Epoch 151/350
31/31  **0s** 4ms/step - loss: 2.7412e-04 - val_loss: 2.6298e-04

Epoch 152/350
31/31  **0s** 4ms/step - loss: 2.8317e-04 - val_loss: 2.6229e-04

Epoch 153/350
31/31  **0s** 4ms/step - loss: 2.7402e-04 - val_loss: 2.6170e-04

Epoch 154/350
31/31  **0s** 4ms/step - loss: 2.9113e-04 - val_loss: 2.6110e-04

Epoch 155/350
31/31  **0s** 4ms/step - loss: 2.8000e-04 - val_loss: 2.6029e-04

Epoch 156/350
31/31  **0s** 4ms/step - loss: 2.7153e-04 - val_loss: 2.5981e-04

Epoch 157/350
31/31  **0s** 4ms/step - loss: 2.6143e-04 - val_loss: 2.5924e-04

Epoch 158/350
31/31  **0s** 4ms/step - loss: 2.7795e-04 - val_loss: 2.5843e-04

Epoch 159/350
31/31  **0s** 3ms/step - loss: 2.9790e-04 - val_loss: 2.5786e-04

Epoch 160/350
31/31  **0s** 4ms/step - loss: 2.7053e-04 - val_loss: 2.5743e-04

Epoch 161/350
31/31  **0s** 3ms/step - loss: 2.7532e-04 - val_loss: 2.5680e-04
Epoch 162/350
31/31  **0s** 3ms/step - loss: 2.6738e-04 - val_loss: 2.5611e-04
Epoch 163/350
31/31  **0s** 4ms/step - loss: 2.9914e-04 - val_loss: 2.5548e-04
Epoch 164/350
31/31  **0s** 4ms/step - loss: 2.7534e-04 - val_loss: 2.5510e-04
Epoch 165/350
31/31  **0s** 4ms/step - loss: 2.6316e-04 - val_loss: 2.5431e-04
Epoch 166/350
31/31  **0s** 4ms/step - loss: 2.7122e-04 - val_loss: 2.5388e-04
Epoch 167/350
31/31  **0s** 3ms/step - loss: 2.6221e-04 - val_loss: 2.5332e-04
Epoch 168/350
31/31  **0s** 4ms/step - loss: 2.5142e-04 - val_loss: 2.5279e-04
Epoch 169/350
31/31  **0s** 4ms/step - loss: 2.6817e-04 - val_loss: 2.5208e-04
Epoch 170/350
31/31  **0s** 4ms/step - loss: 2.7834e-04 - val_loss: 2.5152e-04
Epoch 171/350
31/31  **0s** 4ms/step - loss: 2.5123e-04 - val_loss: 2.5122e-04
Epoch 172/350
31/31  **0s** 3ms/step - loss: 2.6014e-04 - val_loss: 2.5049e-04
Epoch 173/350
31/31  **0s** 4ms/step - loss: 2.5680e-04 - val_loss: 2.4998e-04
Epoch 174/350
31/31  **0s** 4ms/step - loss: 2.8767e-04 - val_loss: 2.4915e-04
Epoch 175/350
31/31  **0s** 4ms/step - loss: 2.5986e-04 - val_loss: 2.4899e-04
Epoch 176/350
31/31  **0s** 4ms/step - loss: 2.6431e-04 - val_loss: 2.4853e-04
Epoch 177/350
31/31  **0s** 3ms/step - loss: 2.6315e-04 - val_loss: 2.4809e-04
Epoch 178/350
31/31  **0s** 4ms/step - loss: 2.6235e-04 - val_loss: 2.4774e-04
Epoch 179/350
31/31  **0s** 4ms/step - loss: 2.7654e-04 - val_loss: 2.4686e-04
Epoch 180/350
31/31  **0s** 4ms/step - loss: 2.3677e-04 - val_loss: 2.4625e-04

Epoch 181/350
31/31  0s 4ms/step - loss: 2.4118e-04 - val_loss: 2.4573e-04

Epoch 182/350
31/31  0s 4ms/step - loss: 2.6377e-04 - val_loss: 2.4525e-04

Epoch 183/350
31/31  0s 4ms/step - loss: 2.6628e-04 - val_loss: 2.4480e-04

Epoch 184/350
31/31  0s 4ms/step - loss: 2.7813e-04 - val_loss: 2.4446e-04

Epoch 185/350
31/31  0s 4ms/step - loss: 2.5880e-04 - val_loss: 2.4380e-04

Epoch 186/350
31/31  0s 3ms/step - loss: 2.5003e-04 - val_loss: 2.4333e-04

Epoch 187/350
31/31  0s 3ms/step - loss: 2.6732e-04 - val_loss: 2.4270e-04

Epoch 188/350
31/31  0s 4ms/step - loss: 2.5005e-04 - val_loss: 2.4226e-04

Epoch 189/350
31/31  0s 3ms/step - loss: 2.6316e-04 - val_loss: 2.4163e-04

Epoch 190/350
31/31  0s 4ms/step - loss: 2.5228e-04 - val_loss: 2.4113e-04

Epoch 191/350
31/31  0s 4ms/step - loss: 2.4807e-04 - val_loss: 2.4077e-04

Epoch 192/350
31/31  0s 4ms/step - loss: 2.5585e-04 - val_loss: 2.4019e-04

Epoch 193/350
31/31  0s 4ms/step - loss: 2.6429e-04 - val_loss: 2.3985e-04

Epoch 194/350
31/31  0s 4ms/step - loss: 2.3686e-04 - val_loss: 2.3929e-04

Epoch 195/350
31/31  0s 4ms/step - loss: 2.5530e-04 - val_loss: 2.3867e-04

Epoch 196/350
31/31  0s 4ms/step - loss: 2.5364e-04 - val_loss: 2.3842e-04

Epoch 197/350
31/31  0s 4ms/step - loss: 2.4861e-04 - val_loss: 2.3746e-04

Epoch 198/350
31/31  0s 4ms/step - loss: 2.4805e-04 - val_loss: 2.3743e-04

Epoch 199/350
31/31  0s 4ms/step - loss: 2.5497e-04 - val_loss: 2.3679e-04

Epoch 200/350
31/31  0s 4ms/step - loss: 2.4203e-04 - val_loss: 2.3681e-04

Epoch 201/350
31/31  0s 3ms/step - loss: 2.5512e-04 - val_loss: 2.3597e-04
Epoch 202/350
31/31  0s 5ms/step - loss: 2.4466e-04 - val_loss: 2.3555e-04
Epoch 203/350
31/31  0s 3ms/step - loss: 2.6862e-04 - val_loss: 2.3504e-04
Epoch 204/350
31/31  0s 3ms/step - loss: 2.4463e-04 - val_loss: 2.3458e-04
Epoch 205/350
31/31  0s 3ms/step - loss: 2.4368e-04 - val_loss: 2.3417e-04
Epoch 206/350
31/31  0s 4ms/step - loss: 2.5015e-04 - val_loss: 2.3349e-04
Epoch 207/350
31/31  0s 4ms/step - loss: 2.5718e-04 - val_loss: 2.3324e-04
Epoch 208/350
31/31  0s 4ms/step - loss: 2.6617e-04 - val_loss: 2.3274e-04
Epoch 209/350
31/31  0s 4ms/step - loss: 2.3474e-04 - val_loss: 2.3237e-04
Epoch 210/350
31/31  0s 4ms/step - loss: 2.2944e-04 - val_loss: 2.3184e-04
Epoch 211/350
31/31  0s 4ms/step - loss: 2.4875e-04 - val_loss: 2.3159e-04
Epoch 212/350
31/31  0s 4ms/step - loss: 2.2187e-04 - val_loss: 2.3111e-04
Epoch 213/350
31/31  0s 4ms/step - loss: 2.5539e-04 - val_loss: 2.3026e-04
Epoch 214/350
31/31  0s 3ms/step - loss: 2.3744e-04 - val_loss: 2.3036e-04
Epoch 215/350
31/31  0s 4ms/step - loss: 2.3950e-04 - val_loss: 2.2968e-04
Epoch 216/350
31/31  0s 4ms/step - loss: 2.2768e-04 - val_loss: 2.2915e-04
Epoch 217/350
31/31  0s 4ms/step - loss: 2.4342e-04 - val_loss: 2.2868e-04
Epoch 218/350
31/31  0s 4ms/step - loss: 2.5912e-04 - val_loss: 2.2843e-04
Epoch 219/350
31/31  0s 4ms/step - loss: 2.3217e-04 - val_loss: 2.2813e-04
Epoch 220/350
31/31  0s 4ms/step - loss: 2.3110e-04 - val_loss: 2.2767e-04

Epoch 221/350
31/31  0s 4ms/step - loss: 2.4011e-04 - val_loss: 2.2723e-04
Epoch 222/350
31/31  0s 4ms/step - loss: 2.5601e-04 - val_loss: 2.2669e-04
Epoch 223/350
31/31  0s 5ms/step - loss: 2.2960e-04 - val_loss: 2.2627e-04
Epoch 224/350
31/31  0s 5ms/step - loss: 2.2889e-04 - val_loss: 2.2597e-04
Epoch 225/350
31/31  0s 5ms/step - loss: 2.3126e-04 - val_loss: 2.2532e-04
Epoch 226/350
31/31  0s 4ms/step - loss: 2.4700e-04 - val_loss: 2.2498e-04
Epoch 227/350
31/31  0s 3ms/step - loss: 2.4021e-04 - val_loss: 2.2449e-04
Epoch 228/350
31/31  0s 3ms/step - loss: 2.4193e-04 - val_loss: 2.2434e-04
Epoch 229/350
31/31  0s 3ms/step - loss: 2.3008e-04 - val_loss: 2.2408e-04
Epoch 230/350
31/31  0s 5ms/step - loss: 2.2903e-04 - val_loss: 2.2374e-04
Epoch 231/350
31/31  0s 3ms/step - loss: 2.1749e-04 - val_loss: 2.2329e-04
Epoch 232/350
31/31  0s 4ms/step - loss: 2.1405e-04 - val_loss: 2.2272e-04
Epoch 233/350
31/31  0s 4ms/step - loss: 2.2114e-04 - val_loss: 2.2250e-04
Epoch 234/350
31/31  0s 4ms/step - loss: 2.1744e-04 - val_loss: 2.2224e-04
Epoch 235/350
31/31  0s 4ms/step - loss: 2.1687e-04 - val_loss: 2.2182e-04
Epoch 236/350
31/31  0s 4ms/step - loss: 2.3701e-04 - val_loss: 2.2141e-04
Epoch 237/350
31/31  0s 3ms/step - loss: 2.4453e-04 - val_loss: 2.2126e-04
Epoch 238/350
31/31  0s 4ms/step - loss: 2.3995e-04 - val_loss: 2.2090e-04
Epoch 239/350
31/31  0s 4ms/step - loss: 2.2271e-04 - val_loss: 2.2052e-04
Epoch 240/350
31/31  0s 4ms/step - loss: 2.2968e-04 - val_loss: 2.2010e-04

Epoch 241/350
31/31  **0s** 4ms/step - loss: 2.3475e-04 - val_loss: 2.1975e-04

Epoch 242/350
31/31  **0s** 4ms/step - loss: 2.2368e-04 - val_loss: 2.1946e-04

Epoch 243/350
31/31  **0s** 4ms/step - loss: 2.2646e-04 - val_loss: 2.1929e-04

Epoch 244/350
31/31  **0s** 4ms/step - loss: 2.3054e-04 - val_loss: 2.1867e-04

Epoch 245/350
31/31  **0s** 4ms/step - loss: 2.1360e-04 - val_loss: 2.1846e-04

Epoch 246/350
31/31  **0s** 4ms/step - loss: 2.2103e-04 - val_loss: 2.1804e-04

Epoch 247/350
31/31  **0s** 4ms/step - loss: 2.3626e-04 - val_loss: 2.1771e-04

Epoch 248/350
31/31  **0s** 4ms/step - loss: 2.2420e-04 - val_loss: 2.1752e-04

Epoch 249/350
31/31  **0s** 4ms/step - loss: 2.2959e-04 - val_loss: 2.1734e-04

Epoch 250/350
31/31  **0s** 4ms/step - loss: 2.2242e-04 - val_loss: 2.1683e-04

Epoch 251/350
31/31  **0s** 4ms/step - loss: 2.2956e-04 - val_loss: 2.1673e-04

Epoch 252/350
31/31  **0s** 4ms/step - loss: 2.3320e-04 - val_loss: 2.1610e-04

Epoch 253/350
31/31  **0s** 4ms/step - loss: 2.2083e-04 - val_loss: 2.1599e-04

Epoch 254/350
31/31  **0s** 3ms/step - loss: 2.2383e-04 - val_loss: 2.1567e-04

Epoch 255/350
31/31  **0s** 4ms/step - loss: 2.3467e-04 - val_loss: 2.1551e-04

Epoch 256/350
31/31  **0s** 4ms/step - loss: 2.2593e-04 - val_loss: 2.1503e-04

Epoch 257/350
31/31  **0s** 4ms/step - loss: 2.1063e-04 - val_loss: 2.1464e-04

Epoch 258/350
31/31  **0s** 4ms/step - loss: 2.2670e-04 - val_loss: 2.1439e-04

Epoch 259/350
31/31  **0s** 3ms/step - loss: 2.1563e-04 - val_loss: 2.1422e-04

Epoch 260/350
31/31  **0s** 4ms/step - loss: 2.1897e-04 - val_loss: 2.1398e-04

Epoch 261/350
31/31  0s 4ms/step - loss: 2.4046e-04 - val_loss: 2.1374e-04

Epoch 262/350
31/31  0s 4ms/step - loss: 2.2263e-04 - val_loss: 2.1334e-04

Epoch 263/350
31/31  0s 4ms/step - loss: 2.1588e-04 - val_loss: 2.1305e-04

Epoch 264/350
31/31  0s 4ms/step - loss: 2.3484e-04 - val_loss: 2.1292e-04

Epoch 265/350
31/31  0s 4ms/step - loss: 2.1303e-04 - val_loss: 2.1268e-04

Epoch 266/350
31/31  0s 3ms/step - loss: 2.2309e-04 - val_loss: 2.1264e-04

Epoch 267/350
31/31  0s 5ms/step - loss: 2.1479e-04 - val_loss: 2.1212e-04

Epoch 268/350
31/31  0s 4ms/step - loss: 2.3818e-04 - val_loss: 2.1205e-04

Epoch 269/350
31/31  0s 4ms/step - loss: 2.2981e-04 - val_loss: 2.1164e-04

Epoch 270/350
31/31  0s 4ms/step - loss: 2.3156e-04 - val_loss: 2.1150e-04

Epoch 271/350
31/31  0s 4ms/step - loss: 2.3433e-04 - val_loss: 2.1120e-04

Epoch 272/350
31/31  0s 4ms/step - loss: 2.1463e-04 - val_loss: 2.1084e-04

Epoch 273/350
31/31  0s 4ms/step - loss: 2.1969e-04 - val_loss: 2.1084e-04

Epoch 274/350
31/31  0s 3ms/step - loss: 2.2105e-04 - val_loss: 2.1052e-04

Epoch 275/350
31/31  0s 4ms/step - loss: 2.0029e-04 - val_loss: 2.1020e-04

Epoch 276/350
31/31  0s 4ms/step - loss: 2.2881e-04 - val_loss: 2.1013e-04

Epoch 277/350
31/31  0s 4ms/step - loss: 2.2436e-04 - val_loss: 2.0973e-04

Epoch 278/350
31/31  0s 4ms/step - loss: 2.0763e-04 - val_loss: 2.0952e-04

Epoch 279/350
31/31  0s 4ms/step - loss: 2.1689e-04 - val_loss: 2.0933e-04

Epoch 280/350
31/31  0s 5ms/step - loss: 2.0525e-04 - val_loss: 2.0914e-04

Epoch 281/350
31/31  **0s** 3ms/step - loss: 2.0969e-04 - val_loss: 2.0886e-04

Epoch 282/350
31/31  **0s** 4ms/step - loss: 2.2953e-04 - val_loss: 2.0877e-04

Epoch 283/350
31/31  **0s** 4ms/step - loss: 2.1767e-04 - val_loss: 2.0853e-04

Epoch 284/350
31/31  **0s** 4ms/step - loss: 2.1847e-04 - val_loss: 2.0819e-04

Epoch 285/350
31/31  **0s** 4ms/step - loss: 2.1546e-04 - val_loss: 2.0811e-04

Epoch 286/350
31/31  **0s** 4ms/step - loss: 2.1574e-04 - val_loss: 2.0793e-04

Epoch 287/350
31/31  **0s** 4ms/step - loss: 2.0963e-04 - val_loss: 2.0764e-04

Epoch 288/350
31/31  **0s** 4ms/step - loss: 2.2068e-04 - val_loss: 2.0757e-04

Epoch 289/350
31/31  **0s** 4ms/step - loss: 1.9897e-04 - val_loss: 2.0738e-04

Epoch 290/350
31/31  **0s** 4ms/step - loss: 2.1572e-04 - val_loss: 2.0719e-04

Epoch 291/350
31/31  **0s** 4ms/step - loss: 2.1309e-04 - val_loss: 2.0697e-04

Epoch 292/350
31/31  **0s** 4ms/step - loss: 2.1309e-04 - val_loss: 2.0680e-04

Epoch 293/350
31/31  **0s** 3ms/step - loss: 2.2720e-04 - val_loss: 2.0671e-04

Epoch 294/350
31/31  **0s** 3ms/step - loss: 2.0990e-04 - val_loss: 2.0659e-04

Epoch 295/350
31/31  **0s** 4ms/step - loss: 2.0501e-04 - val_loss: 2.0619e-04

Epoch 296/350
31/31  **0s** 4ms/step - loss: 2.1179e-04 - val_loss: 2.0606e-04

Epoch 297/350
31/31  **0s** 4ms/step - loss: 2.1826e-04 - val_loss: 2.0585e-04

Epoch 298/350
31/31  **0s** 4ms/step - loss: 1.9870e-04 - val_loss: 2.0593e-04

Epoch 299/350
31/31  **0s** 4ms/step - loss: 2.2780e-04 - val_loss: 2.0575e-04

Epoch 300/350
31/31  **0s** 4ms/step - loss: 2.0097e-04 - val_loss: 2.0546e-04

Epoch 301/350
31/31  0s 4ms/step - loss: 2.2071e-04 - val_loss: 2.0521e-04
Epoch 302/350
31/31  0s 3ms/step - loss: 1.9862e-04 - val_loss: 2.0519e-04
Epoch 303/350
31/31  0s 4ms/step - loss: 2.0336e-04 - val_loss: 2.0494e-04
Epoch 304/350
31/31  0s 4ms/step - loss: 2.1712e-04 - val_loss: 2.0492e-04
Epoch 305/350
31/31  0s 4ms/step - loss: 2.0353e-04 - val_loss: 2.0463e-04
Epoch 306/350
31/31  0s 3ms/step - loss: 2.1409e-04 - val_loss: 2.0450e-04
Epoch 307/350
31/31  0s 4ms/step - loss: 2.1249e-04 - val_loss: 2.0449e-04
Epoch 308/350
31/31  0s 4ms/step - loss: 2.1514e-04 - val_loss: 2.0421e-04
Epoch 309/350
31/31  0s 4ms/step - loss: 2.1128e-04 - val_loss: 2.0394e-04
Epoch 310/350
31/31  0s 4ms/step - loss: 1.9921e-04 - val_loss: 2.0393e-04
Epoch 311/350
31/31  0s 4ms/step - loss: 2.2008e-04 - val_loss: 2.0366e-04
Epoch 312/350
31/31  0s 3ms/step - loss: 2.0216e-04 - val_loss: 2.0357e-04
Epoch 313/350
31/31  0s 4ms/step - loss: 2.0661e-04 - val_loss: 2.0337e-04
Epoch 314/350
31/31  0s 4ms/step - loss: 2.2240e-04 - val_loss: 2.0343e-04
Epoch 315/350
31/31  0s 3ms/step - loss: 2.0416e-04 - val_loss: 2.0314e-04
Epoch 316/350
31/31  0s 3ms/step - loss: 2.1321e-04 - val_loss: 2.0311e-04
Epoch 317/350
31/31  0s 4ms/step - loss: 2.1170e-04 - val_loss: 2.0304e-04
Epoch 318/350
31/31  0s 4ms/step - loss: 2.0606e-04 - val_loss: 2.0284e-04
Epoch 319/350
31/31  0s 4ms/step - loss: 2.1507e-04 - val_loss: 2.0258e-04
Epoch 320/350
31/31  0s 4ms/step - loss: 1.9794e-04 - val_loss: 2.0248e-04

Epoch 321/350
31/31  **0s** 4ms/step - loss: 2.0619e-04 - val_loss: 2.0238e-04
Epoch 322/350
31/31  **0s** 3ms/step - loss: 2.2175e-04 - val_loss: 2.0225e-04
Epoch 323/350
31/31  **0s** 4ms/step - loss: 2.1865e-04 - val_loss: 2.0215e-04
Epoch 324/350
31/31  **0s** 4ms/step - loss: 2.0982e-04 - val_loss: 2.0201e-04
Epoch 325/350
31/31  **0s** 4ms/step - loss: 2.1024e-04 - val_loss: 2.0201e-04
Epoch 326/350
31/31  **0s** 3ms/step - loss: 2.0268e-04 - val_loss: 2.0183e-04
Epoch 327/350
31/31  **0s** 4ms/step - loss: 1.9721e-04 - val_loss: 2.0176e-04
Epoch 328/350
31/31  **0s** 3ms/step - loss: 1.9514e-04 - val_loss: 2.0172e-04
Epoch 329/350
31/31  **0s** 4ms/step - loss: 2.0991e-04 - val_loss: 2.0149e-04
Epoch 330/350
31/31  **0s** 4ms/step - loss: 2.0969e-04 - val_loss: 2.0131e-04
Epoch 331/350
31/31  **0s** 3ms/step - loss: 2.0704e-04 - val_loss: 2.0117e-04
Epoch 332/350
31/31  **0s** 4ms/step - loss: 1.9152e-04 - val_loss: 2.0106e-04
Epoch 333/350
31/31  **0s** 4ms/step - loss: 2.0893e-04 - val_loss: 2.0106e-04
Epoch 334/350
31/31  **0s** 4ms/step - loss: 1.9503e-04 - val_loss: 2.0088e-04
Epoch 335/350
31/31  **0s** 3ms/step - loss: 2.0499e-04 - val_loss: 2.0079e-04
Epoch 336/350
31/31  **0s** 4ms/step - loss: 2.0441e-04 - val_loss: 2.0082e-04
Epoch 337/350
31/31  **0s** 4ms/step - loss: 2.1289e-04 - val_loss: 2.0060e-04
Epoch 338/350
31/31  **0s** 4ms/step - loss: 2.1388e-04 - val_loss: 2.0054e-04
Epoch 339/350
31/31  **0s** 4ms/step - loss: 2.0253e-04 - val_loss: 2.0033e-04
Epoch 340/350
31/31  **0s** 4ms/step - loss: 2.2594e-04 - val_loss: 2.0024e-04

```

Epoch 341/350
31/31 ————— 0s 4ms/step - loss: 1.9862e-04 - val_loss:
2.0020e-04
Epoch 342/350
31/31 ————— 0s 4ms/step - loss: 1.9603e-04 - val_loss:
2.0020e-04
Epoch 343/350
31/31 ————— 0s 4ms/step - loss: 2.0171e-04 - val_loss:
2.0010e-04
Epoch 344/350
31/31 ————— 0s 4ms/step - loss: 2.1142e-04 - val_loss:
1.9984e-04
Epoch 345/350
31/31 ————— 0s 4ms/step - loss: 2.0991e-04 - val_loss:
1.9982e-04
Epoch 346/350
31/31 ————— 0s 4ms/step - loss: 1.9801e-04 - val_loss:
1.9965e-04
Epoch 347/350
31/31 ————— 0s 4ms/step - loss: 2.0323e-04 - val_loss:
1.9969e-04
Epoch 348/350
31/31 ————— 0s 4ms/step - loss: 2.1097e-04 - val_loss:
1.9958e-04
Epoch 349/350
31/31 ————— 0s 3ms/step - loss: 1.8777e-04 - val_loss:
1.9949e-04
Epoch 350/350
31/31 ————— 0s 4ms/step - loss: 1.9985e-04 - val_loss:
1.9914e-04

```

```
Out[12]: <keras.src.callbacks.history.History at 0x7968caeea010>
```

We can analyze the performance of the linear autoencoder by extracting the reconstruction error of the model in the training and test samples. We compute, similar to computing $1 - R^2$ in regression settings, the proportion of the variance of the inputs that is left unexplained by the model.

Remember that the lower bound on the in-sample reconstruction error, using a linear autoencoder and an MSE loss, is set by the sum of the principal eigenvalues $k + 1, \dots, n$. From above, we obtained that the first three principal components explain roughly 67% of the variation in the data. The trained autoencoder has a reconstruction error of 32%, meaning that the error is close to the minimum possible when using a three-dimensional encoding.

Importantly, notice that we are using a validation sample with an early stopping criterion to avoid overfitting. This means that training may stop before the lower bound is achieved, so we should not necessarily reach the PCA's minimum reconstruction error using the trained autoencoder.

Below, we report the reconstruction error of the model in the training set and validation set separately. The results depend on the random seed that initializes the training algorithm. However, after several runs, the model should display a relatively good out-of-sample performance in terms of reconstruction error.

```
In [13]: Y_pred = model.predict(Y_train, verbose=0)
error = np.sum((Y_train - Y_pred) ** 2) / np.sum((Y_train -
np.mean(Y_train)) ** 2)
print("Reconstruction error in training sample", error)
Y_pred = model.predict(Y_test, verbose=0)
error_oos = np.sum((Y_test - Y_pred) ** 2) / np.sum((Y_test -
np.mean(Y_test)) ** 2)
print("Reconstruction error in test sample", error_oos)
```

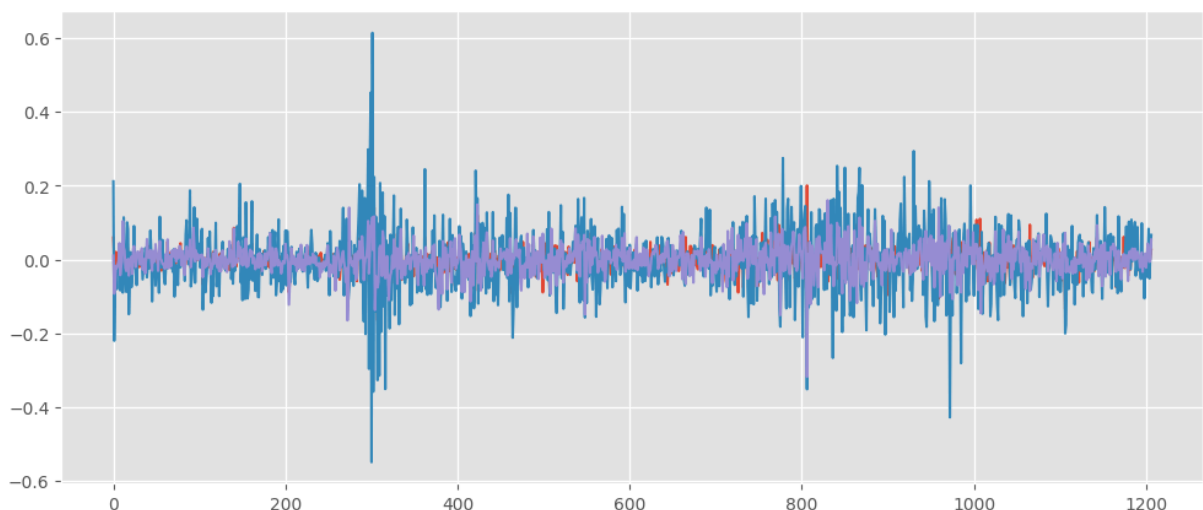
Reconstruction error in training sample 0.3179451109621889
Reconstruction error in test sample 0.4889370196068262

We can recover the time series of each latent factor generated by the autoencoder. Notice that the latent factors recovered by the autoencoder are correlated, as opposed to the PCA factors. Besides, the autoencoder factors are not ordered in terms of total variation.

```
In [14]: factors = model.encoder(Y_train).numpy()
factors_test = model.encoder(Y_test).numpy()
factors_cov = np.cov(factors, rowvar=False)
print(factors_cov)
```

```
[[0.00104645 0.00133243 0.0003107 ]
 [0.00133243 0.00750614 0.00162256]
 [0.0003107  0.00162256 0.00160836]]
```

```
In [15]: # Plot the evolution of latent factors in the training period
plt.figure(figsize=(12, 5))
plt.plot(factors)
plt.show()
```



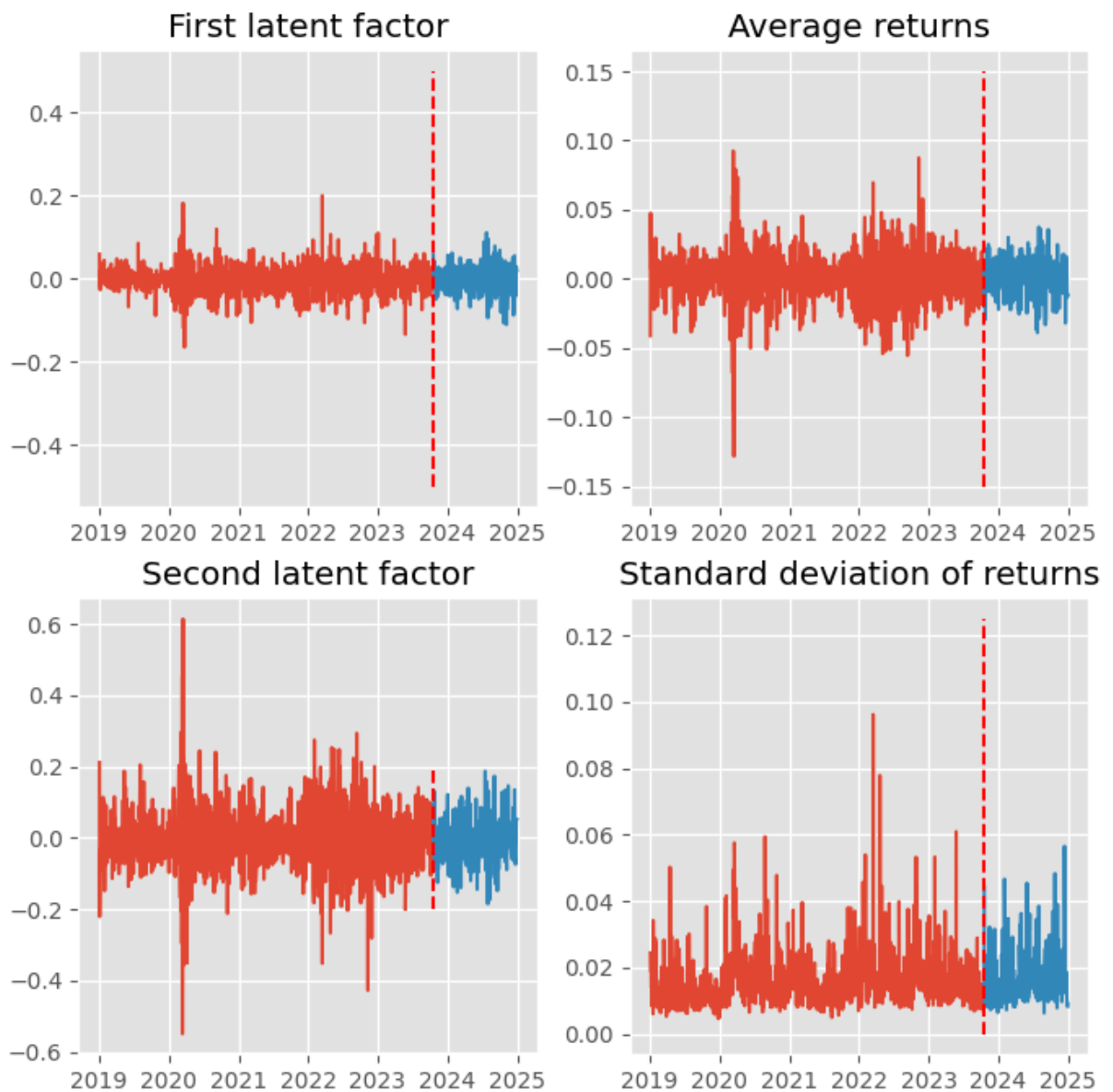
Although the results may vary depending on the seed, the highest-variance latent factor recovered by the autoencoder tends to capture the average level of returns across the stocks considered. The second latent factor in terms of variance seems to capture variations in the cross-sectional standard deviation of returns. That is, when the second factor takes a high value it seems that returns across stocks are more disperse.

Note: The red dashed line denotes the end of the training sample.

```

In [16]: fig, axs = plt.subplots(2, 2, figsize=(8, 8))
# First factor
axs[0, 0].plot(df.index[: train_size + 1], factors[:, 0])
axs[0, 0].plot(df.index[train_size + 1 :], factors_test[:, 0])
axs[0, 0].plot([df.index[train_size + 1], df.index[train_size + 1]],
[-0.5, 0.5], "r--")
axs[0, 0].title.set_text("First latent factor")
# Average of returns
axs[0, 1].plot(df.index[: train_size + 1], np.mean(Y_train, axis=1))
axs[0, 1].plot(df.index[train_size + 1 :], np.mean(Y_test, axis=1))
axs[0, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [-0.15, 0.15],
    "r--"
)
axs[0, 1].title.set_text("Average returns")
# Second factor
axs[1, 0].plot(df.index[: train_size + 1], factors[:, 1])
axs[1, 0].plot(df.index[train_size + 1 :], factors_test[:, 1])
axs[1, 0].plot([df.index[train_size + 1], df.index[train_size + 1]],
[-0.2, 0.2], "r--")
axs[1, 0].title.set_text("Second latent factor")
# Standard deviation of returns
axs[1, 1].plot(df.index[: train_size + 1], np.std(Y_train, axis=1))
axs[1, 1].plot(df.index[train_size + 1 :], np.std(Y_test, axis=1))
axs[1, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [0.0, 0.125],
    "r--"
)
axs[1, 1].title.set_text("Standard deviation of returns")

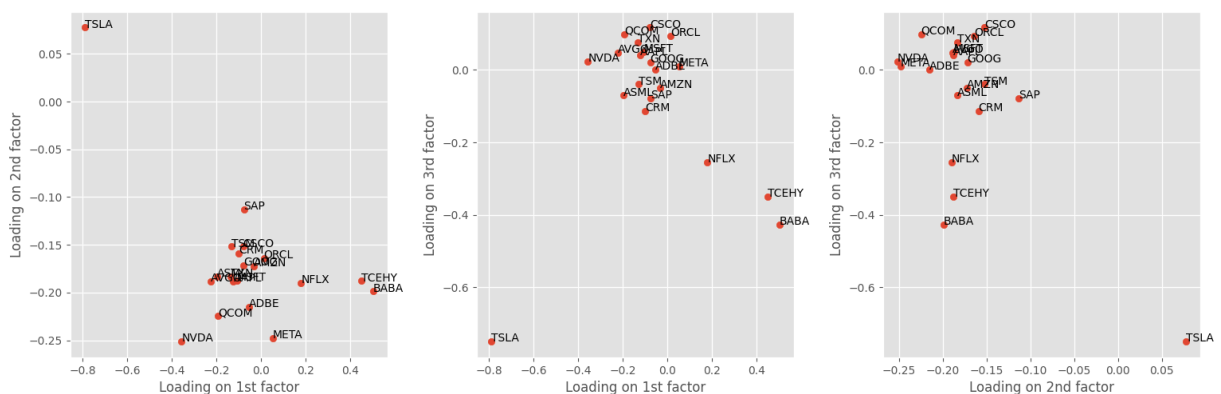
```



How is each stock in our sample exposed to each of the latent factors extracted by the autoencoder? This exposure is measured by each stock's loadings (the trainable weights in the output layer). Below, we generate scatter plots that display the relationship between the loadings of each stock on each pair of factors.

Noticeably, Tesla represents a clear outlier, a results that tends to appear regardless of the random seed that is used in the training algorithm. This suggests that a large portion of the cross-sectional variation in stock returns is actually driven by the behavior of Tesla's stock price. In other words, Tesla's stock price displays a behavior of its own that is separate from other stocks.

```
In [17]: # Extract loadings generated by the autoencoder
w_decoder = model.decoder.layers[0].get_weights()[0]
fig, axs = plt.subplots(1, 3, figsize=(16, 6))
fig.tight_layout(pad=5.0)
axs[0].scatter(w_decoder[0], w_decoder[1])
axs[0].set_ylabel("Loading on 2nd factor")
axs[0].set_xlabel("Loading on 1st factor")
axs[1].scatter(w_decoder[0], w_decoder[2])
axs[1].set_ylabel("Loading on 3rd factor")
axs[1].set_xlabel("Loading on 1st factor")
axs[2].scatter(w_decoder[1], w_decoder[2])
axs[2].set_ylabel("Loading on 3rd factor")
axs[2].set_xlabel("Loading on 2nd factor")
for i, txt in enumerate(tickers):
    axs[0].annotate(txt, (w_decoder[0, i], w_decoder[1, i]))
    axs[1].annotate(txt, (w_decoder[0, i], w_decoder[2, i]))
    axs[2].annotate(txt, (w_decoder[1, i], w_decoder[2, i]))
```



4. AN AUTOENCODER WITH NON-LINEAR ACTIVATIONS

In the blocks of code below we build and train an autoencoder that includes two additional hidden layers, one on each side of the bottleneck, with non-linear activations. The trained model displays slightly less reconstruction error than the linear benchmark in the training sample, although the error in the test sample is approximately the same.

The non-linear autoencoder may be able to capture richer features than the linear version. However, it becomes harder to grasp the type of variation that each latent factor is recovering from the data. As we show below in the figures, the factors are not so tightly related to average returns or dispersion, while the presence of an intermediate hidden layer makes it hard to grasp the exposure of each stock to the latent factors. Thus, adding non-linearities and hidden layers may enhance the explanatory ability of our models, but we likely lose a great deal of interpretability.

```
In [18]: class autoencoder_nonlinear(tf.keras.Model):
    def __init__(self, output_dim, bottleneck_dim=1):
        super(autoencoder_nonlinear, self).__init__()
        self.output_dim = output_dim
        self.bottleneck_dim = bottleneck_dim
        act_fun = "relu"
        self.encoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(12, activation=act_fun),
                tf.keras.layers.Dense(bottleneck_dim),
            ]
        )
        self.decoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(12, activation=act_fun),
                tf.keras.layers.Dense(output_dim),
            ]
        )

    def call(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded
```

```
In [19]: nlatent = 3
# to make this notebook's output stable
# np.random.seed(42)
# tf.random.set_seed(42)
val_split = 0.2
early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss",
patience=50)
model_nonlinear = autoencoder_nonlinear(Y_train.shape[1], nlatent)
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
model_nonlinear.compile(optimizer=adam, loss="mse")
model_nonlinear.fit(
    x=Y_train,
    y=Y_train,
    batch_size=32,
    epochs=350,
    validation_split=val_split,
    callbacks=[early_stop],
    verbose=1,
)
```

Epoch 1/350
31/31  **1s** 8ms/step - loss: 7.6684e-04 - val_loss: 5.6887e-04
Epoch 2/350
31/31  **0s** 4ms/step - loss: 6.7629e-04 - val_loss: 5.6141e-04
Epoch 3/350
31/31  **0s** 4ms/step - loss: 6.2947e-04 - val_loss: 5.5522e-04
Epoch 4/350
31/31  **0s** 4ms/step - loss: 6.3512e-04 - val_loss: 5.4929e-04
Epoch 5/350
31/31  **0s** 4ms/step - loss: 7.1331e-04 - val_loss: 5.4184e-04
Epoch 6/350
31/31  **0s** 4ms/step - loss: 6.2695e-04 - val_loss: 5.3452e-04
Epoch 7/350
31/31  **0s** 4ms/step - loss: 6.1891e-04 - val_loss: 5.2715e-04
Epoch 8/350
31/31  **0s** 4ms/step - loss: 5.8189e-04 - val_loss: 5.1989e-04
Epoch 9/350
31/31  **0s** 3ms/step - loss: 6.3155e-04 - val_loss: 5.1103e-04
Epoch 10/350
31/31  **0s** 4ms/step - loss: 6.0388e-04 - val_loss: 5.0307e-04
Epoch 11/350
31/31  **0s** 4ms/step - loss: 6.0035e-04 - val_loss: 4.9376e-04
Epoch 12/350
31/31  **0s** 4ms/step - loss: 6.0533e-04 - val_loss: 4.8527e-04
Epoch 13/350
31/31  **0s** 4ms/step - loss: 5.7830e-04 - val_loss: 4.7579e-04
Epoch 14/350
31/31  **0s** 4ms/step - loss: 5.5804e-04 - val_loss: 4.6718e-04
Epoch 15/350
31/31  **0s** 4ms/step - loss: 5.6468e-04 - val_loss: 4.5852e-04
Epoch 16/350
31/31  **0s** 3ms/step - loss: 5.5596e-04 - val_loss: 4.5038e-04
Epoch 17/350
31/31  **0s** 4ms/step - loss: 5.2570e-04 - val_loss: 4.4298e-04
Epoch 18/350
31/31  **0s** 4ms/step - loss: 5.4704e-04 - val_loss: 4.3462e-04
Epoch 19/350
31/31  **0s** 4ms/step - loss: 5.0911e-04 - val_loss: 4.2834e-04
Epoch 20/350
31/31  **0s** 4ms/step - loss: 4.6692e-04 - val_loss: 4.2099e-04

Epoch 21/350
31/31  0s 4ms/step - loss: 4.7658e-04 - val_loss: 4.1451e-04
Epoch 22/350
31/31  0s 3ms/step - loss: 5.0085e-04 - val_loss: 4.0858e-04
Epoch 23/350
31/31  0s 4ms/step - loss: 4.9184e-04 - val_loss: 4.0117e-04
Epoch 24/350
31/31  0s 4ms/step - loss: 4.7263e-04 - val_loss: 3.9622e-04
Epoch 25/350
31/31  0s 4ms/step - loss: 4.4502e-04 - val_loss: 3.9005e-04
Epoch 26/350
31/31  0s 4ms/step - loss: 4.5332e-04 - val_loss: 3.8408e-04
Epoch 27/350
31/31  0s 4ms/step - loss: 4.4688e-04 - val_loss: 3.7942e-04
Epoch 28/350
31/31  0s 4ms/step - loss: 4.1738e-04 - val_loss: 3.7461e-04
Epoch 29/350
31/31  0s 3ms/step - loss: 4.2692e-04 - val_loss: 3.7044e-04
Epoch 30/350
31/31  0s 4ms/step - loss: 4.4853e-04 - val_loss: 3.6541e-04
Epoch 31/350
31/31  0s 4ms/step - loss: 3.8681e-04 - val_loss: 3.6099e-04
Epoch 32/350
31/31  0s 4ms/step - loss: 3.9449e-04 - val_loss: 3.5782e-04
Epoch 33/350
31/31  0s 4ms/step - loss: 3.8867e-04 - val_loss: 3.5331e-04
Epoch 34/350
31/31  0s 4ms/step - loss: 3.9347e-04 - val_loss: 3.4975e-04
Epoch 35/350
31/31  0s 4ms/step - loss: 3.8846e-04 - val_loss: 3.4636e-04
Epoch 36/350
31/31  0s 4ms/step - loss: 3.9032e-04 - val_loss: 3.4285e-04
Epoch 37/350
31/31  0s 4ms/step - loss: 3.8907e-04 - val_loss: 3.4014e-04
Epoch 38/350
31/31  0s 4ms/step - loss: 3.9369e-04 - val_loss: 3.3724e-04
Epoch 39/350
31/31  0s 4ms/step - loss: 3.5860e-04 - val_loss: 3.3363e-04
Epoch 40/350
31/31  0s 4ms/step - loss: 3.4960e-04 - val_loss: 3.3139e-04

Epoch 41/350
31/31  0s 6ms/step - loss: 3.9924e-04 - val_loss: 3.2752e-04
Epoch 42/350
31/31  0s 6ms/step - loss: 3.5631e-04 - val_loss: 3.2494e-04
Epoch 43/350
31/31  0s 6ms/step - loss: 3.3630e-04 - val_loss: 3.2365e-04
Epoch 44/350
31/31  0s 6ms/step - loss: 3.5517e-04 - val_loss: 3.2041e-04
Epoch 45/350
31/31  0s 6ms/step - loss: 3.2983e-04 - val_loss: 3.1884e-04
Epoch 46/350
31/31  0s 5ms/step - loss: 3.5246e-04 - val_loss: 3.1634e-04
Epoch 47/350
31/31  0s 6ms/step - loss: 3.5616e-04 - val_loss: 3.1338e-04
Epoch 48/350
31/31  0s 3ms/step - loss: 3.4471e-04 - val_loss: 3.1196e-04
Epoch 49/350
31/31  0s 4ms/step - loss: 3.4331e-04 - val_loss: 3.1018e-04
Epoch 50/350
31/31  0s 4ms/step - loss: 3.3593e-04 - val_loss: 3.0859e-04
Epoch 51/350
31/31  0s 3ms/step - loss: 3.2970e-04 - val_loss: 3.0693e-04
Epoch 52/350
31/31  0s 5ms/step - loss: 3.3326e-04 - val_loss: 3.0488e-04
Epoch 53/350
31/31  0s 5ms/step - loss: 3.2406e-04 - val_loss: 3.0272e-04
Epoch 54/350
31/31  0s 5ms/step - loss: 3.2708e-04 - val_loss: 3.0145e-04
Epoch 55/350
31/31  0s 5ms/step - loss: 3.2104e-04 - val_loss: 3.0047e-04
Epoch 56/350
31/31  0s 3ms/step - loss: 3.3432e-04 - val_loss: 2.9948e-04
Epoch 57/350
31/31  0s 4ms/step - loss: 3.3435e-04 - val_loss: 2.9625e-04
Epoch 58/350
31/31  0s 4ms/step - loss: 3.3143e-04 - val_loss: 2.9559e-04
Epoch 59/350
31/31  0s 4ms/step - loss: 3.5125e-04 - val_loss: 2.9447e-04
Epoch 60/350
31/31  0s 4ms/step - loss: 3.0695e-04 - val_loss: 2.9338e-04

Epoch 61/350
31/31  0s 4ms/step - loss: 3.0815e-04 - val_loss: 2.9146e-04
Epoch 62/350
31/31  0s 4ms/step - loss: 3.0087e-04 - val_loss: 2.9084e-04
Epoch 63/350
31/31  0s 4ms/step - loss: 3.2577e-04 - val_loss: 2.8957e-04
Epoch 64/350
31/31  0s 4ms/step - loss: 3.2133e-04 - val_loss: 2.8841e-04
Epoch 65/350
31/31  0s 6ms/step - loss: 3.0689e-04 - val_loss: 2.8729e-04
Epoch 66/350
31/31  0s 4ms/step - loss: 2.9215e-04 - val_loss: 2.8597e-04
Epoch 67/350
31/31  0s 4ms/step - loss: 3.2651e-04 - val_loss: 2.8513e-04
Epoch 68/350
31/31  0s 4ms/step - loss: 3.0124e-04 - val_loss: 2.8415e-04
Epoch 69/350
31/31  0s 4ms/step - loss: 3.1816e-04 - val_loss: 2.8278e-04
Epoch 70/350
31/31  0s 4ms/step - loss: 2.8879e-04 - val_loss: 2.8250e-04
Epoch 71/350
31/31  0s 4ms/step - loss: 3.0999e-04 - val_loss: 2.8091e-04
Epoch 72/350
31/31  0s 4ms/step - loss: 2.8722e-04 - val_loss: 2.7955e-04
Epoch 73/350
31/31  0s 4ms/step - loss: 2.9748e-04 - val_loss: 2.7961e-04
Epoch 74/350
31/31  0s 4ms/step - loss: 3.1717e-04 - val_loss: 2.7781e-04
Epoch 75/350
31/31  0s 4ms/step - loss: 3.1093e-04 - val_loss: 2.7685e-04
Epoch 76/350
31/31  0s 4ms/step - loss: 2.8151e-04 - val_loss: 2.7589e-04
Epoch 77/350
31/31  0s 4ms/step - loss: 2.6536e-04 - val_loss: 2.7607e-04
Epoch 78/350
31/31  0s 4ms/step - loss: 2.6969e-04 - val_loss: 2.7446e-04
Epoch 79/350
31/31  0s 4ms/step - loss: 3.0753e-04 - val_loss: 2.7360e-04
Epoch 80/350
31/31  0s 4ms/step - loss: 2.9730e-04 - val_loss: 2.7245e-04

Epoch 81/350
31/31  **0s** 4ms/step - loss: 3.1307e-04 - val_loss: 2.7207e-04

Epoch 82/350
31/31  **0s** 4ms/step - loss: 3.1436e-04 - val_loss: 2.7078e-04

Epoch 83/350
31/31  **0s** 3ms/step - loss: 2.6424e-04 - val_loss: 2.7041e-04

Epoch 84/350
31/31  **0s** 4ms/step - loss: 2.8041e-04 - val_loss: 2.6943e-04

Epoch 85/350
31/31  **0s** 5ms/step - loss: 3.0792e-04 - val_loss: 2.6865e-04

Epoch 86/350
31/31  **0s** 6ms/step - loss: 2.8270e-04 - val_loss: 2.6755e-04

Epoch 87/350
31/31  **0s** 5ms/step - loss: 2.8223e-04 - val_loss: 2.6747e-04

Epoch 88/350
31/31  **0s** 4ms/step - loss: 2.6932e-04 - val_loss: 2.6609e-04

Epoch 89/350
31/31  **0s** 4ms/step - loss: 2.6900e-04 - val_loss: 2.6555e-04

Epoch 90/350
31/31  **0s** 4ms/step - loss: 2.9153e-04 - val_loss: 2.6506e-04

Epoch 91/350
31/31  **0s** 4ms/step - loss: 2.7369e-04 - val_loss: 2.6403e-04

Epoch 92/350
31/31  **0s** 4ms/step - loss: 2.8406e-04 - val_loss: 2.6378e-04

Epoch 93/350
31/31  **0s** 4ms/step - loss: 2.8459e-04 - val_loss: 2.6255e-04

Epoch 94/350
31/31  **0s** 4ms/step - loss: 2.6245e-04 - val_loss: 2.6210e-04

Epoch 95/350
31/31  **0s** 4ms/step - loss: 2.7069e-04 - val_loss: 2.6169e-04

Epoch 96/350
31/31  **0s** 4ms/step - loss: 2.7910e-04 - val_loss: 2.6110e-04

Epoch 97/350
31/31  **0s** 4ms/step - loss: 2.6897e-04 - val_loss: 2.5982e-04

Epoch 98/350
31/31  **0s** 3ms/step - loss: 2.7645e-04 - val_loss: 2.5920e-04

Epoch 99/350
31/31  **0s** 5ms/step - loss: 2.8929e-04 - val_loss: 2.5857e-04

Epoch 100/350
31/31  **0s** 4ms/step - loss: 2.6989e-04 - val_loss: 2.5787e-04

Epoch 101/350
31/31  **0s** 3ms/step - loss: 2.7571e-04 - val_loss: 2.5751e-04

Epoch 102/350
31/31  **0s** 4ms/step - loss: 2.8700e-04 - val_loss: 2.5659e-04

Epoch 103/350
31/31  **0s** 4ms/step - loss: 2.8108e-04 - val_loss: 2.5594e-04

Epoch 104/350
31/31  **0s** 4ms/step - loss: 2.5291e-04 - val_loss: 2.5524e-04

Epoch 105/350
31/31  **0s** 4ms/step - loss: 2.8573e-04 - val_loss: 2.5398e-04

Epoch 106/350
31/31  **0s** 4ms/step - loss: 2.4930e-04 - val_loss: 2.5386e-04

Epoch 107/350
31/31  **0s** 4ms/step - loss: 2.7561e-04 - val_loss: 2.5356e-04

Epoch 108/350
31/31  **0s** 4ms/step - loss: 2.8838e-04 - val_loss: 2.5242e-04

Epoch 109/350
31/31  **0s** 4ms/step - loss: 2.5744e-04 - val_loss: 2.5138e-04

Epoch 110/350
31/31  **0s** 4ms/step - loss: 2.7010e-04 - val_loss: 2.5086e-04

Epoch 111/350
31/31  **0s** 4ms/step - loss: 2.7774e-04 - val_loss: 2.4990e-04

Epoch 112/350
31/31  **0s** 4ms/step - loss: 2.5348e-04 - val_loss: 2.4928e-04

Epoch 113/350
31/31  **0s** 3ms/step - loss: 2.6336e-04 - val_loss: 2.4857e-04

Epoch 114/350
31/31  **0s** 4ms/step - loss: 2.4283e-04 - val_loss: 2.4824e-04

Epoch 115/350
31/31  **0s** 4ms/step - loss: 2.7206e-04 - val_loss: 2.4728e-04

Epoch 116/350
31/31  **0s** 4ms/step - loss: 2.6939e-04 - val_loss: 2.4638e-04

Epoch 117/350
31/31  **0s** 4ms/step - loss: 2.5857e-04 - val_loss: 2.4606e-04

Epoch 118/350
31/31  **0s** 4ms/step - loss: 2.7000e-04 - val_loss: 2.4530e-04

Epoch 119/350
31/31  **0s** 3ms/step - loss: 2.3783e-04 - val_loss: 2.4441e-04

Epoch 120/350
31/31  **0s** 4ms/step - loss: 2.6705e-04 - val_loss: 2.4402e-04

Epoch 121/350
31/31  0s 4ms/step - loss: 2.6339e-04 - val_loss: 2.4299e-04
Epoch 122/350
31/31  0s 4ms/step - loss: 2.6074e-04 - val_loss: 2.4203e-04
Epoch 123/350
31/31  0s 4ms/step - loss: 2.5216e-04 - val_loss: 2.4129e-04
Epoch 124/350
31/31  0s 4ms/step - loss: 2.3659e-04 - val_loss: 2.4069e-04
Epoch 125/350
31/31  0s 4ms/step - loss: 2.5897e-04 - val_loss: 2.3959e-04
Epoch 126/350
31/31  0s 4ms/step - loss: 2.5349e-04 - val_loss: 2.3883e-04
Epoch 127/350
31/31  0s 4ms/step - loss: 2.4613e-04 - val_loss: 2.3795e-04
Epoch 128/350
31/31  0s 4ms/step - loss: 2.6000e-04 - val_loss: 2.3687e-04
Epoch 129/350
31/31  0s 4ms/step - loss: 2.3346e-04 - val_loss: 2.3625e-04
Epoch 130/350
31/31  0s 4ms/step - loss: 2.5746e-04 - val_loss: 2.3553e-04
Epoch 131/350
31/31  0s 4ms/step - loss: 2.5393e-04 - val_loss: 2.3473e-04
Epoch 132/350
31/31  0s 4ms/step - loss: 2.6801e-04 - val_loss: 2.3370e-04
Epoch 133/350
31/31  0s 3ms/step - loss: 2.4719e-04 - val_loss: 2.3242e-04
Epoch 134/350
31/31  0s 4ms/step - loss: 2.3875e-04 - val_loss: 2.3209e-04
Epoch 135/350
31/31  0s 4ms/step - loss: 2.2582e-04 - val_loss: 2.3093e-04
Epoch 136/350
31/31  0s 4ms/step - loss: 2.4557e-04 - val_loss: 2.3052e-04
Epoch 137/350
31/31  0s 5ms/step - loss: 2.5404e-04 - val_loss: 2.2927e-04
Epoch 138/350
31/31  0s 3ms/step - loss: 2.7054e-04 - val_loss: 2.2855e-04
Epoch 139/350
31/31  0s 3ms/step - loss: 2.5051e-04 - val_loss: 2.2799e-04
Epoch 140/350
31/31  0s 4ms/step - loss: 2.4993e-04 - val_loss: 2.2683e-04

Epoch 141/350
31/31  **0s** 4ms/step - loss: 2.5848e-04 - val_loss: 2.2593e-04

Epoch 142/350
31/31  **0s** 4ms/step - loss: 2.2620e-04 - val_loss: 2.2545e-04

Epoch 143/350
31/31  **0s** 4ms/step - loss: 2.4727e-04 - val_loss: 2.2433e-04

Epoch 144/350
31/31  **0s** 4ms/step - loss: 2.3988e-04 - val_loss: 2.2371e-04

Epoch 145/350
31/31  **0s** 4ms/step - loss: 2.6667e-04 - val_loss: 2.2333e-04

Epoch 146/350
31/31  **0s** 5ms/step - loss: 2.3766e-04 - val_loss: 2.2209e-04

Epoch 147/350
31/31  **0s** 4ms/step - loss: 2.3444e-04 - val_loss: 2.2155e-04

Epoch 148/350
31/31  **0s** 4ms/step - loss: 2.1763e-04 - val_loss: 2.2096e-04

Epoch 149/350
31/31  **0s** 4ms/step - loss: 2.3864e-04 - val_loss: 2.1989e-04

Epoch 150/350
31/31  **0s** 4ms/step - loss: 2.3049e-04 - val_loss: 2.1913e-04

Epoch 151/350
31/31  **0s** 4ms/step - loss: 2.2448e-04 - val_loss: 2.1825e-04

Epoch 152/350
31/31  **0s** 4ms/step - loss: 2.5715e-04 - val_loss: 2.1774e-04

Epoch 153/350
31/31  **0s** 4ms/step - loss: 2.2150e-04 - val_loss: 2.1705e-04

Epoch 154/350
31/31  **0s** 4ms/step - loss: 2.2644e-04 - val_loss: 2.1627e-04

Epoch 155/350
31/31  **0s** 4ms/step - loss: 2.2553e-04 - val_loss: 2.1573e-04

Epoch 156/350
31/31  **0s** 4ms/step - loss: 2.3170e-04 - val_loss: 2.1511e-04

Epoch 157/350
31/31  **0s** 4ms/step - loss: 2.4861e-04 - val_loss: 2.1423e-04

Epoch 158/350
31/31  **0s** 4ms/step - loss: 2.2992e-04 - val_loss: 2.1336e-04

Epoch 159/350
31/31  **0s** 3ms/step - loss: 2.3305e-04 - val_loss: 2.1322e-04

Epoch 160/350
31/31  **0s** 4ms/step - loss: 2.3245e-04 - val_loss: 2.1255e-04

Epoch 161/350
31/31  0s 4ms/step - loss: 2.3323e-04 - val_loss: 2.1236e-04

Epoch 162/350
31/31  0s 4ms/step - loss: 2.1822e-04 - val_loss: 2.1148e-04

Epoch 163/350
31/31  0s 4ms/step - loss: 2.1644e-04 - val_loss: 2.1089e-04

Epoch 164/350
31/31  0s 4ms/step - loss: 2.0697e-04 - val_loss: 2.1060e-04

Epoch 165/350
31/31  0s 4ms/step - loss: 2.1217e-04 - val_loss: 2.0971e-04

Epoch 166/350
31/31  0s 4ms/step - loss: 2.2394e-04 - val_loss: 2.0958e-04

Epoch 167/350
31/31  0s 4ms/step - loss: 2.1272e-04 - val_loss: 2.0917e-04

Epoch 168/350
31/31  0s 4ms/step - loss: 2.1337e-04 - val_loss: 2.0839e-04

Epoch 169/350
31/31  0s 4ms/step - loss: 2.2021e-04 - val_loss: 2.0805e-04

Epoch 170/350
31/31  0s 4ms/step - loss: 2.2408e-04 - val_loss: 2.0739e-04

Epoch 171/350
31/31  0s 4ms/step - loss: 2.0560e-04 - val_loss: 2.0678e-04

Epoch 172/350
31/31  0s 4ms/step - loss: 2.3028e-04 - val_loss: 2.0696e-04

Epoch 173/350
31/31  0s 4ms/step - loss: 2.2943e-04 - val_loss: 2.0662e-04

Epoch 174/350
31/31  0s 4ms/step - loss: 2.0895e-04 - val_loss: 2.0614e-04

Epoch 175/350
31/31  0s 4ms/step - loss: 2.3674e-04 - val_loss: 2.0590e-04

Epoch 176/350
31/31  0s 4ms/step - loss: 2.3880e-04 - val_loss: 2.0478e-04

Epoch 177/350
31/31  0s 4ms/step - loss: 2.0375e-04 - val_loss: 2.0523e-04

Epoch 178/350
31/31  0s 4ms/step - loss: 2.5151e-04 - val_loss: 2.0448e-04

Epoch 179/350
31/31  0s 4ms/step - loss: 2.2004e-04 - val_loss: 2.0424e-04

Epoch 180/350
31/31  0s 4ms/step - loss: 2.1041e-04 - val_loss: 2.0353e-04

Epoch 181/350
31/31  **0s** 3ms/step - loss: 2.1078e-04 - val_loss: 2.0353e-04

Epoch 182/350
31/31  **0s** 5ms/step - loss: 2.0853e-04 - val_loss: 2.0333e-04

Epoch 183/350
31/31  **0s** 3ms/step - loss: 2.1251e-04 - val_loss: 2.0316e-04

Epoch 184/350
31/31  **0s** 4ms/step - loss: 2.2410e-04 - val_loss: 2.0262e-04

Epoch 185/350
31/31  **0s** 4ms/step - loss: 2.1171e-04 - val_loss: 2.0160e-04

Epoch 186/350
31/31  **0s** 4ms/step - loss: 2.1867e-04 - val_loss: 2.0206e-04

Epoch 187/350
31/31  **0s** 3ms/step - loss: 2.4135e-04 - val_loss: 2.0151e-04

Epoch 188/350
31/31  **0s** 4ms/step - loss: 2.1081e-04 - val_loss: 2.0120e-04

Epoch 189/350
31/31  **0s** 3ms/step - loss: 2.3482e-04 - val_loss: 2.0101e-04

Epoch 190/350
31/31  **0s** 4ms/step - loss: 2.4107e-04 - val_loss: 2.0070e-04

Epoch 191/350
31/31  **0s** 4ms/step - loss: 2.0674e-04 - val_loss: 2.0036e-04

Epoch 192/350
31/31  **0s** 4ms/step - loss: 2.0915e-04 - val_loss: 2.0060e-04

Epoch 193/350
31/31  **0s** 4ms/step - loss: 2.1824e-04 - val_loss: 2.0027e-04

Epoch 194/350
31/31  **0s** 4ms/step - loss: 2.0515e-04 - val_loss: 1.9937e-04

Epoch 195/350
31/31  **0s** 4ms/step - loss: 1.9874e-04 - val_loss: 1.9950e-04

Epoch 196/350
31/31  **0s** 4ms/step - loss: 2.0570e-04 - val_loss: 1.9918e-04

Epoch 197/350
31/31  **0s** 4ms/step - loss: 2.1135e-04 - val_loss: 1.9897e-04

Epoch 198/350
31/31  **0s** 4ms/step - loss: 2.0553e-04 - val_loss: 1.9887e-04

Epoch 199/350
31/31  **0s** 4ms/step - loss: 2.2948e-04 - val_loss: 1.9851e-04

Epoch 200/350
31/31  **0s** 4ms/step - loss: 2.1294e-04 - val_loss: 1.9835e-04

Epoch 201/350
31/31  **0s** 5ms/step - loss: 2.1422e-04 - val_loss: 1.9769e-04

Epoch 202/350
31/31  **0s** 4ms/step - loss: 2.1610e-04 - val_loss: 1.9801e-04

Epoch 203/350
31/31  **0s** 4ms/step - loss: 2.1306e-04 - val_loss: 1.9771e-04

Epoch 204/350
31/31  **0s** 4ms/step - loss: 2.1151e-04 - val_loss: 1.9732e-04

Epoch 205/350
31/31  **0s** 3ms/step - loss: 2.1114e-04 - val_loss: 1.9735e-04

Epoch 206/350
31/31  **0s** 4ms/step - loss: 2.1810e-04 - val_loss: 1.9716e-04

Epoch 207/350
31/31  **0s** 4ms/step - loss: 2.3451e-04 - val_loss: 1.9690e-04

Epoch 208/350
31/31  **0s** 4ms/step - loss: 2.0838e-04 - val_loss: 1.9668e-04

Epoch 209/350
31/31  **0s** 4ms/step - loss: 2.0958e-04 - val_loss: 1.9666e-04

Epoch 210/350
31/31  **0s** 3ms/step - loss: 2.1117e-04 - val_loss: 1.9639e-04

Epoch 211/350
31/31  **0s** 4ms/step - loss: 2.0318e-04 - val_loss: 1.9612e-04

Epoch 212/350
31/31  **0s** 4ms/step - loss: 2.1623e-04 - val_loss: 1.9563e-04

Epoch 213/350
31/31  **0s** 5ms/step - loss: 2.1122e-04 - val_loss: 1.9611e-04

Epoch 214/350
31/31  **0s** 4ms/step - loss: 2.0964e-04 - val_loss: 1.9558e-04

Epoch 215/350
31/31  **0s** 4ms/step - loss: 2.0865e-04 - val_loss: 1.9563e-04

Epoch 216/350
31/31  **0s** 3ms/step - loss: 1.9419e-04 - val_loss: 1.9535e-04

Epoch 217/350
31/31  **0s** 4ms/step - loss: 2.1475e-04 - val_loss: 1.9530e-04

Epoch 218/350
31/31  **0s** 4ms/step - loss: 2.1312e-04 - val_loss: 1.9541e-04

Epoch 219/350
31/31  **0s** 4ms/step - loss: 2.1069e-04 - val_loss: 1.9475e-04

Epoch 220/350
31/31  **0s** 3ms/step - loss: 2.1244e-04 - val_loss: 1.9465e-04

Epoch 221/350
31/31  **0s** 4ms/step - loss: 2.0752e-04 - val_loss: 1.9488e-04

Epoch 222/350
31/31  **0s** 4ms/step - loss: 2.2567e-04 - val_loss: 1.9507e-04

Epoch 223/350
31/31  **0s** 3ms/step - loss: 2.1963e-04 - val_loss: 1.9451e-04

Epoch 224/350
31/31  **0s** 4ms/step - loss: 2.0757e-04 - val_loss: 1.9438e-04

Epoch 225/350
31/31  **0s** 3ms/step - loss: 2.0296e-04 - val_loss: 1.9452e-04

Epoch 226/350
31/31  **0s** 4ms/step - loss: 2.0718e-04 - val_loss: 1.9388e-04

Epoch 227/350
31/31  **0s** 3ms/step - loss: 2.2052e-04 - val_loss: 1.9403e-04

Epoch 228/350
31/31  **0s** 4ms/step - loss: 2.1412e-04 - val_loss: 1.9360e-04

Epoch 229/350
31/31  **0s** 4ms/step - loss: 2.0990e-04 - val_loss: 1.9378e-04

Epoch 230/350
31/31  **0s** 4ms/step - loss: 1.9940e-04 - val_loss: 1.9390e-04

Epoch 231/350
31/31  **0s** 4ms/step - loss: 2.0434e-04 - val_loss: 1.9331e-04

Epoch 232/350
31/31  **0s** 3ms/step - loss: 2.2831e-04 - val_loss: 1.9321e-04

Epoch 233/350
31/31  **0s** 4ms/step - loss: 2.0902e-04 - val_loss: 1.9321e-04

Epoch 234/350
31/31  **0s** 4ms/step - loss: 2.0122e-04 - val_loss: 1.9331e-04

Epoch 235/350
31/31  **0s** 4ms/step - loss: 2.1467e-04 - val_loss: 1.9304e-04

Epoch 236/350
31/31  **0s** 4ms/step - loss: 2.0322e-04 - val_loss: 1.9314e-04

Epoch 237/350
31/31  **0s** 4ms/step - loss: 1.9376e-04 - val_loss: 1.9283e-04

Epoch 238/350
31/31  **0s** 4ms/step - loss: 2.0861e-04 - val_loss: 1.9266e-04

Epoch 239/350
31/31  **0s** 4ms/step - loss: 1.8477e-04 - val_loss: 1.9247e-04

Epoch 240/350
31/31  **0s** 4ms/step - loss: 2.1122e-04 - val_loss: 1.9269e-04

Epoch 241/350
31/31  0s 4ms/step - loss: 2.1183e-04 - val_loss: 1.9264e-04

Epoch 242/350
31/31  0s 4ms/step - loss: 2.0603e-04 - val_loss: 1.9244e-04

Epoch 243/350
31/31  0s 4ms/step - loss: 2.0928e-04 - val_loss: 1.9231e-04

Epoch 244/350
31/31  0s 4ms/step - loss: 2.2061e-04 - val_loss: 1.9208e-04

Epoch 245/350
31/31  0s 3ms/step - loss: 2.1300e-04 - val_loss: 1.9226e-04

Epoch 246/350
31/31  0s 4ms/step - loss: 2.0945e-04 - val_loss: 1.9201e-04

Epoch 247/350
31/31  0s 4ms/step - loss: 2.0921e-04 - val_loss: 1.9215e-04

Epoch 248/350
31/31  0s 4ms/step - loss: 2.0047e-04 - val_loss: 1.9198e-04

Epoch 249/350
31/31  0s 4ms/step - loss: 2.0061e-04 - val_loss: 1.9182e-04

Epoch 250/350
31/31  0s 4ms/step - loss: 1.9471e-04 - val_loss: 1.9159e-04

Epoch 251/350
31/31  0s 4ms/step - loss: 2.2025e-04 - val_loss: 1.9179e-04

Epoch 252/350
31/31  0s 4ms/step - loss: 2.1785e-04 - val_loss: 1.9166e-04

Epoch 253/350
31/31  0s 4ms/step - loss: 1.9310e-04 - val_loss: 1.9155e-04

Epoch 254/350
31/31  0s 4ms/step - loss: 1.9552e-04 - val_loss: 1.9153e-04

Epoch 255/350
31/31  0s 4ms/step - loss: 2.0099e-04 - val_loss: 1.9164e-04

Epoch 256/350
31/31  0s 3ms/step - loss: 1.9817e-04 - val_loss: 1.9139e-04

Epoch 257/350
31/31  0s 4ms/step - loss: 2.2082e-04 - val_loss: 1.9148e-04

Epoch 258/350
31/31  0s 3ms/step - loss: 2.1275e-04 - val_loss: 1.9137e-04

Epoch 259/350
31/31  0s 4ms/step - loss: 2.1050e-04 - val_loss: 1.9125e-04

Epoch 260/350
31/31  0s 3ms/step - loss: 2.0023e-04 - val_loss: 1.9105e-04

Epoch 261/350
31/31  **0s** 5ms/step - loss: 2.0978e-04 - val_loss: 1.9092e-04

Epoch 262/350
31/31  **0s** 4ms/step - loss: 2.1119e-04 - val_loss: 1.9122e-04

Epoch 263/350
31/31  **0s** 4ms/step - loss: 1.9421e-04 - val_loss: 1.9110e-04

Epoch 264/350
31/31  **0s** 4ms/step - loss: 2.2247e-04 - val_loss: 1.9086e-04

Epoch 265/350
31/31  **0s** 4ms/step - loss: 2.0417e-04 - val_loss: 1.9094e-04

Epoch 266/350
31/31  **0s** 4ms/step - loss: 2.1107e-04 - val_loss: 1.9087e-04

Epoch 267/350
31/31  **0s** 4ms/step - loss: 2.0984e-04 - val_loss: 1.9075e-04

Epoch 268/350
31/31  **0s** 4ms/step - loss: 2.0120e-04 - val_loss: 1.9068e-04

Epoch 269/350
31/31  **0s** 4ms/step - loss: 2.0953e-04 - val_loss: 1.9071e-04

Epoch 270/350
31/31  **0s** 4ms/step - loss: 2.1178e-04 - val_loss: 1.9047e-04

Epoch 271/350
31/31  **0s** 4ms/step - loss: 1.9452e-04 - val_loss: 1.9066e-04

Epoch 272/350
31/31  **0s** 4ms/step - loss: 2.1369e-04 - val_loss: 1.9071e-04

Epoch 273/350
31/31  **0s** 4ms/step - loss: 2.0784e-04 - val_loss: 1.9045e-04

Epoch 274/350
31/31  **0s** 4ms/step - loss: 2.0072e-04 - val_loss: 1.9014e-04

Epoch 275/350
31/31  **0s** 4ms/step - loss: 2.0140e-04 - val_loss: 1.9037e-04

Epoch 276/350
31/31  **0s** 4ms/step - loss: 2.0593e-04 - val_loss: 1.9065e-04

Epoch 277/350
31/31  **0s** 4ms/step - loss: 2.0505e-04 - val_loss: 1.9022e-04

Epoch 278/350
31/31  **0s** 4ms/step - loss: 1.9466e-04 - val_loss: 1.9002e-04

Epoch 279/350
31/31  **0s** 4ms/step - loss: 2.0033e-04 - val_loss: 1.9028e-04

Epoch 280/350
31/31  **0s** 4ms/step - loss: 1.8371e-04 - val_loss: 1.9030e-04

Epoch 281/350
31/31  **0s** 3ms/step - loss: 2.1231e-04 - val_loss: 1.8999e-04

Epoch 282/350
31/31  **0s** 4ms/step - loss: 1.9531e-04 - val_loss: 1.9046e-04

Epoch 283/350
31/31  **0s** 4ms/step - loss: 2.1411e-04 - val_loss: 1.9029e-04

Epoch 284/350
31/31  **0s** 4ms/step - loss: 2.1142e-04 - val_loss: 1.9007e-04

Epoch 285/350
31/31  **0s** 3ms/step - loss: 2.0217e-04 - val_loss: 1.9016e-04

Epoch 286/350
31/31  **0s** 4ms/step - loss: 2.1213e-04 - val_loss: 1.8995e-04

Epoch 287/350
31/31  **0s** 4ms/step - loss: 1.8802e-04 - val_loss: 1.8999e-04

Epoch 288/350
31/31  **0s** 4ms/step - loss: 2.0642e-04 - val_loss: 1.9021e-04

Epoch 289/350
31/31  **0s** 4ms/step - loss: 2.0092e-04 - val_loss: 1.9008e-04

Epoch 290/350
31/31  **0s** 4ms/step - loss: 1.9661e-04 - val_loss: 1.8975e-04

Epoch 291/350
31/31  **0s** 4ms/step - loss: 1.9533e-04 - val_loss: 1.8989e-04

Epoch 292/350
31/31  **0s** 4ms/step - loss: 2.1424e-04 - val_loss: 1.8968e-04

Epoch 293/350
31/31  **0s** 4ms/step - loss: 2.0287e-04 - val_loss: 1.8986e-04

Epoch 294/350
31/31  **0s** 4ms/step - loss: 2.0000e-04 - val_loss: 1.9010e-04

Epoch 295/350
31/31  **0s** 4ms/step - loss: 2.0037e-04 - val_loss: 1.8980e-04

Epoch 296/350
31/31  **0s** 4ms/step - loss: 2.0847e-04 - val_loss: 1.8980e-04

Epoch 297/350
31/31  **0s** 4ms/step - loss: 1.9677e-04 - val_loss: 1.8966e-04

Epoch 298/350
31/31  **0s** 4ms/step - loss: 1.9978e-04 - val_loss: 1.8975e-04

Epoch 299/350
31/31  **0s** 4ms/step - loss: 2.0915e-04 - val_loss: 1.8989e-04

Epoch 300/350
31/31  **0s** 4ms/step - loss: 1.9817e-04 - val_loss: 1.8949e-04

Epoch 301/350
31/31  **0s** 4ms/step - loss: 2.0183e-04 - val_loss: 1.8987e-04

Epoch 302/350
31/31  **0s** 4ms/step - loss: 2.1049e-04 - val_loss: 1.8973e-04

Epoch 303/350
31/31  **0s** 4ms/step - loss: 2.0544e-04 - val_loss: 1.8961e-04

Epoch 304/350
31/31  **0s** 4ms/step - loss: 1.9764e-04 - val_loss: 1.8942e-04

Epoch 305/350
31/31  **0s** 3ms/step - loss: 1.9379e-04 - val_loss: 1.8957e-04

Epoch 306/350
31/31  **0s** 4ms/step - loss: 2.0660e-04 - val_loss: 1.8942e-04

Epoch 307/350
31/31  **0s** 4ms/step - loss: 2.0000e-04 - val_loss: 1.8958e-04

Epoch 308/350
31/31  **0s** 3ms/step - loss: 1.9812e-04 - val_loss: 1.8942e-04

Epoch 309/350
31/31  **0s** 4ms/step - loss: 1.9726e-04 - val_loss: 1.8940e-04

Epoch 310/350
31/31  **0s** 6ms/step - loss: 2.2123e-04 - val_loss: 1.8947e-04

Epoch 311/350
31/31  **0s** 5ms/step - loss: 2.1080e-04 - val_loss: 1.8898e-04

Epoch 312/350
31/31  **0s** 4ms/step - loss: 2.1011e-04 - val_loss: 1.8994e-04

Epoch 313/350
31/31  **0s** 4ms/step - loss: 2.1423e-04 - val_loss: 1.8910e-04

Epoch 314/350
31/31  **0s** 4ms/step - loss: 1.9890e-04 - val_loss: 1.8919e-04

Epoch 315/350
31/31  **0s** 5ms/step - loss: 1.9970e-04 - val_loss: 1.8936e-04

Epoch 316/350
31/31  **0s** 4ms/step - loss: 2.1570e-04 - val_loss: 1.8908e-04

Epoch 317/350
31/31  **0s** 4ms/step - loss: 2.1581e-04 - val_loss: 1.8916e-04

Epoch 318/350
31/31  **0s** 3ms/step - loss: 1.9793e-04 - val_loss: 1.8928e-04

Epoch 319/350
31/31  **0s** 4ms/step - loss: 1.9354e-04 - val_loss: 1.8905e-04

Epoch 320/350
31/31  **0s** 4ms/step - loss: 2.0648e-04 - val_loss: 1.8890e-04

Epoch 321/350
31/31  0s 4ms/step - loss: 2.0039e-04 - val_loss: 1.8914e-04
Epoch 322/350
31/31  0s 4ms/step - loss: 2.0062e-04 - val_loss: 1.8893e-04
Epoch 323/350
31/31  0s 4ms/step - loss: 2.0052e-04 - val_loss: 1.8900e-04
Epoch 324/350
31/31  0s 4ms/step - loss: 2.0428e-04 - val_loss: 1.8899e-04
Epoch 325/350
31/31  0s 4ms/step - loss: 1.8988e-04 - val_loss: 1.8915e-04
Epoch 326/350
31/31  0s 4ms/step - loss: 2.1172e-04 - val_loss: 1.8897e-04
Epoch 327/350
31/31  0s 4ms/step - loss: 1.9617e-04 - val_loss: 1.8871e-04
Epoch 328/350
31/31  0s 4ms/step - loss: 2.0471e-04 - val_loss: 1.8886e-04
Epoch 329/350
31/31  0s 5ms/step - loss: 2.0391e-04 - val_loss: 1.8920e-04
Epoch 330/350
31/31  0s 4ms/step - loss: 2.1750e-04 - val_loss: 1.8889e-04
Epoch 331/350
31/31  0s 4ms/step - loss: 1.8783e-04 - val_loss: 1.8890e-04
Epoch 332/350
31/31  0s 4ms/step - loss: 1.9765e-04 - val_loss: 1.8906e-04
Epoch 333/350
31/31  0s 3ms/step - loss: 2.0759e-04 - val_loss: 1.8882e-04
Epoch 334/350
31/31  0s 4ms/step - loss: 1.9244e-04 - val_loss: 1.8888e-04
Epoch 335/350
31/31  0s 4ms/step - loss: 2.1368e-04 - val_loss: 1.8897e-04
Epoch 336/350
31/31  0s 3ms/step - loss: 2.1242e-04 - val_loss: 1.8894e-04
Epoch 337/350
31/31  0s 4ms/step - loss: 2.0768e-04 - val_loss: 1.8857e-04
Epoch 338/350
31/31  0s 4ms/step - loss: 2.1237e-04 - val_loss: 1.8893e-04
Epoch 339/350
31/31  0s 4ms/step - loss: 2.0179e-04 - val_loss: 1.8875e-04
Epoch 340/350
31/31  0s 4ms/step - loss: 1.9195e-04 - val_loss: 1.8893e-04

```

Epoch 341/350
31/31 ————— 0s 4ms/step - loss: 1.8425e-04 - val_loss: 1.8900e-04
Epoch 342/350
31/31 ————— 0s 4ms/step - loss: 1.9478e-04 - val_loss: 1.8862e-04
Epoch 343/350
31/31 ————— 0s 4ms/step - loss: 1.9228e-04 - val_loss: 1.8914e-04
Epoch 344/350
31/31 ————— 0s 4ms/step - loss: 2.0520e-04 - val_loss: 1.8873e-04
Epoch 345/350
31/31 ————— 0s 4ms/step - loss: 2.1454e-04 - val_loss: 1.8882e-04
Epoch 346/350
31/31 ————— 0s 4ms/step - loss: 2.1062e-04 - val_loss: 1.8889e-04
Epoch 347/350
31/31 ————— 0s 4ms/step - loss: 1.8721e-04 - val_loss: 1.8845e-04
Epoch 348/350
31/31 ————— 0s 4ms/step - loss: 2.0844e-04 - val_loss: 1.8903e-04
Epoch 349/350
31/31 ————— 0s 4ms/step - loss: 2.0617e-04 - val_loss: 1.8870e-04
Epoch 350/350
31/31 ————— 0s 4ms/step - loss: 1.9899e-04 - val_loss: 1.8859e-04

```

Out[19]: <keras.src.callbacks.history.History at 0x7968c842fe50>

```

In [20]: Y_pred = model_nonlinear.predict(Y_train, verbose=0)
error_non = np.sum((Y_train - Y_pred) ** 2) / np.sum((Y_train - np.mean(Y_train)) ** 2)
print("Reconstruction error in training sample", error_non)
Y_pred = model_nonlinear.predict(Y_test, verbose=0)
error_non_oos = np.sum((Y_test - Y_pred) ** 2) / np.sum((Y_test - np.mean(Y_test)) ** 2)
print("Reconstruction error in test sample", error_non_oos)

```

```

Reconstruction error in training sample 0.3094527914957498
Reconstruction error in test sample 0.4788719014902

```

```

In [21]: factors = model_nonlinear.encoder(Y_train).numpy()
factors_test = model_nonlinear.encoder(Y_test).numpy()
print(np.cov(factors, rowvar=False))

```

```

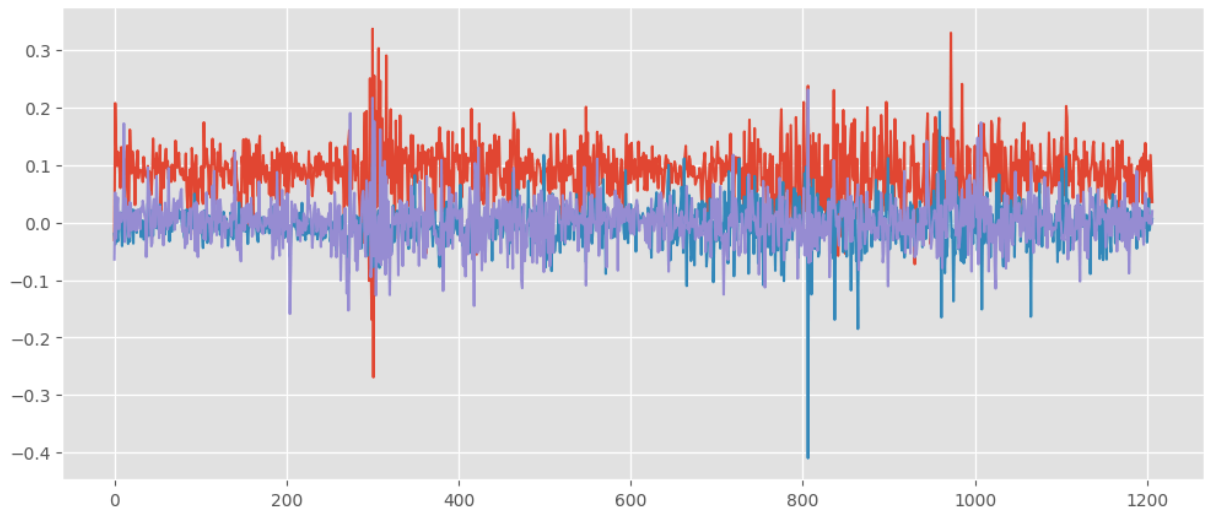
[[ 2.41902423e-03  1.45583215e-04  5.84779443e-04]
 [ 1.45583215e-04  1.35614948e-03 -5.56242140e-06]
 [ 5.84779443e-04 -5.56242140e-06  1.74229230e-03]]

```

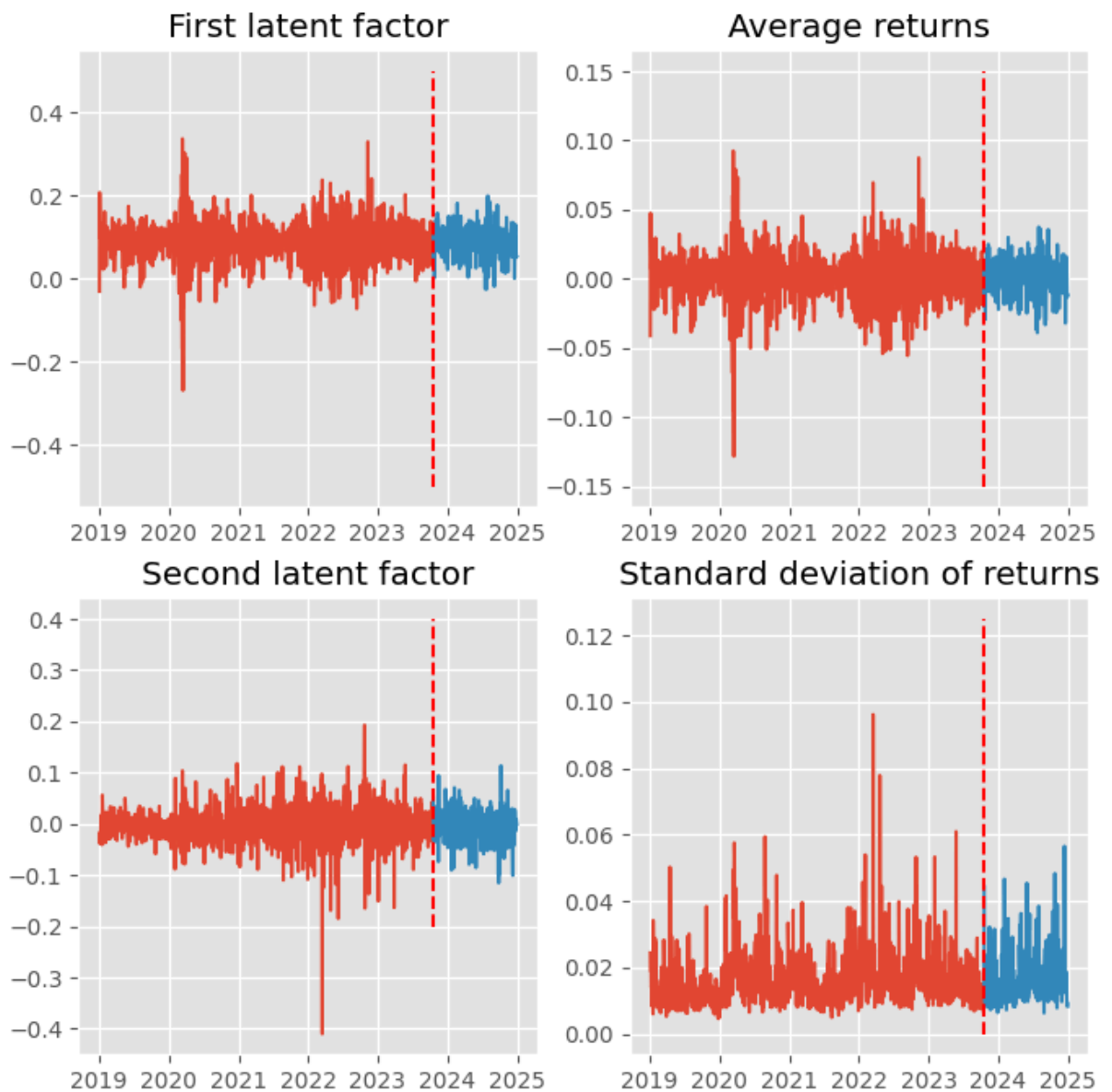
```

In [22]: # Plot the evolution of latent factors in the training period
plt.figure(figsize=(12, 5))
plt.plot(factors)
plt.show()

```

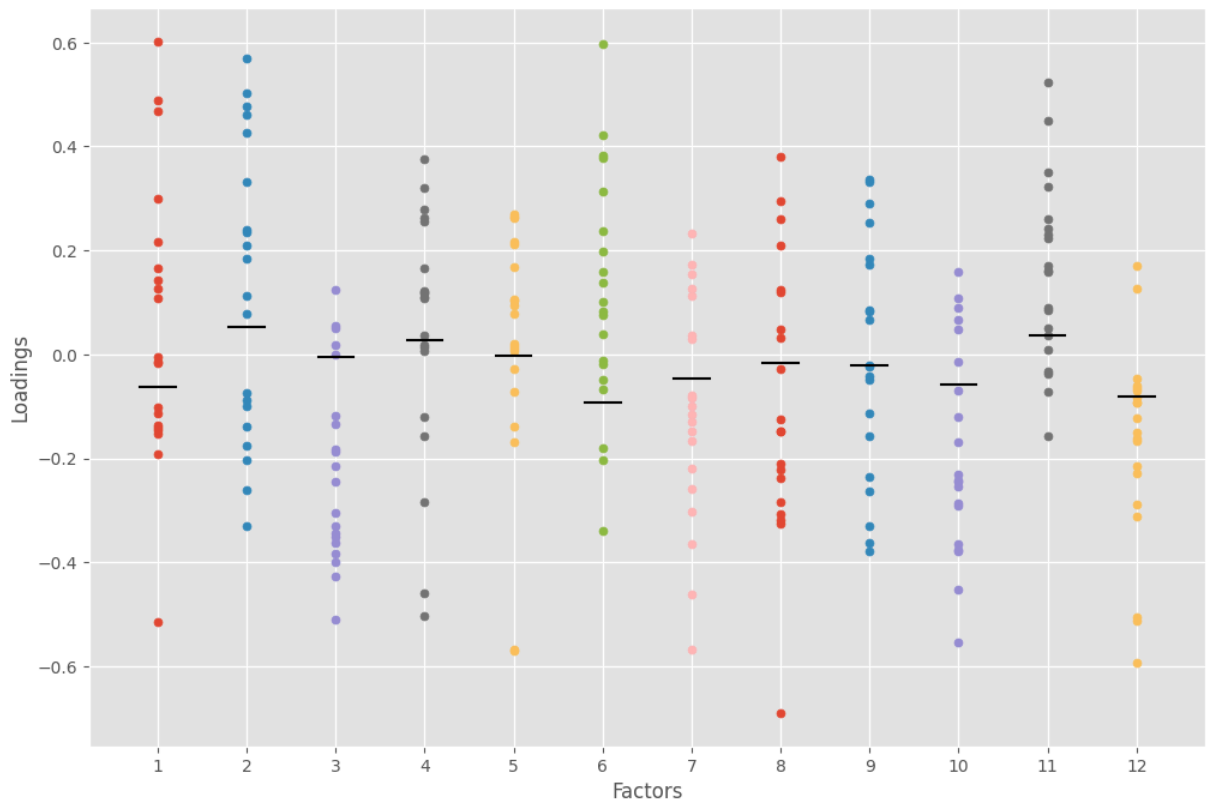


```
In [23]: fig, axs = plt.subplots(2, 2, figsize=(8, 8))
# First factor
axs[0, 0].plot(df.index[: train_size + 1], factors[:, 0])
axs[0, 0].plot(df.index[train_size + 1 :], factors_test[:, 0])
axs[0, 0].plot([df.index[train_size + 1], df.index[train_size + 1]],
[-0.5, 0.5], "r--")
axs[0, 0].title.set_text("First latent factor")
# Average of returns
axs[0, 1].plot(df.index[: train_size + 1], np.mean(Y_train, axis=1))
axs[0, 1].plot(df.index[train_size + 1 :], np.mean(Y_test, axis=1))
axs[0, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [-0.15, 0.15],
    "r--"
)
axs[0, 1].title.set_text("Average returns")
# Second factor
axs[1, 0].plot(df.index[: train_size + 1], factors[:, 1])
axs[1, 0].plot(df.index[train_size + 1 :], factors_test[:, 1])
axs[1, 0].plot([df.index[train_size + 1], df.index[train_size + 1]],
[-0.2, 0.4], "r--")
axs[1, 0].title.set_text("Second latent factor")
# Standard deviation of returns
axs[1, 1].plot(df.index[: train_size + 1], np.std(Y_train, axis=1))
axs[1, 1].plot(df.index[train_size + 1 :], np.std(Y_test, axis=1))
axs[1, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [0.0, 0.125],
    "r--"
)
axs[1, 1].title.set_text("Standard deviation of returns")
```



```
In [24]: # Extract loadings generated by the autoencoder in the output layer
w_decoder = model_nonlinear.decoder.layers[1].get_weights()[0]
data = w_decoder
labels = list(range(1, 13))
width = 0.4
fig, ax = plt.subplots(figsize=(12, 8))
for i, l in enumerate(labels):
    x = np.ones(data.shape[1]) * i
    ax.scatter(x, data[i, :], s=25)
    mean = data[:, i].mean()
    ax.plot([i - width / 2.0, i + width / 2.0], [mean, mean], color="k")
    ax.set_xlabel("Factors")
    ax.set_ylabel("Loadings")
ax.set_xticks(range(len(labels)))
ax.set_xticklabels(labels)

plt.show()
```



5. CONCLUSION

In this notebook, we have learned to design and train autoencoders with TensorFlow as a tool to extract the main latent factors that drive the variation of financial time series.

In the next lesson, we will explore tools to refine autoencoders and describe other types of architectures.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Ranjan, Chitta. "Build the Right Autoencoder – Tune and Optimize using PCA Principles, Part 1." *Medium*, 12 July 2019, <https://towardsdatascience.com/build-the-right-autoencoder-tune-and-optimize-using-pca-principles-part-i-1f01f821999b>.
 - **Required pages:** Entire article
 - **Estimated reading time:** 15 minutes
2. Ranjan, Chitta. "Build the Right Autoencoder – Tune and Optimize using PCA Principles, Part 2." *Medium*, 12 July 2019, <https://towardsdatascience.com/build-the-right-autoencoder-tune-and-optimize-using-pca-principles-part-ii-24b9cca69bd6>.
 - **Required pages:** Entire article
 - **Estimated reading time:** 15 minutes
3. Geron, Aurélien. "Machine Learning Notebooks: Autoencoders and GANS." *Github*, 9 March 2020, https://machine-learning-apps.github.io/hands-on-ml2/17_autoencoders_and_gans.
 - **Required pages:** Entire article
 - **Estimated reading time:** 1 hour

Module 6: Lesson 3

Further autoencoder topics



Outline

- ▶ Sparse autoencoders.
- ▶ Setting constraints on autoencoders.
- ▶ Autoencoders for images.

Sparse autoencoders

Sparse autoencoders represent a type of autoencoder where a penalty term is imposed on the realizations of the encodings.

This type of penalization may be of interest in overcomplete autoencoders where the number of encodings in the hidden layers is greater than the number of features.

In a sparse autoencoder, we add an L1-penalty term on the encodings Z .

If we use the MSE as the performance metric, the loss function of a sparse autoencoder can be as follows:

$$\left\| X - \hat{X} \right\|_F^2 + \lambda_1 \| \text{vec}(Z) \|_1$$

$\text{vec}(\cdot)$ is the operator that transforms a matrix into a column vector and $\| \cdot \|_1$ is the L1-norm.

In TensorFlow, we can add an activity regularizer to the layer that leads to the bottleneck:

```
activity_regularizer= tf.keras.regularizers.l1(l1weight)
```

where **l1weight** is the λ_1 hyperparameter.

Setting constraints on autoencoders

In Lesson 1, we described the architecture of an autoencoder and its association with PCA.

A single-layer linear autoencoder performs the same task as PCA, but with fewer constraints on the architecture.

Such a lack of constraints may provide too much leeway and lead to ill-posed models.

- ▶ Usual outcomes of such flexibility are the overfitting and exploding/vanishing gradient problems.
- ▶ To avoid those issues we may opt for excessively deep and complex networks, losing interpretability.

We can add the constraints from PCA to discipline the behavior of autoencoders. Namely, in PCA:

- ▶ The weight matrix is orthonormal (orthogonality + unit norm), $W^T W = I_{k \times k}$.
- ▶ The matrix of decoding weights is the transpose matrix of encoding weights, $W = W_2 = W_1^T$.
- ▶ The encodings are uncorrelated, $\text{Cov}(Z_k)$ is a diagonal matrix.

We are going to exploit TensorFlow-Keras capabilities to include constraints and loss terms in the layers of autoencoders as tools that may alleviate the under-performance of an unconstrained autoencoder.

Constraints: Unit norm weights

In PCA, the eigenvectors are constrained to have a unit norm.

The elements of the weight matrices may grow arbitrarily large with the norm of its vectors, leading to arbitrarily large variances.

- ▶ A unit-norm constraint may solve exploding-gradient problems.

TensorFlow provides a built-in method to constrain the weights to have a unit norm:

```
kernel_constraint = tf.keras.constraints.UnitNorm(axis=axis)
```

Remember that, in PCA, the encoding layer is computed as $Z_k = P_k^T X_0$, meaning that the unit norm must be imposed over the rows of the weights of the encoding layers (**axis=0**).

Thus, in the decoding layers, we must impose the unit norm over the columns in the corresponding weight matrix (**axis=1**).

- ▶ The same applies to the orthogonality constraint that we introduce next.

Constraints: Orthogonal weights

Weight orthogonality implies that weights transmit valuable information that is not shared across layers.

- An orthogonality constraint may solve vanishing-gradient problems.

Orthogonality imposes the inner products of the decoding vectors to be zero:

$$w_i^T w_j = 0, \text{ for } i, j = 1, \dots, k \text{ and } i \neq j$$

Via subclassing, we can build a custom kernel regularizer that penalizes the sum of the inner products:

kernel_regularizer=WeightsOrthogonalityRegularizer(encoding_dim, weight, axis=axis)

```
class WeightsOrthogonalityRegularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, encoding_dim, weightage = 1.0, axis = 0):
        self.encoding_dim = encoding_dim
        self.weightage = weightage
        self.axis = axis
    def weights_orthogonality(self, w):
        if(self.axis==1):
            w = tf.transpose(w)
            m = tf.tensordot(tf.transpose(w), w, axes=1) - tf.math.multiply(w,tf.eye(self.encoding_dim))
            return self.weightage * tf.math.sqrt(tf.math.reduce_sum(tf.square(m)))
    def __call__(self, w):
        return self.weights_orthogonality(w)
```

Constraints: Tied weights

Another manner of constraining and regularizing the autoencoder is to tie weights across layers (we considerably reduce the number of trainable parameters).

This means setting the weight matrix in layer ℓ equal to the transpose weight matrix in layer $-\ell$ (ℓ layers before the output layer).

To implement this, we need to define a new type of layer in TensorFlow:

`decoder_l = DenseTranspose(encoder_l, activation)`
`encoder_l` refers to the layer that `decoder_l` has tied weights with.

```
class DenseTranspose(tf.keras.layers.Layer):
    def __init__(self, dense, activation=None):
        self.dense = dense
        self.activation = tf.keras.activations.get(activation)
    def build(self, batch_input_shape):
        self.biases = self.add_weight(name="bias", shape=[self.dense.input_shape[-1]], initializer="zeros")
        super().build(batch_input_shape)
    def call(self, inputs):
        z = tf.matmul(inputs, self.dense.weights[0], transpose_b=True)
        return self.activation(z + self.biases)
```

Constraints: Uncorrelated encodings

When the encodings (factors) are correlated, redundant information is transferred through the network.

We can also impose the encodings to be uncorrelated through an activity regularizer.

```
activity_regularizer = UncorrelatedFeaturesConstraint(encoding_dim, weightage)
```

```
class UncorrelatedFeaturesConstraint(tf.keras.constraints.Constraint):  
    def __init__(self, encoding_dim, weightage = 0.0):  
        self.encoding_dim = encoding_dim  
        self.weightage = weightage  
    def uncorrelated_feature(self, x):  
        covariance = tfp.stats.covariance(x)  
        m = tf.square(covariance - tf.math.multiply(covariance, tf.eye(self.encoding_dim)))  
        output = self.weightage * tf.math.sqrt(tf.math.reduce_sum(m))  
        return output  
    def __call__(self, x):  
        return self.uncorrelated_feature(x)
```

Autoencoders for images: Dense layers

Image feature extraction and recognition are the main applications of deep learning architectures. This also applies to autoencoders.

We can easily build autoencoders for images with dense layers by including a flattening layer as the input layer and a reshaping layer as the output layer.

```
encoder = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=[window_size, window_size]),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
])
decoder = tf.keras.models.Sequential([
    tf.keras.layers.Dense(100, activation="relu", input_shape=[30]),
    tf.keras.layers.Dense(window_size * window_size, activation="sigmoid"),
    tf.keras.layers.Reshape([window_size, window_size])
])
autoencoder = tf.keras.models.Sequential([encoder, decoder])
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
autoencoder.compile(loss="mse", optimizer=adam)
autoencoder.fit(X_data_train, X_data_train, epochs=20, validation_data=(X_data_test, X_data_test))
```

Autoencoders for images: Convolutional layers

We can also build autoencoders with convolutional layers, which tend to reduce the number of parameters of the architecture relative to dense layers.

The layers of the decoder must be “Transpose convolutions” that perform the reverse operations of the convolutions that we studied in Modules 2 and 3.

```
conv_encoder = tf.keras.models.Sequential([
    tf.keras.layers.Reshape([window_size, window_size, 1], input_shape=[window_size, window_size]),
    tf.keras.layers.Conv2D(16, kernel_size=3, padding="SAME", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),
    tf.keras.layers.Conv2D(32, kernel_size=3, padding="SAME", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),
    tf.keras.layers.Conv2D(64, kernel_size=3, padding="SAME", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2)
])
conv_decoder = tf.keras.models.Sequential([
    tf.keras.layers.Conv2DTranspose(32, kernel_size=3, strides=2, padding="VALID", activation="relu", input_shape=[3, 3, 64]),
    tf.keras.layers.Conv2DTranspose(16, kernel_size=3, strides=2, padding="SAME", activation="relu"),
    tf.keras.layers.Conv2DTranspose(1, kernel_size=3, strides=2, padding="SAME", activation="sigmoid"),
    tf.keras.layers.Reshape([window_size, window_size])
])
conv_ae = tf.keras.models.Sequential([conv_encoder, conv_decoder])
```

Summary of Lesson 3

In Lesson 3, we have:

- ▶ Described the concept of sparse autoencoder.
- ▶ Enumerated several tools to regularize autoencoders based on PCA.
- ▶ Outlined the use of autoencoders for image data.

⇒ **References for this lesson:**

Ranjan, C. "Build the Right Autoencoder — Tune and Optimize using PCA Principles (Parts I and II)." *Medium*. July 2019.

Geron, A. "Machine Learning Notebooks: Autoencoders and GANS". *Github*. Mar 2020).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will train a sparse autoencoder and implement it for portfolio choice.

See you there!

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Conlon, T., et al. "Machine Learning and Factor-Based Portfolio Optimization." *arXiv* preprint, 2021, <https://arxiv.org/pdf/2107.13866>.

- **Required pages:** Entire article
- **Estimated reading time** 1 hour 30 minutes

Module 6: Lesson 4

Portfolio choice with a sparse autoencoder



Outline

- ▶ Building and training a sparse autoencoder.
- ▶ Autoencoder-based portfolio choice.

Building a sparse autoencoder

In this lesson, we build and train a sparse autoencoder to extract the latent factors that explain the variation of the 20 tech companies' stock returns that we already analyzed in Lesson 2.

We use the autoencoder model to design a portfolio that optimizes the risk-return trade-off as measured by the Sharpe Ratio.

The model is designed as an overcomplete linear autoencoder with a single hidden layer, whose dimension is equal to the number of stocks, $k = n = 20$.

The model shrinks the dimensionality of the encoding vector by including a Lasso-type regularization term in the loss function:

$$\left\| X - \hat{X} \right\|_F^2 + \lambda_1 \left\| \text{vec}(Z) \right\|_1$$

The value of the hyperparameter λ_1 determines how narrow is the bottleneck of the autoencoder.

Remember that we can implement the sparsity term in TensorFlow by specifying an activity regularizer:

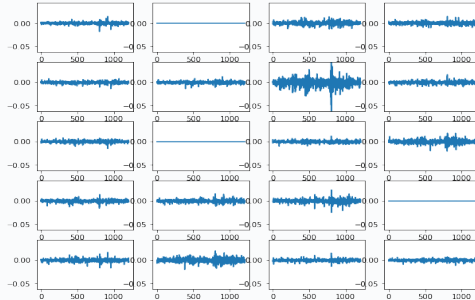
```
activity_regularizer= tf.keras.regularizers.l1(l1weight)
```

Training a sparse autoencoder

We train the autoencoder using the data from the 20 stocks, with the usual 80-20 training-test sample split.

We set aside an additional 20% of the training sample left for validation using an early stopping criterion.

- ▶ We train the model for 1,000 epochs, setting $\lambda_1 = 0.0025$.
- ▶ Training leaves 17 encodings as “relevant,” generating a small reconstruction error of roughly 5%.



Autoencoder-based portfolio choice: A model for stock returns

The trained model can be used as a basis for the design of investment strategies.

- ▶ As a byproduct, the autoencoder generates estimates for the expected returns of each stock, the exposures to systematic factors, and the amount of noise.
- ▶ We can use all this information to design diversified portfolios.

The decoding layer of the autoencoder implies that we can model the returns of stocks at time t as:

$$R_t = \hat{R}_t + \mathbf{e}_t = \mathbf{a} + B\mathbf{F}_t + \mathbf{e}_t$$

where

- ▶ $\mathbf{a}$ is an $n \times 1$ vector that includes all the trained biases in the output layer.
- ▶ B is an $n \times k$ matrix that includes all the trained weights of the output layer.
- ▶ $\mathbf{F}_t$ is a $k \times 1$ vector of realizations of the factors, which in a single-layer model are just the encodings, $\mathbf{Z}_t$.
- ▶ $\mathbf{e}_t$ is an $n \times 1$ vector of reconstruction errors, that we assume of zero-mean and uncorrelated.

Autoencoder-based portfolio choice: Risk and expected return

Using the model for stock returns from the autoencoder, we can recover estimates for the expected returns (μ) and the variance-covariance matrix of returns (Σ) from the training sample.

- ▶ $\mu = E(\hat{R}_t) = \mathbf{a} + BE(F_t)$: This is the average of the reconstructions in the training sample.
- ▶ $\Sigma = \text{Var}(R_t) = \text{Var}(\hat{R}_t) + \text{Var}(\mathbf{e}_t)$: This is the variance-covariance matrix of returns of the reconstructions, generated by the factors and the exposures, plus the diagonal (by assumption) variance-covariance matrix of reconstruction errors.

Let a portfolio be defined by a vector $w = (w_1, \dots, w_n)^T$ of weights, such that $\sum_i^n w_i = 1$.

The expected return and variance of a portfolio w can be expressed as:

$$\begin{aligned} E(R_t^w) &= w^T \mu \\ \text{Var}(R_t^w) &= w^T \Sigma w \end{aligned}$$

And the Sharpe Ratio is given by $SR^w = \frac{w^T \mu}{(w^T \Sigma w)^{1/2}}$.

Autoencoder-based portfolio choice: Performance

We devise an optimization problem, whereby we want to choose a portfolio that maximizes the return per unit of risk, as measured by the Sharpe Ratio, given the estimated μ and σ from the autoencoder.

- ▶ The solution is analytical if we do not impose short-selling restrictions.
- ▶ The optimal portfolio yields an estimated, annualized, Sharpe Ratio of 2.89 in the training sample.

We evaluate the performance of the resulting portfolio in the test sample (**Disclaimer:** These results refer to a particular initialization of the trainable parameters and may vary if other seeds are used).

- ▶ The out-of-sample Sharpe Ratio is just 0.09.
- ▶ Portfolios based on other strategies (using the sample averages and variance-covariance matrix or PCA) yield substantial negative Sharpe ratios.

The test sample features a quite different market environment than the training sample, which naturally leads to generalization issues.

However, the autoencoder seems to extract informative features that significantly outperform other, more naïve, estimation methods.

Summary of Lesson 4

In Lesson 4, we have:

- ▶ Built and trained a sparse autoencoder to extract the latent factors from a sample of stock returns.
- ▶ Used the trained model to design a portfolio that maximizes the Sharpe Ratio.

⇒ **References for this lesson:**

Conlon, T. et al. "Machine Learning and Factor-Based Portfolio Optimization." *arXiv*, 2021.

TO DO NEXT: Please go to the Jupyter Notebook associated with Lesson 4.

In the next module, we will describe transformer methods.

See you there!

DEEP LEARNING FOR FINANCE

MODULE 6 | LESSON 4

PORTFOLIO CHOICE WITH A SPARSE AUTOENCODER

Reading Time	90 minutes
Prior Knowledge	Neural Networks, machine learning, PCA, linear algebra
Keywords	Autoencoder, portfolio choice

1. INTRODUCTION

In this notebook, we will build and train autoencoder models using the same data that we exploited in Lesson 2's notebook and some of the tools that we discussed in Lesson 3.

Additionally, we will implement the predictions of the autoencoder model to design a portfolio of stocks that maximizes the expected risk-weighted return in sample and compare its performance with other, more naïve strategies.

First of all, we load the libraries and data that we need for this notebook.

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: import tensorflow as tf
```

```
2025-05-26 12:36:29.001036: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-26 12:36:29.052234: I
tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16
AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
2025-05-26 12:36:30.581926: I tensorflow/core/util/port.cc:153] oneDNN
custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

```

In [3]: # List of tickers (same order)
tickers = [
    "AAPL", "MSFT", "GOOG", "AMZN", "TSLA",
    "TCEHY", "NVDA", "TSM", "META", "BABA",
    "ASML", "AVGO", "ORCL", "CSCO", "ADBE",
    "CRM", "NFLX", "TXN", "QCOM", "SAP"
]

# Read CSV directly with Date as index
print("Loading data from CSV...")
data = pd.read_csv('adjusted_prices.csv', index_col='Date',
parse_dates=True)

print(f"Data loaded: {data.shape}")
print(f"Complete range: {data.index[0].date()} to
{data.index[-1].date()}")

# Calculate daily returns (pct_change) for all tickers
print("Calculating daily returns...")
returns_data = data[tickers].pct_change()

# Remove first row (NaN from pct_change)
df = returns_data.dropna()

df.head()

```

```

Loading data from CSV...
Data loaded: (1509, 20)
Complete range: 2019-01-02 to 2024-12-30
Calculating daily returns...

```

Out[3]:

	A A P L	M S F T	G O O G	A M Z N	T S L A	T C E H Y	N V D A	T S M	M E T A	B A B A	A S M L	A V G O	O R C L	C S C O	A D B E	C R M	N F L X	T X N	Q C O M	S A P
D a t e																				
2019-01-03 00:00:00	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.0	0.0	-0.0	-0.0	-0.0
2019-01-03 00:00:00	9.9	3.6	2.8	2.5	3.1	2.8	6.0	5.9	2.9	4.4	5.5	8.8	0.9	3.6	3.9	3.7	1.3	5.8	2.9	3.7
2019-01-03 00:00:00	6.0	7.8	4.8	2.4	4.7	3.4	4.1	1.4	0.3	6.2	1.5	9.5	7.3	6.6	4.9	9.9	2.2	9.7	6.1	0.8
2019-01-03 00:00:00	7.7	8.8	4.4	1.2	2.7	7.7	7.7	5.5	9.9	3.3	0.0	1.1	0.0	8.8	8.8	3.3	6.6	3.3	7.7	8.8
2019-01-04 00:00:00	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2019-01-04 00:00:00	0.4	4.4	5.5	5.5	5.5	5.5	6.6	1.7	4.7	7.0	4.3	0.9	4.3	4.5	4.8	5.7	7.7	4.5	1.6	3.4
2019-01-04 00:00:00	2.6	5.5	7.7	0.0	6.6	0.0	0.0	7.7	1.1	0.0	2.2	8.8	1.0	0.4	6.3	9.7	2.3	2.3	1.5	3.6
2019-01-04 00:00:00	8.9	0.9	8.6	6.4	9.7	8.8	6.8	5.3	6.8	1.1	9.9	9.9	0.0	5.5	2.2	6.6	4.4	0.0	8.8	0.0
2019-01-07 00:00:00	-0.0	0.0	-0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	-0.0	0.0
2019-01-07 00:00:00	0.0	0.0	0.0	0.3	5.5	0.0	5.5	0.0	0.0	0.2	2.2	0.2	1.0	0.6	3.3	0.3	5.9	1.7	0.2	0.9
2019-01-07 00:00:00	2.2	2.2	1.4	3.4	3.4	8.8	2.9	4.4	7.7	9.9	8.8	3.3	8.8	7.7	5.5	8.8	7.7	2.2	8.8	8.8
2019-01-07 00:00:00	2.2	7.5	6.7	5.3	6.1	9.3	4.0	3.5	2.5	7.1	4.7	6.6	4.2	5.7	7.3	7.8	1.7	2.3	2.7	2.4

	A A P L	M S F T	G O O G	A M Z N	T S L A	T C E H Y	N V D A	T S M	M E T A	B A B A	A S M L	A V G O	O R C L	C S C O	A D B E	C R M	N F L X	T X N	Q C O M	S A P
D a t e																				
2019-08-01	0.	0.	0.	0.	0.	-0	-0	-0	0.	0.	-0	-0	0.	0.	0.	0.	0.	-0	-0	0.
2019-08-01	0	0	0	0	0	.0	.0	.0	0	0	.0	.0	0	0	0	0	0	.0	.0	0
2019-08-01	1	1	0	0	1	0	0	2	0	2	0	0	0	0	1	2	1	0	0	1
2019-08-01	9	9	7	7	6	1	3	4	8	2	5	7	8	9	8	4	4	5	1	8
2019-08-01	0	0	2	3	6	1	4	8	2	4	7	3	0	0	1	9	6	6	2	6
2019-08-01	1	6	5	8	1	6	2	9	3	5	8	6	2	6	0	1	1	3	7	8
2019-08-01	0	3	1	5	2	4	3	5	2	2	6	1	6	2	0	8	0	4	0	5
2019-08-01	8																			
2019-08-01	0																			
2019-08-01	0																			
2019-08-01	0																			
2019-08-01	0																			
2019-08-01	5																			
2019-08-01	0																			
2019-08-01	0																			
2019-08-01	2	0.	0.	-0	0.	0.	0.	0.	0.	0.	0.	0.	-0	-0	0.	0.	-0	0.	0.	0.
2019-08-01	0	0	0	.0	0	0	0	0	0	0	0	0	.0	.0	0	0	.0	0	0	0
2019-08-01	1	1	1	0	0	0	3	1	1	3	4	4	0	0	1	0	0	1	1	1
2019-08-01	9	6	4	1	1	9	8	9	9	1	4	1	3	2	5	1	1	0	9	1
2019-08-01	0	9	3	5	7	4	5	6	1	9	9	4	2	0	5	8	8	9	7	7
2019-08-01	1	8	0	0	1	8	1	6	7	2	4	8	5	8	0	1	5	6	0	9
2019-08-01	0	1	0	5	4	3	8	7	6	7	8	8	0	9	9	9	3	8	7	6
2019-08-01	9																			
2019-08-01	0																			
2019-08-01	0																			
2019-08-01	0																			
2019-08-01	0																			
2019-08-01	0																			
2019-08-01	5																			
2019-08-01	0																			
2019-08-01	0																			

```
In [4]: print(f"Final DataFrame: {df.shape}")
print("Tickers included:", df.columns.tolist())

# Train/test split (80/20)
train_size = int(0.8 * df.shape[0])
Y_train = df.iloc[:train_size + 1].to_numpy()
Y_test = df.iloc[train_size + 1:].to_numpy()
```

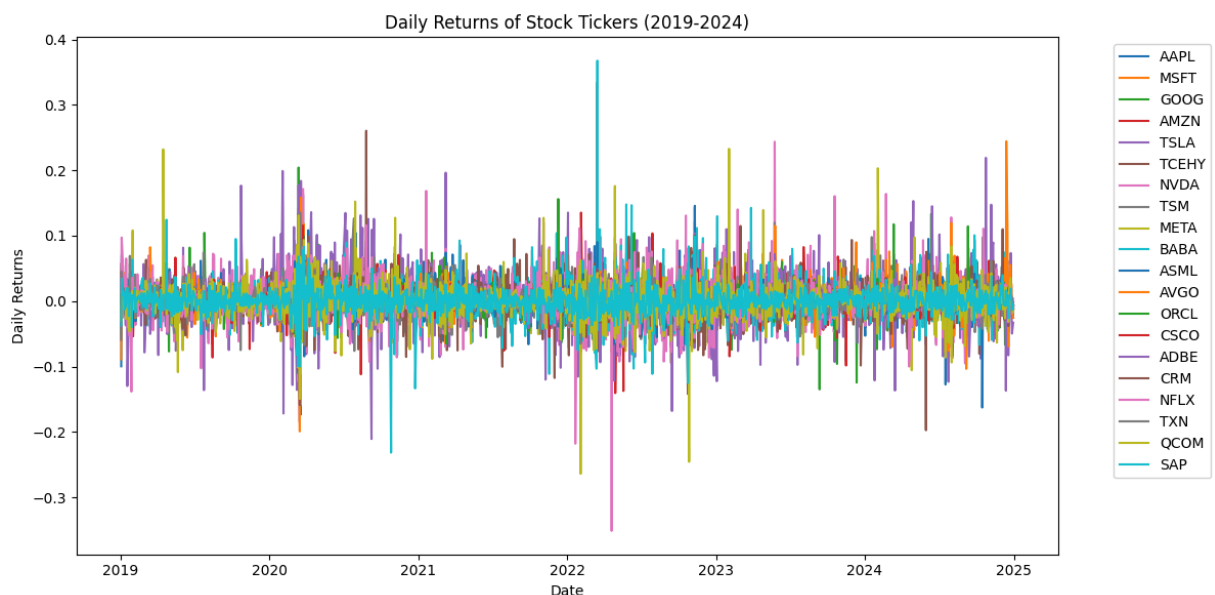
Final DataFrame: (1508, 20)
Tickers included: ['AAPL', 'MSFT', 'GOOG', 'AMZN', 'TSLA', 'TCEHY', 'NVDA', 'TSM', 'META', 'BABA', 'ASML', 'AVGO', 'ORCL', 'CSCO', 'ADBE', 'CRM', 'NFLX', 'TXN', 'QCOM', 'SAP']

```
In [5]: print(f"\nBeginning of training sample: {df.index[0].date()}")
print(f"End of training sample: {df.index[train_size].date()}")
print(f"End of test sample: {df.index[-1].date()}")
print(f"Training samples: {Y_train.shape}")
print(f"Test samples: {Y_test.shape}")
```

```
Beginning of training sample: 2019-01-03
End of training sample: 2023-10-18
End of test sample: 2024-12-30
Training samples: (1207, 20)
Test samples: (301, 20)
```

```
In [6]: import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
plt.plot(df.index, df)
plt.title(f'Daily Returns of Stock Tickers ({df.index[0].year}-
{df.index[-1].year})')
plt.xlabel('Date')
plt.ylabel('Daily Returns')
plt.legend(df.columns, bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



2. BUILDING AND TRAINING A SPARSE AUTOENCODER

We will design a portfolio based on the predictions of an autoencoder model with a loss function that includes a regularization term on the value of the factors generated by the model.

In TensorFlow, we add an L1-regularizer in the bottleneck

"tf.keras.regularizers.l1(l1weight)", where "l1weight" is a hyperparameter that governs the degree of regularization on the factor space. This parameter will determine the actual dimension of the bottleneck, without us having to impose it beforehand.

```
In [7]: class sparse_autoencoder(tf.keras.Model):
    def __init__(self, output_dim, bottleneck_dim=1, llweight=0.0):
        super(sparse_autoencoder, self).__init__()
        self.output_dim = output_dim
        self.bottleneck_dim = bottleneck_dim
        self.llweight = llweight
        initializer = tf.keras.initializers.GlorotUniform()
        # Set the activity regularizer that will generate the sparsity of
        factors
        activity_regularizer = tf.keras.regularizers.l1(llweight)
        # Build the encoding layer, including the activity regularizer
        self.encoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(
                    bottleneck_dim,
                    activity_regularizer=activity_regularizer,
                    kernel_initializer=initializer,
                    bias_initializer=tf.keras.initializers.Zeros(),
                )
            ]
        )
        # Build the decoding layer
        self.decoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(
                    output_dim,
                    kernel_initializer=initializer,
                    bias_initializer=tf.keras.initializers.Zeros(),
                )
            ]
        )

    def call(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded
```

Let's compile and train the model. We set the L1-regularization parameter to $\lambda_1 = 0.0025$.

```
In [8]: # Set the bottleneck size equal to the input size
nlatent = Y_train.shape[1]
# Set the parameter on the L1 activity-regularization term
llweight = 0.0025
# Choose the proportion of training data used for validation
val_split = 0.2
# Determine early stopping criterion
early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss",
patience=50)
# Choose optimizer
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
# Build and compile the model
model = sparse_autoencoder(Y_train.shape[1], nlatent, llweight)
model.compile(optimizer=adam, loss="mse")
# Train the model
model.fit(
    x=Y_train,
    y=Y_train,
    batch_size=32,
    epochs=350,
    validation_split=val_split,
    callbacks=[early_stop],
    verbose=1,
)
```

Epoch 1/350			
31/31		1s	8ms/step - loss: 0.0273 - val_loss: 0.0245
Epoch 2/350			
31/31		0s	4ms/step - loss: 0.0267 - val_loss: 0.0239
Epoch 3/350			
31/31		0s	4ms/step - loss: 0.0268 - val_loss: 0.0234
Epoch 4/350			
31/31		0s	5ms/step - loss: 0.0252 - val_loss: 0.0229
Epoch 5/350			
31/31		0s	4ms/step - loss: 0.0241 - val_loss: 0.0225
Epoch 6/350			
31/31		0s	4ms/step - loss: 0.0243 - val_loss: 0.0220
Epoch 7/350			
31/31		0s	4ms/step - loss: 0.0238 - val_loss: 0.0216
Epoch 8/350			
31/31		0s	4ms/step - loss: 0.0231 - val_loss: 0.0213
Epoch 9/350			
31/31		0s	4ms/step - loss: 0.0232 - val_loss: 0.0209
Epoch 10/350			
31/31		0s	4ms/step - loss: 0.0224 - val_loss: 0.0206
Epoch 11/350			
31/31		0s	3ms/step - loss: 0.0214 - val_loss: 0.0203
Epoch 12/350			
31/31		0s	4ms/step - loss: 0.0210 - val_loss: 0.0200
Epoch 13/350			
31/31		0s	4ms/step - loss: 0.0214 - val_loss: 0.0197
Epoch 14/350			
31/31		0s	4ms/step - loss: 0.0216 - val_loss: 0.0195
Epoch 15/350			
31/31		0s	4ms/step - loss: 0.0200 - val_loss: 0.0192
Epoch 16/350			
31/31		0s	4ms/step - loss: 0.0202 - val_loss: 0.0190
Epoch 17/350			
31/31		0s	3ms/step - loss: 0.0204 - val_loss: 0.0188
Epoch 18/350			
31/31		0s	4ms/step - loss: 0.0198 - val_loss: 0.0186
Epoch 19/350			
31/31		0s	4ms/step - loss: 0.0196 - val_loss: 0.0184
Epoch 20/350			
31/31		0s	4ms/step - loss: 0.0188 - val_loss: 0.0182
Epoch 21/350			
31/31		0s	4ms/step - loss: 0.0183 - val_loss: 0.0180
Epoch 22/350			
31/31		0s	4ms/step - loss: 0.0185 - val_loss: 0.0179
Epoch 23/350			
31/31		0s	4ms/step - loss: 0.0186 - val_loss: 0.0177
Epoch 24/350			
31/31		0s	4ms/step - loss: 0.0184 - val_loss: 0.0175
Epoch 25/350			
31/31		0s	3ms/step - loss: 0.0181 - val_loss: 0.0174
Epoch 26/350			
31/31		0s	4ms/step - loss: 0.0177 - val_loss: 0.0172
Epoch 27/350			
31/31		0s	4ms/step - loss: 0.0181 - val_loss: 0.0171
Epoch 28/350			
31/31		0s	4ms/step - loss: 0.0176 - val_loss: 0.0169
Epoch 29/350			
31/31		0s	4ms/step - loss: 0.0171 - val_loss: 0.0168
Epoch 30/350			
31/31		0s	3ms/step - loss: 0.0170 - val_loss: 0.0166

Epoch 31/350			
31/31		0s	3ms/step - loss: 0.0167 - val_loss: 0.0165
Epoch 32/350			
31/31		0s	3ms/step - loss: 0.0171 - val_loss: 0.0163
Epoch 33/350			
31/31		0s	4ms/step - loss: 0.0171 - val_loss: 0.0162
Epoch 34/350			
31/31		0s	4ms/step - loss: 0.0164 - val_loss: 0.0161
Epoch 35/350			
31/31		0s	4ms/step - loss: 0.0165 - val_loss: 0.0159
Epoch 36/350			
31/31		0s	4ms/step - loss: 0.0160 - val_loss: 0.0158
Epoch 37/350			
31/31		0s	4ms/step - loss: 0.0165 - val_loss: 0.0156
Epoch 38/350			
31/31		0s	4ms/step - loss: 0.0163 - val_loss: 0.0155
Epoch 39/350			
31/31		0s	3ms/step - loss: 0.0161 - val_loss: 0.0154
Epoch 40/350			
31/31		0s	4ms/step - loss: 0.0154 - val_loss: 0.0152
Epoch 41/350			
31/31		0s	4ms/step - loss: 0.0158 - val_loss: 0.0151
Epoch 42/350			
31/31		0s	4ms/step - loss: 0.0150 - val_loss: 0.0150
Epoch 43/350			
31/31		0s	3ms/step - loss: 0.0156 - val_loss: 0.0148
Epoch 44/350			
31/31		0s	4ms/step - loss: 0.0151 - val_loss: 0.0147
Epoch 45/350			
31/31		0s	4ms/step - loss: 0.0148 - val_loss: 0.0146
Epoch 46/350			
31/31		0s	4ms/step - loss: 0.0147 - val_loss: 0.0144
Epoch 47/350			
31/31		0s	5ms/step - loss: 0.0147 - val_loss: 0.0143
Epoch 48/350			
31/31		0s	4ms/step - loss: 0.0147 - val_loss: 0.0142
Epoch 49/350			
31/31		0s	4ms/step - loss: 0.0144 - val_loss: 0.0140
Epoch 50/350			
31/31		0s	4ms/step - loss: 0.0146 - val_loss: 0.0139
Epoch 51/350			
31/31		0s	3ms/step - loss: 0.0141 - val_loss: 0.0138
Epoch 52/350			
31/31		0s	3ms/step - loss: 0.0140 - val_loss: 0.0136
Epoch 53/350			
31/31		0s	4ms/step - loss: 0.0139 - val_loss: 0.0135
Epoch 54/350			
31/31		0s	3ms/step - loss: 0.0134 - val_loss: 0.0134
Epoch 55/350			
31/31		0s	4ms/step - loss: 0.0136 - val_loss: 0.0133
Epoch 56/350			
31/31		0s	4ms/step - loss: 0.0135 - val_loss: 0.0131
Epoch 57/350			
31/31		0s	4ms/step - loss: 0.0133 - val_loss: 0.0130
Epoch 58/350			
31/31		0s	4ms/step - loss: 0.0132 - val_loss: 0.0129
Epoch 59/350			
31/31		0s	4ms/step - loss: 0.0130 - val_loss: 0.0127
Epoch 60/350			
31/31		0s	4ms/step - loss: 0.0128 - val_loss: 0.0126

Epoch 61/350			
31/31		0s	4ms/step - loss: 0.0126 - val_loss: 0.0125
Epoch 62/350			
31/31		0s	4ms/step - loss: 0.0127 - val_loss: 0.0124
Epoch 63/350			
31/31		0s	4ms/step - loss: 0.0123 - val_loss: 0.0122
Epoch 64/350			
31/31		0s	3ms/step - loss: 0.0124 - val_loss: 0.0121
Epoch 65/350			
31/31		0s	4ms/step - loss: 0.0121 - val_loss: 0.0120
Epoch 66/350			
31/31		0s	4ms/step - loss: 0.0122 - val_loss: 0.0119
Epoch 67/350			
31/31		0s	4ms/step - loss: 0.0117 - val_loss: 0.0118
Epoch 68/350			
31/31		0s	4ms/step - loss: 0.0118 - val_loss: 0.0116
Epoch 69/350			
31/31		0s	3ms/step - loss: 0.0118 - val_loss: 0.0115
Epoch 70/350			
31/31		0s	4ms/step - loss: 0.0118 - val_loss: 0.0114
Epoch 71/350			
31/31		0s	4ms/step - loss: 0.0114 - val_loss: 0.0113
Epoch 72/350			
31/31		0s	4ms/step - loss: 0.0114 - val_loss: 0.0112
Epoch 73/350			
31/31		0s	4ms/step - loss: 0.0113 - val_loss: 0.0110
Epoch 74/350			
31/31		0s	4ms/step - loss: 0.0109 - val_loss: 0.0109
Epoch 75/350			
31/31		0s	4ms/step - loss: 0.0106 - val_loss: 0.0108
Epoch 76/350			
31/31		0s	4ms/step - loss: 0.0106 - val_loss: 0.0107
Epoch 77/350			
31/31		0s	3ms/step - loss: 0.0106 - val_loss: 0.0106
Epoch 78/350			
31/31		0s	3ms/step - loss: 0.0107 - val_loss: 0.0104
Epoch 79/350			
31/31		0s	4ms/step - loss: 0.0103 - val_loss: 0.0103
Epoch 80/350			
31/31		0s	4ms/step - loss: 0.0104 - val_loss: 0.0102
Epoch 81/350			
31/31		0s	4ms/step - loss: 0.0100 - val_loss: 0.0101
Epoch 82/350			
31/31		0s	3ms/step - loss: 0.0102 - val_loss: 0.0100
Epoch 83/350			
31/31		0s	4ms/step - loss: 0.0099 - val_loss: 0.0099
Epoch 84/350			
31/31		0s	4ms/step - loss: 0.0099 - val_loss: 0.0097
Epoch 85/350			
31/31		0s	4ms/step - loss: 0.0097 - val_loss: 0.0096
Epoch 86/350			
31/31		0s	4ms/step - loss: 0.0096 - val_loss: 0.0095
Epoch 87/350			
31/31		0s	4ms/step - loss: 0.0094 - val_loss: 0.0094
Epoch 88/350			
31/31		0s	4ms/step - loss: 0.0094 - val_loss: 0.0093
Epoch 89/350			
31/31		0s	4ms/step - loss: 0.0094 - val_loss: 0.0092
Epoch 90/350			
31/31		0s	4ms/step - loss: 0.0091 - val_loss: 0.0091

Epoch 91/350			
31/31		0s	3ms/step - loss: 0.0091 - val_loss: 0.0089
Epoch 92/350			
31/31		0s	4ms/step - loss: 0.0090 - val_loss: 0.0088
Epoch 93/350			
31/31		0s	4ms/step - loss: 0.0084 - val_loss: 0.0087
Epoch 94/350			
31/31		0s	4ms/step - loss: 0.0087 - val_loss: 0.0086
Epoch 95/350			
31/31		0s	4ms/step - loss: 0.0086 - val_loss: 0.0085
Epoch 96/350			
31/31		0s	4ms/step - loss: 0.0084 - val_loss: 0.0084
Epoch 97/350			
31/31		0s	3ms/step - loss: 0.0084 - val_loss: 0.0083
Epoch 98/350			
31/31		0s	4ms/step - loss: 0.0082 - val_loss: 0.0082
Epoch 99/350			
31/31		0s	4ms/step - loss: 0.0082 - val_loss: 0.0081
Epoch 100/350			
31/31		0s	4ms/step - loss: 0.0078 - val_loss: 0.0080
Epoch 101/350			
31/31		0s	3ms/step - loss: 0.0078 - val_loss: 0.0079
Epoch 102/350			
31/31		0s	3ms/step - loss: 0.0079 - val_loss: 0.0078
Epoch 103/350			
31/31		0s	4ms/step - loss: 0.0078 - val_loss: 0.0077
Epoch 104/350			
31/31		0s	4ms/step - loss: 0.0075 - val_loss: 0.0076
Epoch 105/350			
31/31		0s	4ms/step - loss: 0.0074 - val_loss: 0.0075
Epoch 106/350			
31/31		0s	3ms/step - loss: 0.0075 - val_loss: 0.0074
Epoch 107/350			
31/31		0s	4ms/step - loss: 0.0071 - val_loss: 0.0073
Epoch 108/350			
31/31		0s	4ms/step - loss: 0.0072 - val_loss: 0.0072
Epoch 109/350			
31/31		0s	4ms/step - loss: 0.0073 - val_loss: 0.0071
Epoch 110/350			
31/31		0s	6ms/step - loss: 0.0069 - val_loss: 0.0070
Epoch 111/350			
31/31		0s	5ms/step - loss: 0.0071 - val_loss: 0.0069
Epoch 112/350			
31/31		0s	4ms/step - loss: 0.0067 - val_loss: 0.0068
Epoch 113/350			
31/31		0s	4ms/step - loss: 0.0066 - val_loss: 0.0067
Epoch 114/350			
31/31		0s	4ms/step - loss: 0.0067 - val_loss: 0.0066
Epoch 115/350			
31/31		0s	4ms/step - loss: 0.0064 - val_loss: 0.0065
Epoch 116/350			
31/31		0s	4ms/step - loss: 0.0065 - val_loss: 0.0064
Epoch 117/350			
31/31		0s	4ms/step - loss: 0.0063 - val_loss: 0.0063
Epoch 118/350			
31/31		0s	5ms/step - loss: 0.0063 - val_loss: 0.0062
Epoch 119/350			
31/31		0s	4ms/step - loss: 0.0060 - val_loss: 0.0061
Epoch 120/350			
31/31		0s	4ms/step - loss: 0.0061 - val_loss: 0.0060

Epoch 121/350			
31/31		0s	4ms/step - loss: 0.0059 - val_loss: 0.0060
Epoch 122/350			
31/31		0s	4ms/step - loss: 0.0057 - val_loss: 0.0059
Epoch 123/350			
31/31		0s	4ms/step - loss: 0.0056 - val_loss: 0.0058
Epoch 124/350			
31/31		0s	3ms/step - loss: 0.0056 - val_loss: 0.0057
Epoch 125/350			
31/31		0s	4ms/step - loss: 0.0057 - val_loss: 0.0056
Epoch 126/350			
31/31		0s	4ms/step - loss: 0.0054 - val_loss: 0.0055
Epoch 127/350			
31/31		0s	4ms/step - loss: 0.0055 - val_loss: 0.0054
Epoch 128/350			
31/31		0s	3ms/step - loss: 0.0052 - val_loss: 0.0053
Epoch 129/350			
31/31		0s	4ms/step - loss: 0.0053 - val_loss: 0.0052
Epoch 130/350			
31/31		0s	4ms/step - loss: 0.0050 - val_loss: 0.0052
Epoch 131/350			
31/31		0s	4ms/step - loss: 0.0050 - val_loss: 0.0051
Epoch 132/350			
31/31		0s	4ms/step - loss: 0.0049 - val_loss: 0.0050
Epoch 133/350			
31/31		0s	4ms/step - loss: 0.0050 - val_loss: 0.0049
Epoch 134/350			
31/31		0s	4ms/step - loss: 0.0048 - val_loss: 0.0048
Epoch 135/350			
31/31		0s	4ms/step - loss: 0.0046 - val_loss: 0.0048
Epoch 136/350			
31/31		0s	4ms/step - loss: 0.0046 - val_loss: 0.0047
Epoch 137/350			
31/31		0s	4ms/step - loss: 0.0045 - val_loss: 0.0046
Epoch 138/350			
31/31		0s	3ms/step - loss: 0.0046 - val_loss: 0.0045
Epoch 139/350			
31/31		0s	4ms/step - loss: 0.0044 - val_loss: 0.0044
Epoch 140/350			
31/31		0s	3ms/step - loss: 0.0044 - val_loss: 0.0043
Epoch 141/350			
31/31		0s	4ms/step - loss: 0.0042 - val_loss: 0.0043
Epoch 142/350			
31/31		0s	4ms/step - loss: 0.0042 - val_loss: 0.0042
Epoch 143/350			
31/31		0s	3ms/step - loss: 0.0040 - val_loss: 0.0041
Epoch 144/350			
31/31		0s	4ms/step - loss: 0.0039 - val_loss: 0.0040
Epoch 145/350			
31/31		0s	4ms/step - loss: 0.0041 - val_loss: 0.0039
Epoch 146/350			
31/31		0s	4ms/step - loss: 0.0039 - val_loss: 0.0039
Epoch 147/350			
31/31		0s	4ms/step - loss: 0.0037 - val_loss: 0.0038
Epoch 148/350			
31/31		0s	4ms/step - loss: 0.0035 - val_loss: 0.0037
Epoch 149/350			
31/31		0s	3ms/step - loss: 0.0036 - val_loss: 0.0037
Epoch 150/350			
31/31		0s	4ms/step - loss: 0.0035 - val_loss: 0.0036

Epoch 151/350			
31/31		0s 4ms/step	loss: 0.0035 - val_loss: 0.0035
Epoch 152/350			
31/31		0s 4ms/step	loss: 0.0033 - val_loss: 0.0034
Epoch 153/350			
31/31		0s 4ms/step	loss: 0.0034 - val_loss: 0.0034
Epoch 154/350			
31/31		0s 4ms/step	loss: 0.0033 - val_loss: 0.0033
Epoch 155/350			
31/31		0s 4ms/step	loss: 0.0032 - val_loss: 0.0032
Epoch 156/350			
31/31		0s 3ms/step	loss: 0.0032 - val_loss: 0.0032
Epoch 157/350			
31/31		0s 5ms/step	loss: 0.0030 - val_loss: 0.0031
Epoch 158/350			
31/31		0s 4ms/step	loss: 0.0029 - val_loss: 0.0030
Epoch 159/350			
31/31		0s 4ms/step	loss: 0.0029 - val_loss: 0.0030
Epoch 160/350			
31/31		0s 3ms/step	loss: 0.0028 - val_loss: 0.0029
Epoch 161/350			
31/31		0s 4ms/step	loss: 0.0029 - val_loss: 0.0028
Epoch 162/350			
31/31		0s 4ms/step	loss: 0.0027 - val_loss: 0.0027
Epoch 163/350			
31/31		0s 4ms/step	loss: 0.0027 - val_loss: 0.0027
Epoch 164/350			
31/31		0s 4ms/step	loss: 0.0026 - val_loss: 0.0026
Epoch 165/350			
31/31		0s 4ms/step	loss: 0.0025 - val_loss: 0.0025
Epoch 166/350			
31/31		0s 4ms/step	loss: 0.0024 - val_loss: 0.0025
Epoch 167/350			
31/31		0s 4ms/step	loss: 0.0024 - val_loss: 0.0024
Epoch 168/350			
31/31		0s 3ms/step	loss: 0.0024 - val_loss: 0.0024
Epoch 169/350			
31/31		0s 4ms/step	loss: 0.0024 - val_loss: 0.0023
Epoch 170/350			
31/31		0s 3ms/step	loss: 0.0022 - val_loss: 0.0022
Epoch 171/350			
31/31		0s 3ms/step	loss: 0.0022 - val_loss: 0.0022
Epoch 172/350			
31/31		0s 4ms/step	loss: 0.0021 - val_loss: 0.0021
Epoch 173/350			
31/31		0s 4ms/step	loss: 0.0021 - val_loss: 0.0021
Epoch 174/350			
31/31		0s 3ms/step	loss: 0.0020 - val_loss: 0.0020
Epoch 175/350			
31/31		0s 4ms/step	loss: 0.0020 - val_loss: 0.0020
Epoch 176/350			
31/31		0s 4ms/step	loss: 0.0019 - val_loss: 0.0019
Epoch 177/350			
31/31		0s 4ms/step	loss: 0.0019 - val_loss: 0.0019
Epoch 178/350			
31/31		0s 4ms/step	loss: 0.0018 - val_loss: 0.0018
Epoch 179/350			
31/31		0s 4ms/step	loss: 0.0018 - val_loss: 0.0018
Epoch 180/350			
31/31		0s 4ms/step	loss: 0.0018 - val_loss: 0.0017

```
Epoch 181/350
31/31 ————— 0s 3ms/step - loss: 0.0017 - val_loss: 0.0017
Epoch 182/350
31/31 ————— 0s 4ms/step - loss: 0.0017 - val_loss: 0.0016
Epoch 183/350
31/31 ————— 0s 4ms/step - loss: 0.0017 - val_loss: 0.0016
Epoch 184/350
31/31 ————— 0s 4ms/step - loss: 0.0017 - val_loss: 0.0016
Epoch 185/350
31/31 ————— 0s 4ms/step - loss: 0.0016 - val_loss: 0.0015
Epoch 186/350
31/31 ————— 0s 3ms/step - loss: 0.0015 - val_loss: 0.0015
Epoch 187/350
31/31 ————— 0s 3ms/step - loss: 0.0015 - val_loss: 0.0014
Epoch 188/350
31/31 ————— 0s 4ms/step - loss: 0.0014 - val_loss: 0.0014
Epoch 189/350
31/31 ————— 0s 4ms/step - loss: 0.0014 - val_loss: 0.0014
Epoch 190/350
31/31 ————— 0s 4ms/step - loss: 0.0014 - val_loss: 0.0013
Epoch 191/350
31/31 ————— 0s 3ms/step - loss: 0.0013 - val_loss: 0.0013
Epoch 192/350
31/31 ————— 0s 4ms/step - loss: 0.0013 - val_loss: 0.0012
Epoch 193/350
31/31 ————— 0s 3ms/step - loss: 0.0012 - val_loss: 0.0012
Epoch 194/350
31/31 ————— 0s 4ms/step - loss: 0.0012 - val_loss: 0.0012
Epoch 195/350
31/31 ————— 0s 4ms/step - loss: 0.0012 - val_loss: 0.0011
Epoch 196/350
31/31 ————— 0s 4ms/step - loss: 0.0011 - val_loss: 0.0011
Epoch 197/350
31/31 ————— 0s 4ms/step - loss: 0.0011 - val_loss: 0.0011
Epoch 198/350
31/31 ————— 0s 4ms/step - loss: 0.0012 - val_loss: 0.0010
Epoch 199/350
31/31 ————— 0s 4ms/step - loss: 0.0011 - val_loss: 0.0010
Epoch 200/350
31/31 ————— 0s 4ms/step - loss: 0.0011 - val_loss: 9.9598e-04
Epoch 201/350
31/31 ————— 0s 4ms/step - loss: 0.0010 - val_loss: 9.6657e-04
Epoch 202/350
31/31 ————— 0s 4ms/step - loss: 0.0010 - val_loss: 9.4100e-04
Epoch 203/350
31/31 ————— 0s 4ms/step - loss: 0.0010 - val_loss: 9.2630e-04
Epoch 204/350
31/31 ————— 0s 4ms/step - loss: 9.4837e-04 - val_loss:
9.0591e-04
Epoch 205/350
31/31 ————— 0s 3ms/step - loss: 0.0010 - val_loss: 8.8512e-04
Epoch 206/350
31/31 ————— 0s 4ms/step - loss: 0.0010 - val_loss: 8.5939e-04
Epoch 207/350
31/31 ————— 0s 4ms/step - loss: 9.4026e-04 - val_loss:
8.5006e-04
Epoch 208/350
31/31 ————— 0s 4ms/step - loss: 8.9723e-04 - val_loss:
8.3320e-04
Epoch 209/350
```

31/31  0s 4ms/step - loss: 9.0880e-04 - val_loss: 8.0905e-04
Epoch 210/350

31/31  0s 4ms/step - loss: 8.8811e-04 - val_loss: 8.0341e-04
Epoch 211/350

31/31  0s 4ms/step - loss: 8.6462e-04 - val_loss: 7.8496e-04
Epoch 212/350

31/31  0s 4ms/step - loss: 9.6199e-04 - val_loss: 7.6977e-04
Epoch 213/350

31/31  0s 4ms/step - loss: 8.7442e-04 - val_loss: 7.5621e-04
Epoch 214/350

31/31  0s 4ms/step - loss: 7.9466e-04 - val_loss: 7.4551e-04
Epoch 215/350

31/31  0s 4ms/step - loss: 8.4721e-04 - val_loss: 7.3255e-04
Epoch 216/350

31/31  0s 4ms/step - loss: 7.9239e-04 - val_loss: 7.2596e-04
Epoch 217/350

31/31  0s 4ms/step - loss: 8.6110e-04 - val_loss: 7.1625e-04
Epoch 218/350

31/31  0s 4ms/step - loss: 7.5705e-04 - val_loss: 7.0012e-04
Epoch 219/350

31/31  0s 4ms/step - loss: 7.4169e-04 - val_loss: 6.9393e-04
Epoch 220/350

31/31  0s 4ms/step - loss: 7.7664e-04 - val_loss: 6.9053e-04
Epoch 221/350

31/31  0s 4ms/step - loss: 7.9245e-04 - val_loss: 6.7629e-04
Epoch 222/350

31/31  0s 4ms/step - loss: 7.8312e-04 - val_loss: 6.7416e-04
Epoch 223/350

31/31  0s 4ms/step - loss: 7.2096e-04 - val_loss: 6.6117e-04
Epoch 224/350

31/31  0s 4ms/step - loss: 7.1062e-04 - val_loss: 6.5456e-04
Epoch 225/350

31/31  0s 4ms/step - loss: 7.7285e-04 - val_loss: 6.4517e-04
Epoch 226/350

31/31  0s 4ms/step - loss: 7.5388e-04 - val_loss: 6.4444e-04
Epoch 227/350

31/31  0s 3ms/step - loss: 7.4369e-04 - val_loss: 6.3181e-04
Epoch 228/350

31/31  0s 3ms/step - loss: 6.8822e-04 - val_loss: 6.2895e-04
Epoch 229/350

31/31  0s 4ms/step - loss: 6.6178e-04 - val_loss: 6.2198e-04
Epoch 230/350

31/31  0s 3ms/step - loss: 7.3060e-04 - val_loss: 6.1930e-04
Epoch 231/350

31/31  0s 4ms/step - loss: 6.6227e-04 - val_loss: 6.1389e-04
Epoch 232/350

31/31  0s 3ms/step - loss: 7.1834e-04 - val_loss: 6.0468e-04
Epoch 233/350

31/31  0s 4ms/step - loss: 6.9119e-04 - val_loss: 6.0104e-04
Epoch 234/350

31/31  0s 4ms/step - loss: 7.3072e-04 - val_loss: 5.9892e-04
Epoch 235/350

31/31  0s 4ms/step - loss: 7.1064e-04 - val_loss: 5.9482e-04
Epoch 236/350

31/31  0s 4ms/step - loss: 6.8085e-04 - val_loss: 5.9023e-04
Epoch 237/350

31/31  0s 4ms/step - loss: 7.3317e-04 - val_loss: 5.8188e-04
Epoch 238/350

31/31  0s 4ms/step - loss: 7.2154e-04 - val_loss: 5.8049e-04
Epoch 239/350

31/31  0s 4ms/step - loss: 7.2158e-04 - val_loss: 5.8148e-04
Epoch 240/350

31/31  0s 4ms/step - loss: 7.0546e-04 - val_loss: 5.7321e-04
Epoch 241/350

31/31  0s 4ms/step - loss: 7.0606e-04 - val_loss: 5.8069e-04
Epoch 242/350

31/31  0s 3ms/step - loss: 6.5722e-04 - val_loss: 5.7367e-04
Epoch 243/350

31/31  0s 4ms/step - loss: 6.5207e-04 - val_loss: 5.7832e-04
Epoch 244/350

31/31  0s 4ms/step - loss: 6.5185e-04 - val_loss: 5.7417e-04
Epoch 245/350

31/31  0s 4ms/step - loss: 6.8971e-04 - val_loss: 5.7500e-04
Epoch 246/350

31/31  0s 4ms/step - loss: 6.9838e-04 - val_loss: 5.7275e-04
Epoch 247/350

31/31  0s 4ms/step - loss: 6.6729e-04 - val_loss: 5.7409e-04
Epoch 248/350

31/31  0s 4ms/step - loss: 6.7206e-04 - val_loss: 5.7674e-04
Epoch 249/350

31/31  0s 3ms/step - loss: 7.1314e-04 - val_loss: 5.7268e-04
Epoch 250/350

31/31  0s 4ms/step - loss: 6.7272e-04 - val_loss: 5.7438e-04
Epoch 251/350

31/31  0s 4ms/step - loss: 6.4517e-04 - val_loss: 5.7017e-04
Epoch 252/350

31/31  0s 4ms/step - loss: 6.8427e-04 - val_loss: 5.7634e-04
Epoch 253/350

31/31  0s 4ms/step - loss: 7.2278e-04 - val_loss: 5.7780e-04
Epoch 254/350

31/31  0s 4ms/step - loss: 6.2318e-04 - val_loss: 5.7251e-04
Epoch 255/350

31/31  0s 5ms/step - loss: 7.1412e-04 - val_loss: 5.7501e-04
Epoch 256/350

31/31  0s 4ms/step - loss: 7.0545e-04 - val_loss: 5.7014e-04
Epoch 257/350

31/31  0s 4ms/step - loss: 6.5516e-04 - val_loss: 5.7730e-04
Epoch 258/350

31/31  0s 4ms/step - loss: 6.9275e-04 - val_loss: 5.7832e-04
Epoch 259/350

31/31  0s 4ms/step - loss: 6.2398e-04 - val_loss: 5.7191e-04
Epoch 260/350

31/31  0s 4ms/step - loss: 6.8853e-04 - val_loss: 5.6961e-04
Epoch 261/350

31/31  0s 3ms/step - loss: 7.0878e-04 - val_loss: 5.7236e-04
Epoch 262/350

31/31  0s 3ms/step - loss: 7.1000e-04 - val_loss: 5.7069e-04
Epoch 263/350

31/31  0s 4ms/step - loss: 6.3854e-04 - val_loss: 5.7092e-04
Epoch 264/350

31/31  0s 3ms/step - loss: 6.2705e-04 - val_loss: 5.6668e-04
Epoch 265/350

31/31  0s 3ms/step - loss: 6.5795e-04 - val_loss: 5.7109e-04
Epoch 266/350

31/31  0s 4ms/step - loss: 6.3558e-04 - val_loss: 5.7240e-04
Epoch 267/350

31/31  0s 4ms/step - loss: 6.6518e-04 - val_loss: 5.7040e-04
Epoch 268/350

31/31  0s 4ms/step - loss: 6.9295e-04 - val_loss: 5.6849e-04
Epoch 269/350

31/31  0s 3ms/step - loss: 6.7890e-04 - val_loss: 5.7241e-04
Epoch 270/350

31/31  0s 4ms/step - loss: 6.5882e-04 - val_loss: 5.6973e-04
Epoch 271/350

31/31  0s 4ms/step - loss: 6.8025e-04 - val_loss: 5.7501e-04
Epoch 272/350

31/31  0s 4ms/step - loss: 6.9398e-04 - val_loss: 5.7323e-04
Epoch 273/350

31/31  0s 3ms/step - loss: 6.7430e-04 - val_loss: 5.7056e-04
Epoch 274/350

31/31  0s 4ms/step - loss: 6.5795e-04 - val_loss: 5.6806e-04
Epoch 275/350

31/31  0s 4ms/step - loss: 6.8659e-04 - val_loss: 5.7478e-04
Epoch 276/350

31/31  0s 4ms/step - loss: 6.4343e-04 - val_loss: 5.6935e-04
Epoch 277/350

31/31  0s 4ms/step - loss: 6.3373e-04 - val_loss: 5.7445e-04
Epoch 278/350

31/31  0s 4ms/step - loss: 7.3480e-04 - val_loss: 5.7532e-04
Epoch 279/350

31/31  0s 4ms/step - loss: 7.5003e-04 - val_loss: 5.7371e-04
Epoch 280/350

31/31  0s 4ms/step - loss: 7.1944e-04 - val_loss: 5.7226e-04
Epoch 281/350

31/31  0s 4ms/step - loss: 6.6910e-04 - val_loss: 5.7201e-04
Epoch 282/350

31/31  0s 4ms/step - loss: 6.4400e-04 - val_loss: 5.7376e-04
Epoch 283/350

31/31  0s 3ms/step - loss: 6.4126e-04 - val_loss: 5.7532e-04
Epoch 284/350

31/31  0s 4ms/step - loss: 6.5850e-04 - val_loss: 5.7647e-04
Epoch 285/350

31/31  0s 4ms/step - loss: 7.3727e-04 - val_loss: 5.7140e-04
Epoch 286/350

31/31  0s 3ms/step - loss: 7.0063e-04 - val_loss: 5.7145e-04
Epoch 287/350

31/31  0s 4ms/step - loss: 6.8961e-04 - val_loss: 5.7049e-04
Epoch 288/350

31/31  0s 4ms/step - loss: 6.5630e-04 - val_loss: 5.7349e-04
Epoch 289/350

31/31  0s 4ms/step - loss: 6.7309e-04 - val_loss: 5.7201e-04
Epoch 290/350

31/31  0s 3ms/step - loss: 6.4513e-04 - val_loss: 5.7040e-04
Epoch 291/350

31/31  0s 3ms/step - loss: 6.8965e-04 - val_loss: 5.7257e-04
Epoch 292/350

31/31  0s 4ms/step - loss: 7.5192e-04 - val_loss: 5.7336e-04
Epoch 293/350

31/31  0s 4ms/step - loss: 6.5996e-04 - val_loss: 5.6850e-04
Epoch 294/350

31/31  0s 4ms/step - loss: 6.4314e-04 - val_loss: 5.7440e-04
Epoch 295/350

31/31  0s 4ms/step - loss: 6.5353e-04 - val_loss: 5.7100e-04
Epoch 296/350

31/31  0s 4ms/step - loss: 7.2492e-04 - val_loss: 5.7014e-04
Epoch 297/350

31/31  0s 5ms/step - loss: 7.4983e-04 - val_loss: 5.6864e-04
Epoch 298/350

31/31  0s 4ms/step - loss: 6.9108e-04 - val_loss: 5.6596e-04
Epoch 299/350

31/31  0s 4ms/step - loss: 6.5284e-04 - val_loss: 5.6752e-04
Epoch 300/350

31/31  0s 4ms/step - loss: 7.1888e-04 - val_loss: 5.7256e-04
Epoch 301/350

31/31  0s 4ms/step - loss: 7.5145e-04 - val_loss: 5.6867e-04
Epoch 302/350

31/31  0s 4ms/step - loss: 6.9928e-04 - val_loss: 5.6738e-04
Epoch 303/350

31/31  0s 4ms/step - loss: 7.2237e-04 - val_loss: 5.7326e-04
Epoch 304/350

31/31  0s 4ms/step - loss: 6.4127e-04 - val_loss: 5.7441e-04
Epoch 305/350

31/31  0s 4ms/step - loss: 6.5799e-04 - val_loss: 5.7343e-04
Epoch 306/350

31/31  0s 4ms/step - loss: 6.7468e-04 - val_loss: 5.7567e-04
Epoch 307/350

31/31  0s 4ms/step - loss: 6.7632e-04 - val_loss: 5.7237e-04
Epoch 308/350

31/31  0s 4ms/step - loss: 6.9157e-04 - val_loss: 5.7145e-04
Epoch 309/350

31/31  0s 4ms/step - loss: 6.7008e-04 - val_loss: 5.7113e-04
Epoch 310/350

31/31  0s 3ms/step - loss: 6.9951e-04 - val_loss: 5.6989e-04
Epoch 311/350

31/31  0s 4ms/step - loss: 6.4539e-04 - val_loss: 5.6870e-04
Epoch 312/350

31/31  0s 4ms/step - loss: 6.6928e-04 - val_loss: 5.7360e-04
Epoch 313/350

31/31  0s 4ms/step - loss: 6.7048e-04 - val_loss: 5.7327e-04
Epoch 314/350

31/31  0s 4ms/step - loss: 6.4485e-04 - val_loss: 5.7562e-04
Epoch 315/350

31/31  0s 4ms/step - loss: 6.9467e-04 - val_loss: 5.7438e-04
Epoch 316/350

31/31  0s 3ms/step - loss: 6.4320e-04 - val_loss: 5.7521e-04
Epoch 317/350

31/31  0s 4ms/step - loss: 6.7703e-04 - val_loss: 5.6928e-04
Epoch 318/350

31/31  0s 4ms/step - loss: 7.2475e-04 - val_loss: 5.7233e-04
Epoch 319/350

31/31  0s 4ms/step - loss: 6.8130e-04 - val_loss: 5.7276e-04
Epoch 320/350

31/31  0s 4ms/step - loss: 6.8497e-04 - val_loss: 5.6880e-04
Epoch 321/350

31/31  0s 4ms/step - loss: 6.8441e-04 - val_loss: 5.7140e-04
Epoch 322/350

31/31  0s 4ms/step - loss: 6.7678e-04 - val_loss: 5.7315e-04
Epoch 323/350

31/31  0s 4ms/step - loss: 6.4901e-04 - val_loss: 5.6909e-04
Epoch 324/350

31/31  0s 3ms/step - loss: 7.1097e-04 - val_loss: 5.7130e-04
Epoch 325/350

31/31  0s 4ms/step - loss: 7.2447e-04 - val_loss: 5.6986e-04
Epoch 326/350

31/31  0s 4ms/step - loss: 6.6603e-04 - val_loss: 5.6759e-04
Epoch 327/350

31/31  0s 4ms/step - loss: 7.2407e-04 - val_loss: 5.7266e-04
Epoch 328/350

31/31  0s 4ms/step - loss: 6.7521e-04 - val_loss: 5.6604e-04
Epoch 329/350

```

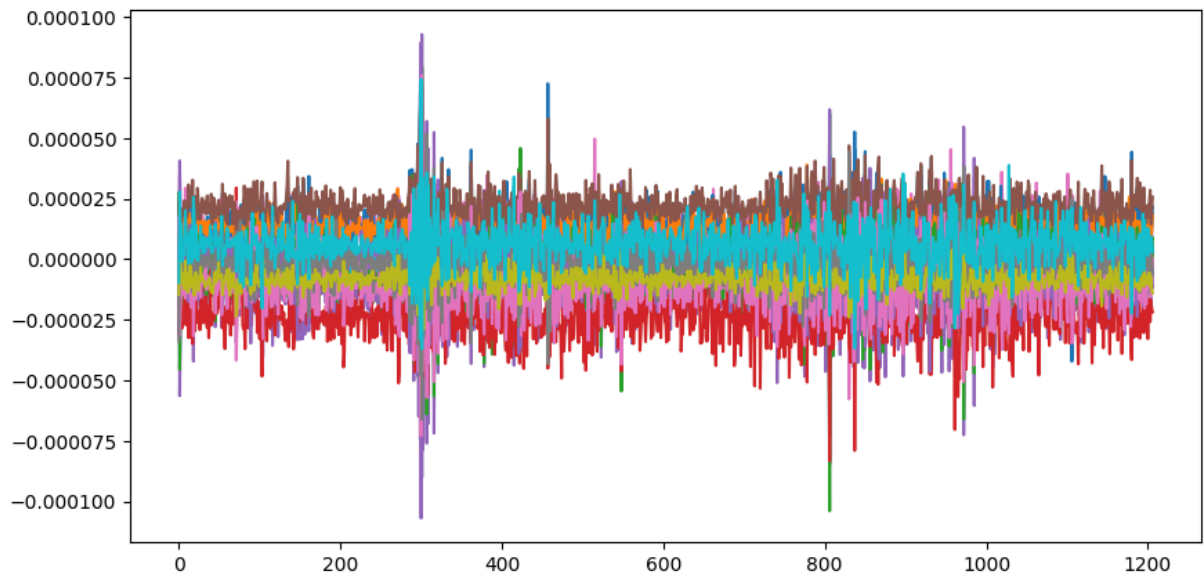
31/31  0s 4ms/step - loss: 6.7511e-04 - val_loss:
5.6804e-04
Epoch 330/350
31/31  0s 4ms/step - loss: 7.0720e-04 - val_loss:
5.7600e-04
Epoch 331/350
31/31  0s 4ms/step - loss: 6.2524e-04 - val_loss:
5.7016e-04
Epoch 332/350
31/31  0s 4ms/step - loss: 7.1132e-04 - val_loss:
5.7241e-04
Epoch 333/350
31/31  0s 3ms/step - loss: 6.7122e-04 - val_loss:
5.7555e-04
Epoch 334/350
31/31  0s 4ms/step - loss: 7.1109e-04 - val_loss:
5.6844e-04
Epoch 335/350
31/31  0s 4ms/step - loss: 6.3204e-04 - val_loss:
5.6854e-04
Epoch 336/350
31/31  0s 4ms/step - loss: 6.2831e-04 - val_loss:
5.7063e-04
Epoch 337/350
31/31  0s 4ms/step - loss: 6.6687e-04 - val_loss:
5.6860e-04
Epoch 338/350
31/31  0s 4ms/step - loss: 6.4531e-04 - val_loss:
5.6928e-04
Epoch 339/350
31/31  0s 4ms/step - loss: 6.7892e-04 - val_loss:
5.6799e-04
Epoch 340/350
31/31  0s 6ms/step - loss: 7.2527e-04 - val_loss:
5.6921e-04
Epoch 341/350
31/31  0s 5ms/step - loss: 6.7714e-04 - val_loss:
5.7220e-04
Epoch 342/350
31/31  0s 5ms/step - loss: 6.9821e-04 - val_loss:
5.6902e-04
Epoch 343/350
31/31  0s 4ms/step - loss: 7.7455e-04 - val_loss:
5.7205e-04
Epoch 344/350
31/31  0s 4ms/step - loss: 6.4468e-04 - val_loss:
5.7150e-04
Epoch 345/350
31/31  0s 4ms/step - loss: 6.8735e-04 - val_loss:
5.6835e-04
Epoch 346/350
31/31  0s 4ms/step - loss: 6.4893e-04 - val_loss:
5.6746e-04
Epoch 347/350
31/31  0s 4ms/step - loss: 6.7634e-04 - val_loss:
5.6932e-04
Epoch 348/350
31/31  0s 4ms/step - loss: 6.8348e-04 - val_loss:
5.6930e-04

```

Out[8]: <keras.src.callbacks.history.History at 0x7f9295627e90>

By plotting the time series of the factors generated by the model, we can observe how the regularization term shrinks the dimensionality of the bottleneck. The model is forced to generate a number of latent factors with lower dimensionality than the data and that are relevant to explain its variation.

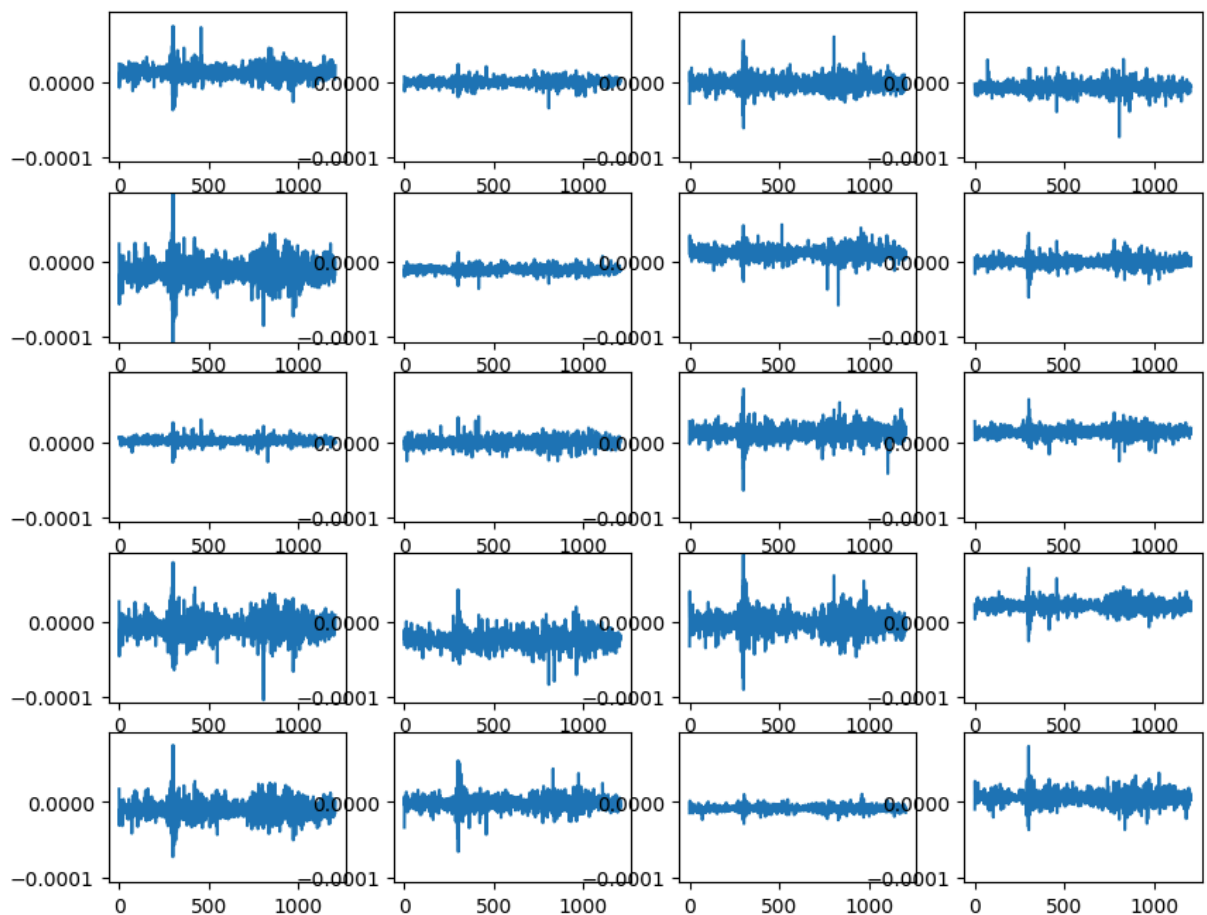
```
In [9]: factors = model.encoder(Y_train).numpy()
plt.figure(figsize=(10, 5))
plt.plot(factors)
plt.show()
```



Below, we plot the time series of each factor. Observe how, out of 20 possible dimensions, the addition of the regularization term enables us to "drop" some dimensions. Notice the resemblance of this regularization technique with the L1-regularization of weights in MLPs. In that setup, we were interested in filtering a set of input features that are relevant for predictions. In the autoencoder setup, the activity regularization chooses a few factors that are relevant at explaining the variation in the data.

You can play with the parameter λ_1 above and observe its impact on the number of factors generated and the performance of the model. Notice also that the results depend on the random seed that is used to initialize the model's parameters while training.

```
In [10]: custom_ylim = [np.min(factors), np.max(factors)]
fig, axs = plt.subplots(5, 4, figsize=(10, 8))
for i in range(5):
    axs[i, 0].plot(factors[:, 0 + i * 4])
    axs[i, 0].set_ylim(custom_ylim)
    axs[i, 1].plot(factors[:, 1 + i * 4])
    axs[i, 1].set_ylim(custom_ylim)
    axs[i, 2].plot(factors[:, 2 + i * 4])
    axs[i, 2].set_ylim(custom_ylim)
    axs[i, 3].plot(factors[:, 3 + i * 4])
    axs[i, 3].set_ylim(custom_ylim)
```



In terms of reconstruction performance, the regularization parameter λ_1 is quite small, so the reconstruction errors from this model should be below those in Lesson 2 using an undercomplete autoencoder with three factors. This is a natural outcome because increasing the number of latent factors should increase the proportion of the variance explained by the model.

```
In [11]: Y_pred = model.predict(Y_train, verbose=0)
error = np.sum((Y_train - Y_pred) ** 2) / np.sum((Y_train -
np.mean(Y_train)) ** 2)
print("Reconstruction error in training sample", error)
Y_pred = model.predict(Y_test, verbose=0)
error_oos = np.sum((Y_test - Y_pred) ** 2) / np.sum((Y_test -
np.mean(Y_test)) ** 2)
print("Reconstruction error in test sample", error_oos)
```

Reconstruction error in training sample 0.9996295625952354
Reconstruction error in test sample 1.0017582451379592

3. PORTFOLIO CHOICE USING THE PREDICTIONS OF THE AUTOENCODER

Now we are going to use the trained autoencoder to choose an "optimal" portfolio of stocks. Based on the estimated exposures of each stock to the factors that determine the comovement in stock returns, we can design a diversified portfolio that maximizes the expected risk-weighted returns. Let's formally describe this.

Notice that the decoding layer of the autoencoder implies that we can model the returns of a stock i at time t as:

$$r_{it} = \alpha_i + \beta_{i1}f_{1t} + \beta_{i2}f_{2t} + \cdots + \beta_{ik}f_{kt} + \varepsilon_{it}$$

where $f_{1t}, f_{2t}, \dots, f_{kt}$ are the factors generated by the autoencoder, and $\alpha_i, \beta_{i1}, \beta_{i2}, \dots, \beta_{ik}$ are the trained parameters. ε_{it} are zero-mean error terms. The representation above is called a linear factor model.

If we stack the observations of returns of all the stocks $i = 1, \dots, n$ at a period t in a column vector R_t , stock returns are given by:

$$R_t = \mathbf{a} + BF_t + \mathbf{e}_t$$

where (bold faces are meant to differentiate vectors from scalars):

- $\mathbf{a}$ is an $n \times 1$ vector that includes all the trained α_i 's
- B is an $n \times k$ matrix that includes all the trained β_{ij} 's.
- F_t is a $k \times 1$ vector of realizations of the factors.
- $\mathbf{e}_t$ is an $n \times 1$ vector of idiosyncratic (stock-specific) errors ε_{it} 's

Assume that the variance of $\mathbf{e}_t$ is a diagonal $n \times n$ matrix Σ_e , while the variance of F_t is denoted by the $k \times k$ matrix Σ_f . These matrices capture the variation and covariation of the factors and idiosyncratic terms over time and that we can estimate, for instance, from our autoencoder model. Thus, the expectation and variance of the return vector R_t are given by

$$\begin{aligned} E(R_t) &= \mu = \mathbf{a} + BE(F_t) \\ Var(R_t) &= \Sigma = B\Sigma_f B^T + \Sigma_e \end{aligned}$$

The first component of the variance of returns is called the systematic component of the variance, which arises from the common latent factors that determine all the returns, depending on the stock-specific exposures to those factors included in B . The second component is the idiosyncratic component of the variance. This variation can be interpreted as noise, which is assumed to be stock-specific (uncorrelated across stocks).

Using the autoencoder, we can build our estimates for μ and Σ . For μ , we take the average of the reconstruction of each stock return over the training sample, $E(\hat{R}_t)$. For Σ , we first compute the variance covariance matrix of the reconstructions, $\Sigma_f = Var(\hat{R}_t)$, and then we add the idiosyncratic component from the variance-covariance matrix of the reconstruction errors, $\Sigma_e = diag(Var(\mathbf{e}_t))$. Following our assumption above, we impose a diagonal Σ_e . Although the matrix $Var(\mathbf{e}_t)$ may not be diagonal, we treat the remaining off-diagonal variation just as "noise" that is irrelevant for generalization.

```
In [12]: Y_pred = model.predict(Y_train, verbose=0)
mu = np.mean(Y_pred, axis=0)
sigma_f = np.cov(Y_pred, rowvar=False)
sigma_e = np.cov(Y_train - Y_pred, rowvar=False)
sigma_d = np.diag(np.diag(sigma_e))
sigma = sigma_f + sigma_d
```

Let's now consider building a portfolio with the modelization approach that we have derived above. A portfolio is defined as a vector $w = (w_1, \dots, w_n)^T$ where each w_i represents the proportion of the investor's wealth that we invest in an asset i . We must impose the constraint $\sum_i^n w_i = 1$, meaning that the investor has all of its wealth invested in the n assets. This does not rule out the possibility of short-selling, $w_i < 0$, so that the portfolio can take long positions on some assets and short positions on others.

Given a choice of portfolio w on the n assets, the return of the portfolio in period t is given by $R_{w,t} = w^T R_t$. Hence, the expected return and variance of this portfolio can be denoted by the following scalars:

$$E(R_{w,t}) = w^T \mu$$

$$Var(R_{w,t}) = w^T \Sigma w$$

Let's define the Sharpe Ratio, SR , of a portfolio as its expected return per unit of risk, i.e.:

$$SR_w = \frac{E(R_{w,t})}{[Var(R_{w,t})]^{1/2}} = \frac{w^T \mu}{(w^T \Sigma w)^{1/2}}$$

Note: Formally, to compute a Sharpe Ratio we should subtract the risk-free rate from the numerator, but we are going to abstract from it. You can read the [Investopedia entry](#) as further reference.

We are going to use the Sharpe Ratio as the performance metric that determines whether a portfolio is better than other. Thus, our objective is to find the portfolio that generates the maximum Sharpe Ratio given our estimates of expected returns and variance covariance matrix of returns.

For this, we set up an optimization problem, where the objective function is the Sharpe Ratio, given vectors of weights, expected returns, and the variance-covariance matrix. We put a minus term in front as we are going to define the optimization problem as a minimization objective.

```
In [13]: def sharpe_ratio(weights, mean, var):
          return -np.dot(weights, mean) / np.sqrt(np.dot(np.dot(weights, var),
          weights))
```

We then define a function where the optimization problem is performed, including the constraint that $\sum_i^n w_i = 1$. If we were interested in imposing a

long-only portfolio (short-selling constraint) then we should change the "bound" tuple to take the value $(0, 1)$, instead of $(None, None)$.

```
In [14]: import scipy.optimize as sco
```

```
In [15]: def max_sharpe(mean, var):
    args = (mean, var)
    num_assets = len(mean)
    args = (mean, var)
    constraints = {"type": "eq", "fun": lambda x: np.sum(x) - 1}
    bound = (None, None)
    bounds = tuple(bound for asset in range(num_assets))
    result = sco.minimize(
        sharpe_ratio,
        num_assets
        * [
            1.0 / num_assets,
        ],
        args=args,
        method="SLSQP",
        bounds=bounds,
        constraints=constraints,
        tol=1e-12,
    )
    return result
```

Given, the estimates of μ and Σ from the autoencoder, we get the following portfolio weights for the 20 stocks. We also verify that the constraint is roughly satisfied.

```
In [16]: opt = max_sharpe(mu, sigma)
opt.x
```

```
Out[16]: array([ 0.13625731,  0.09874778,  0.0523024 ,  0.03819203,  0.06961968,
                  0.00350657,  0.05610333,  0.05169225, -0.00699324, -0.01166656,
                  0.07362228,  0.06228513,  0.07470726,  0.03380913,  0.04850011,
                  0.03878507,  0.02906145,  0.07333414,  0.05462749,  0.02350638])
```

```
In [17]: np.sum(opt.x)
```

```
Out[17]: 1.0
```

Actually, from portfolio choice theory, we already know the analytical expression for the portfolio with maximum Sharpe Ratio in the absence of short-selling constraints:

$$w^* = \frac{\Sigma^{-1}\mu}{\iota_n^T \Sigma^{-1}\mu}$$

where ι_n is an $n \times 1$ vector of ones. See this [link](#) for a proof of this result. Below, we show that the results from the optimization problem are roughly identical (reduce the tolerance of the optimization problem to obtain closer results).

```
In [18]: # Analytic solution (only when short positions are allowed)
weights = np.dot(np.linalg.inv(sigma), mu)
weights = weights / np.sum(weights)
weights
```

```
Out[18]: array([ 0.13625741,  0.09874773,  0.05230228,  0.03819205,  0.06961961,
                 0.00350667,  0.05610339,  0.05169231, -0.00699334, -0.01166649,
                 0.07362227,  0.06228522,  0.07470719,  0.03380929,  0.04850003,
                 0.03878504,  0.02906152,  0.0733341 ,  0.05462746,  0.02350625])
```

Now, let's look at the performance of the portfolio in the training sample and in the test sample. Note that the results may vary depending on the seed used while training. You can train the model several times to have a general view on the out-of-sample performance of the strategy.

```
In [19]: mu_true_test = np.mean(Y_test, axis=0)
sigma_true_test = np.cov(Y_test, rowvar=False)
print(
    "Sharpe ratio in training (estimated): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu, sigma),
)
print(
    "Sharpe ratio in test (actual): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true_test, sigma_true_test),
)
```

```
Sharpe ratio in training (estimated):  3.3954331835003013
Sharpe ratio in test (actual):  2.07543332497095
```

Does the portfolio obtained from the autoencoder perform better than a "naive" portfolio that simply uses the mean returns and variance covariance matrix of returns over the training sample to construct the portfolio? Below, we compare the autoencoder model with that benchmark. All rights reserved WQU
WorldQuant University QQQQ

```
In [20]: mu_true = np.mean(Y_train, axis=0)
sigma_true = np.cov(Y_train, rowvar=False)
opt = max_sharpe(mu_true, sigma_true)
print(
    "Sharpe ratio in training: ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true, sigma_true),
)
print(
    "Sharpe ratio in test (actual): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true_test, sigma_true_test),
)
```

```
Sharpe ratio in training:  1.9655875931884363
Sharpe ratio in test (actual):  1.6941130374559856
```

Below, we show the results from choosing the optimal portfolio using estimates from PCA, using as many principal components as those obtained from the autoencoder model. Since this method does not produce estimates of the average returns, we rely on using the principal components (which are calculated from the centered or demeaned data) to calculate the average returns over the training sample as estimates for the expected returns, μ .

Notice that the standard PCA method does not use a validation sample to avoid overfitting, which naturally leads to different results than an autoencoder approach. However, the results are independent from the choice of random seed.

```
In [21]: from sklearn.decomposition import PCA

pca = PCA(n_components=17)
pca.fit(Y_train)
pca_factors = pca.transform(Y_train)
Y_pca_pred = np.dot(pca_factors, pca.components_)
sigma_f = np.cov(Y_pca_pred, rowvar=False)
sigma_e = np.cov(Y_train - Y_pca_pred, rowvar=False)
sigma_d = np.diag(np.diag(sigma_e))
sigma_pca = sigma_f + sigma_d
```

Below we compute the Sharpe ratios of the portfolios built using PCA. The estimated Sharpe ratio from the PCA is 2.02 (in training), but in the test sample, it drops to -0.31. Notice that the optimal portfolio that arises from PCA will give greater weight to stocks with higher average returns over the training sample. Hence, the Sharpe ratio turns negative because those stocks that performed well in the training sample seem to have performed badly in the test sample.

```
In [22]: opt = max_sharpe(mu_true, sigma_pca)
print(opt.x)
print(
    "Sharpe ratio in training (estimated): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true, sigma_pca),
)
print(
    "Sharpe ratio in test (actual): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true_test, sigma_true_test),
)

[ 1.45251094  1.02761215  2.5769373  -1.79584562  0.87488948  0.49026512
  1.87165622 -0.07636212 -0.06377933 -1.14006306  0.21441525  3.11683583
  2.1610222  -2.3176693  -1.57076272 -0.84432432 -0.21190979 -3.12019528
 -0.83857693 -0.80665601]
Sharpe ratio in training (estimated):  2.016511478146714
Sharpe ratio in test (actual):  2.0072948864942446
```

5. CONCLUSION

Although the autoencoder may not generate extremely satisfactory results, its performance may improve upon other more naïve estimation procedures. The results are also largely dependent on the initialization of the autoencoder's weights. In the next module, we'll deal with ensemble methods that, at least partly, deal with this annoying variability.

Notice also that the train and test samples display very different market environments. It is quite challenging to find machine learning algorithms that adapt well to such changes in the environment.

Lastly, as you have probably grasped in the previous modules of this course, obtaining a good deep learning model for finance applications requires a great deal of tweaking and understanding of how the model is a suitable tool for the kind of data that we have at hand.

With this notebook, we finalize the descriptions of autoencoders and their application to financial time series.

In Module 7, we will study further advanced topics in Deep Learning for Finance.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.